



深度学习II: Transformer

Deep Learning II: Transformer



授课对象: 计算机科学与技术专业 二年级

课程名称: 人工智能 (专业必修)

课程学分: 3学分





什么是自然语言处理(NLP)

- **自然语言处理技术（NLP）是文本生成的基础。** NLP探索计算机和人类（自然）语言之间相互作用，研究实现人与计算机之间用自然语言进行有效通信的各种理论和方法。最早的自然语言处理研究工作是机器翻译，后逐渐向文本摘要、分类、校对、信息抽取、语音合成、语音识别等方面深入
- 从基于规则的经验主义到基于统计的理性主义，再到基于深度学习的方法，NLP在70年历程中逐渐发展进步。受益于预训练语言模型的突破发展，Transformer等底层架构不断精进，NLP取得跨越式提升

①释义：

自然语言处理

用计算机对字、词、句、篇章等自然语言的输入、输出、识别、分析、理解、生成等的操作和加工，实现人机间的信息交流

自然语言

人类社会约定俗成的，区别于如程序设计的人工语言

处理

输入、输出、识别、分析、理解、生成等计算机操作过程

②构成：

自然语言处理

自然语言理解

让机器具备正常人的语言理解能力（识别人讲的话）

自然语言产生

将非语言格式的数据转换成人类可以理解的语言格式（输出为人讲的话）

自然语言处理的两种解释

1950s-1970s

采用基于规则的方法

✓1950年，“图灵测试”被提出，**自然语言处理思想诞生**
✓认为自然语言处理过程和人类学习认知一门语言类似，NLP停留在**经验主义思潮阶段**
✓只能基于手写规则，处理少量数据

1970s-2000s

采用基于统计的方法

✓从数学统计的角度预测下个词的出现概率，代表模型如N-Gram等，推理过程非常直观，但是推理结果非常受数据集的影响，容易出现数据稀疏（即空值）等问题

2000-至今

采用基于神经网络的方法

✓模型开始像人脑一样学习，2017年以前主要是小模型阶段，2017年Transformer发布之后，模型开始尝试大量数据的训练学习，进入大语言模型阶段，在加入人工干预的反馈基础上，模型效果攀上新的台阶

NLP发展阶段

NLP基础: Word Embedding 词嵌入/词表征

我们如何表示单词的意思?

定义: **含义** (meaning) (韦氏词典)

- 由单词、短语等表示的思想
- 一个人想通过使用文字、符号等来表达的想法
- 在写作、艺术作品等作品中表达的思想

Commonest linguistic way of thinking of meaning:

signifier (symbol) $\Leftrightarrow$ signified (idea or thing)

= denotational semantics

tree $\Leftrightarrow$ {, , , ...}

WordNet

如何在计算机建立可用的词义？

先前NLP的常用解决方案：例如WordNet，包含同义词集（synonym sets）和上位词列表（hypernyms）的同义词库（“is a”关系）

e.g., synonym sets containing “good”:

```
from nltk.corpus import wordnet as wn
poses = { 'n': 'noun', 'v': 'verb', 's': 'adj (s)', 'a': 'adj', 'r': 'adv' }
for synset in wn.synsets("good"):
    print("{}: {}".format(poses[synset.pos()],
        ", ".join([l.name() for l in synset.lemmas()])))
```

```
noun: good
noun: good, goodness
noun: good, goodness
noun: commodity, trade_good, good
adj: good
adj (sat): full, good
adj: good
adj (sat): estimable, good, honorable, respectable
adj (sat): beneficial, good
adj (sat): good
adj (sat): good, just, upright
...
adverb: well, good
adverb: thoroughly, soundly, good
```

e.g., hypernyms of “panda”:

```
from nltk.corpus import wordnet as wn
panda = wn.synset("panda.n.01")
hyper = lambda s: s.hypernyms()
list(panda.closure(hyper))
```

```
[Synset('procyonid.n.01'),
Synset('carnivore.n.01'),
Synset('placental.n.01'),
Synset('mammal.n.01'),
Synset('vertebrate.n.01'),
Synset('chordate.n.01'),
Synset('animal.n.01'),
Synset('organism.n.01'),
Synset('living_thing.n.01'),
Synset('whole.n.02'),
Synset('object.n.01'),
Synset('physical_entity.n.01'),
Synset('entity.n.01')]
```

WordNet

WordNet等语言资源的问题

- 资源可用，但缺少细微差别：
 - 例如，“proficient 精通”被列为“good”的同义词
这只在某些语境中是正确的
- 缺少某些新的含义：
 - 例如，闪电五连鞭, 坤坤, 蕉绿, 甲亢哥, wicked, badass, nifty, wizard, genius, ninja
 - 基本不可能保持最新！
- 主观性
- 需要人工来创建和调整
- 不能用于准确计算单词相似度（后面介绍）

将单词表示为离散符号

自然语言处理系统把单词作为离散的原子符号

- 'cat' is encoded as Id537
- 'dog' is encoded as Id143
- ...

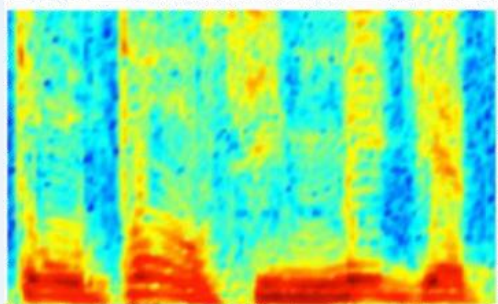
Items in a dictionary ...

A document becomes a Bag of Words

0	0	0	0.2	0	0.7	0	0	0	...	...
---	---	---	-----	---	-----	---	---	---	-----	-----

Sparse and high dimensional -> Curse of Dimensionality!

Word, context, or document vectors



Audio Spectrogram

DENSE



Image pixels

DENSE

SPARSE

将单词表示为离散符号

在传统的 NLP 中，我们将单词视为离散符号：

hotel, conference, motel – 局部主义 (localist) 表示

Means one 1, the rest 0s

这样的单词符号可以用 one-hot 向量来表示：

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

向量维度大小 = 词典中单词的数量(比如说500,000+)

将单词表示为离散符号

会有什么问题？

例子：在网络搜索中，如果用户搜索“Seattle motel”，需要匹配包含“Seattle hotel”的文档
但是：

motel = [0 0 0 0 0 0 0 0 0 0 1 0 0 0 0]

hotel = [0 0 0 0 0 0 0 1 0 0 0 0 0 0 0]

这两个向量是**正交的**，也就没有了它们在自然概念上的**相似性(similarity)!**

解决方案：

- 可否尝试依靠 WordNet 的同义词列表来获得相似性？
 - 但已知因为意义不完整等问题，似乎不可行
- 反之：学习对向量本身的相似性进行编码

利用上下文语境表示单词

分布语义学: 单词的含义由经常出现在附近的单词给出

- “*You shall know a word by the company it keeps*” (J. R. Firth 1957: 11)
- 是现代统计 NLP 最成功的想法之一!
- 当单词 w 出现在文本中时, 其**上下文 (context)** 是出现在附近的单词集 (在一个固定大小的窗口内).
- 我们利用 w 的丰富上下文来构建 w 的表示

...government debt problems turning into **banking** crises as happened in 2009...
...saying that Europe needs unified **banking** regulation to replace the hodgepodge...
...India has just given its **banking** system a shot in the arm...

These context words will represent **banking**

词向量

简短但密集的实数向量，大概50-300维

为每个单词构建一个密集向量，使其类似于出现在相似上下文中的单词向量，使用点(数量)积衡量相似性

banking =

$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \end{pmatrix}$$

monetary =

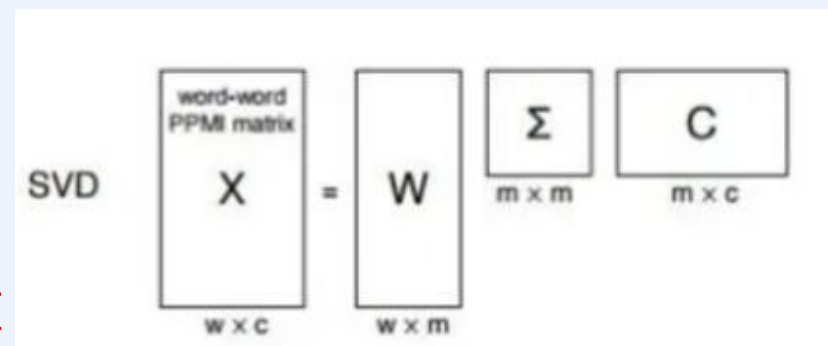
$$\begin{pmatrix} 0.413 \\ 0.582 \\ -0.007 \\ 0.247 \\ 0.216 \\ -0.718 \\ 0.147 \\ 0.051 \end{pmatrix}$$

注意: 词向量也叫(词)嵌入或(自然)词表征
它们是分布式的表征

词向量

为什么要用**简短而密集**的实数向量？

- 短向量更容易作为ML系统的特征使用
- 密集向量可能相比存储显式的计数更泛用
- 更容易捕捉到同义：
 - w_1 同现为“car”， w_2 同现为“automobile”
- 获得密集向量的不同方法：
 - 奇异值分解（SVD）
 - **Word2Vec及其相关方法：“学习”这样的向量**



词嵌入的SVD：共现作为特征

- 行向量描述单词在文本语料库中的使用情况
- 可以看作是在n维欧氏空间中的点的坐标

太高维了！

	get	see	use	hear	eat	kill
knife	51	20	84	0	3	0
cat	52	58	4	4	6	26
dog	115	83	10	42	33	17
boat	59	39	23	4	0	0
cup	98	14	6	2	1	0
pig	12	17	3	2	9	27
banana	11	2	2	0	18	0

同现矩阵 (Co-occurrence Matrix)

词嵌入的SVD：共现作为特征

SVD用于降维

$$\begin{array}{ccccc}
 \begin{array}{c} m \\ \boxed{} \\ n \end{array} & = & \begin{array}{c} r \\ \boxed{\begin{array}{c} | | | | \cdots \\ U_1 U_2 U_3 \cdots \end{array}} \\ n \end{array} & \begin{array}{c} r \\ \boxed{\begin{array}{c} S_1 \cdots 0 \\ 0 \cdots S_r \end{array}} \\ r \end{array} & \begin{array}{c} m \\ \boxed{\begin{array}{c} \text{---} V_1 \text{---} \\ \text{---} V_2 \text{---} \\ \text{---} V_3 \text{---} \\ \vdots \end{array}} \\ r \end{array} \\
 X & & U & S & V^T \\
 \\
 \begin{array}{c} m \\ \boxed{\phantom{\hat{X}}} \\ n \end{array} & = & \begin{array}{c} k \\ \boxed{\begin{array}{c} | | | | \cdots \\ U_1 U_2 U_3 \cdots \end{array}} \\ n \end{array} & \begin{array}{c} k \\ \boxed{\begin{array}{c} S_1 \cdots 0 \\ 0 \cdots S_k \end{array}} \\ k \end{array} & \begin{array}{c} m \\ \boxed{\begin{array}{c} \text{---} V_1 \text{---} \\ \text{---} V_2 \text{---} \\ \text{---} V_3 \text{---} \\ \vdots \end{array}} \\ k \end{array} \\
 \hat{X} & & \hat{U} & \hat{S} & \hat{V}^T
 \end{array}$$

词嵌入的SVD：问题

SVD的问题：

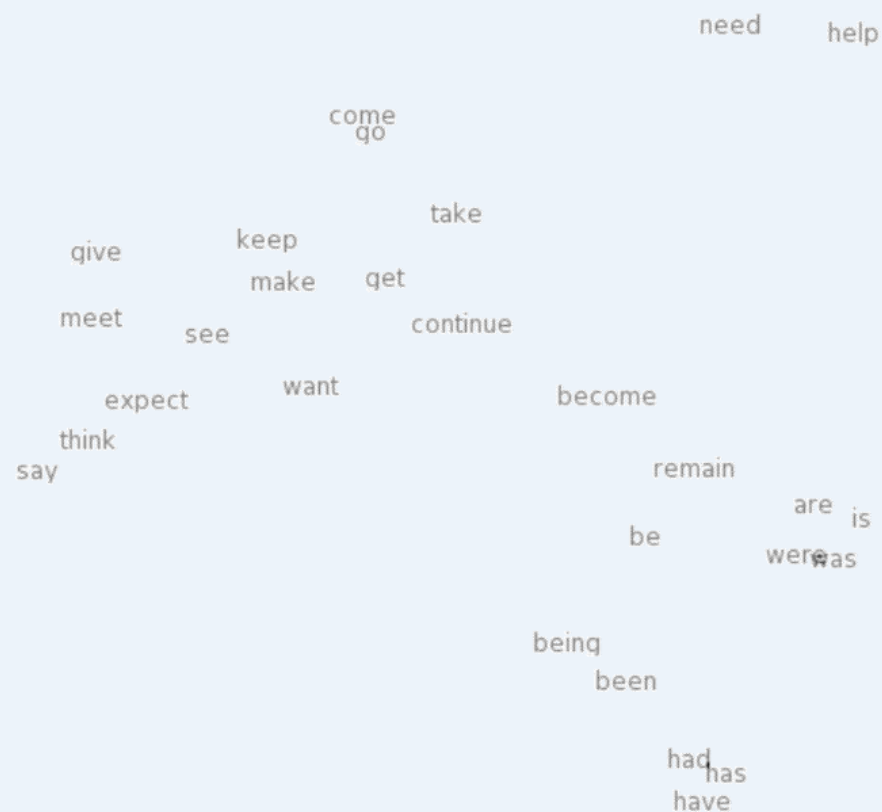
在一个 $d \times n$ 矩阵上的计算开销是 $O(dn^2)$ ，其中 $d < n$

- 使其几乎不可能用在有大量单词的词典或文档中
- 难以合并样本外的新单词或文档

神经化词向量

使用神经化词向量表示词义 – 可视化

expect =

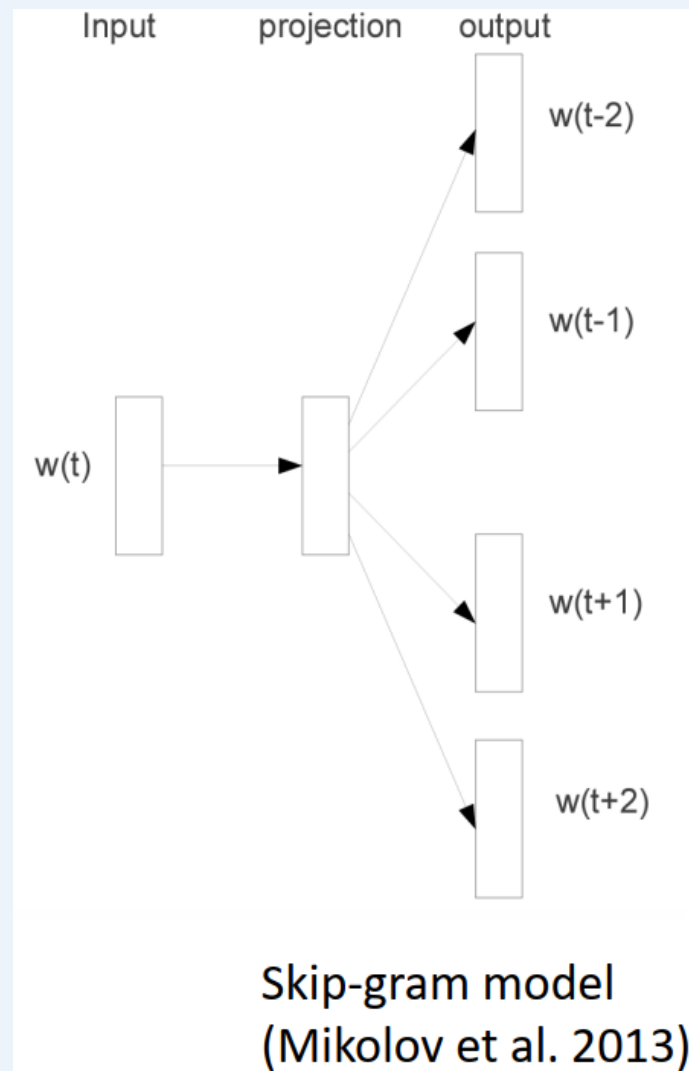
$$\begin{pmatrix} 0.286 \\ 0.792 \\ -0.177 \\ -0.107 \\ 0.109 \\ -0.542 \\ 0.349 \\ 0.271 \\ 0.487 \end{pmatrix}$$


Word2Vec概述

Word2vec是一个学习词向量的框架(Mikolov et al. 2013)

思想:

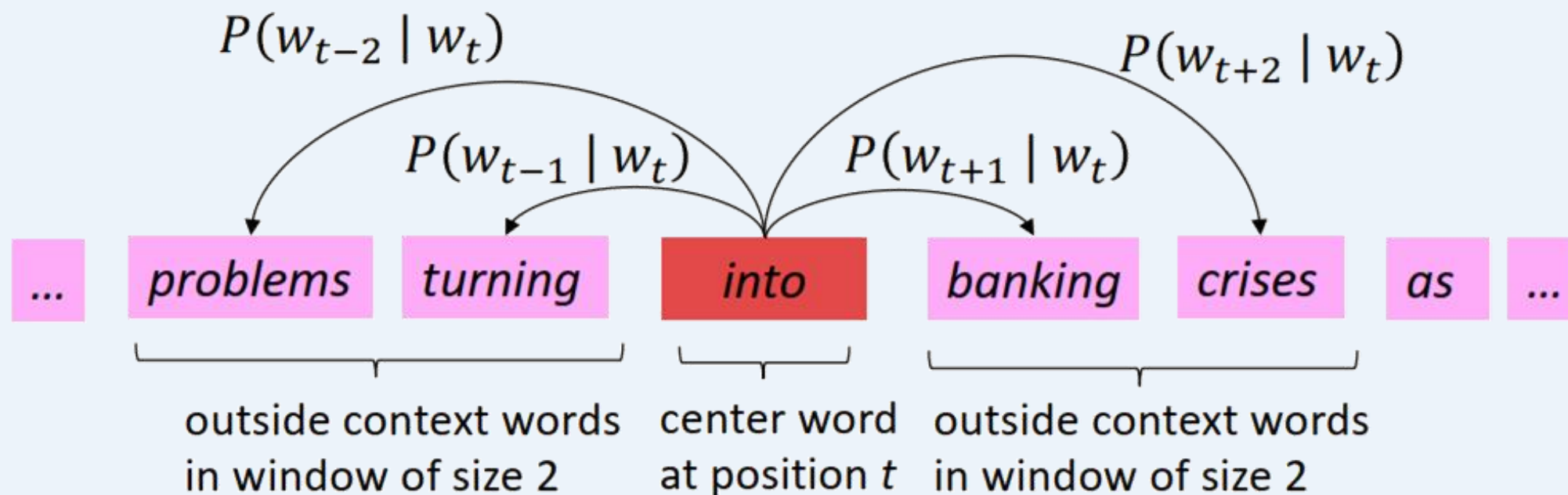
- 我们有一个很大的文本语料库(“正文”)：一长串单词
- 固定词汇表中的每个单词都由一个vector表示
- 遍历文本中的每个位置 t ，其中包含一个中心词 c 和上下文(“外部”)词 o
- 使用 c 和 o 的词向量相似性来计算给定 c 时， o 的概率(反之亦然)
- 不断调整单词向量来最大化此概率



[Mikolov et al. 2013: “Distributed Representations of Words and Phrases and their Compositionality”]

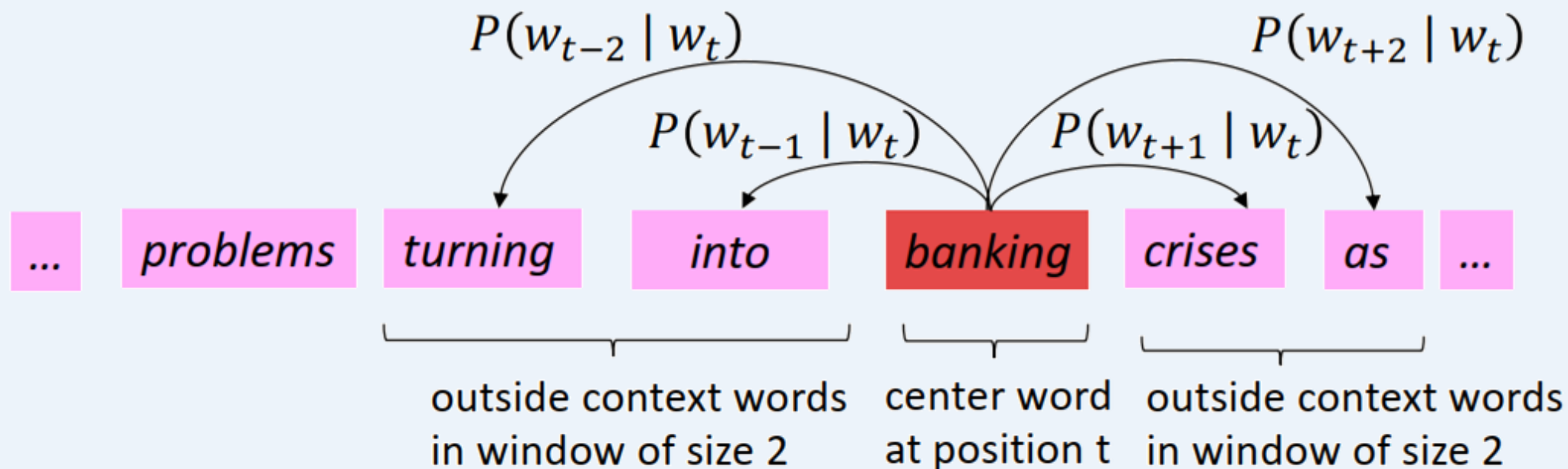
Word2Vec概述

以一个窗口为例，计算 $P(w_{t+j} | w_t)$ 的过程：



Word2Vec概述

以一个窗口为例，计算 $P(w_{t+j}|w_t)$ 的过程：



Word2Vec目标函数

对每个位置 $t = 1, \dots, T$, 在固定大小为 m 的窗口内预测上下文的单词。
给定中心词 w_t , 数据的似然:

$$\text{Likelihood} = L(\theta) = \prod_{t=1}^T \prod_{\substack{-m \leq j \leq m \\ j \neq 0}} P(w_{t+j} | w_t; \theta)$$

θ is all variables
to be optimized

sometimes called a *cost* or *loss* function

目标函数 $J(\theta)$ 为似然负数的均值

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

最小化目标函数 $\Leftrightarrow$ 最大化预测准确率

Word2Vec目标函数

- 我们要最小化下面的目标函数：

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{\substack{-m \leq j \leq m \\ j \neq 0}} \log P(w_{t+j} | w_t; \theta)$$

- 问题：如何计算 $P(w_{t+j} | w_t; \theta)$
- 答案：对每个单词 w ，使用两个向量：

- v_w ：当 w 为中心词时
- u_w ：当 w 为上下文词时

} 这两个向量都是所有参数 θ 确定的更大向量的子部分

- 然后对中心词 c 和上下文单词 o ：

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

Word2Vec预测函数

②指数操作使其为正

①点积操作比较o和c的相似度。
 $u^T v = u \cdot v = \sum_{i=1}^n u_i v_i$.
更大的点积=更大的概率

$$P(o|c) = \frac{\exp(u_o^T v_c)}{\sum_{w \in V} \exp(u_w^T v_c)}$$

③在整个词典上进行归一化, 给出概率分布

- 这是个softmax损失函数的例子($\mathbb{R}^n \rightarrow (0,1)^n$)

$$\text{softmax}(x_i) = \frac{\exp(x_i)}{\sum_{j=1}^n \exp(x_j)} = p_i$$

- 这个softmax函数把任意 x_i 映射为概率分布 p_i
 - "max" 是因为它放大了最大的 x_i 的概率
 - "soft" 是因为它仍然为较小的 x_i 赋予了一些概率
 - 在深度学习中频繁用到

但还是比较奇怪的叫法, 因为它返回的是个分布

Word2Vec其他细节

为什么是两个向量？ → 更易优化，最后两者均取平均值

- 但是可以实现每个单词只有一个向量的算法

两种变体：

1. Skip-grams (SG)

预测给定中心词的上下文（“外部”）词（与位置无关）

2. Continuous Bag of Words (CBOW)

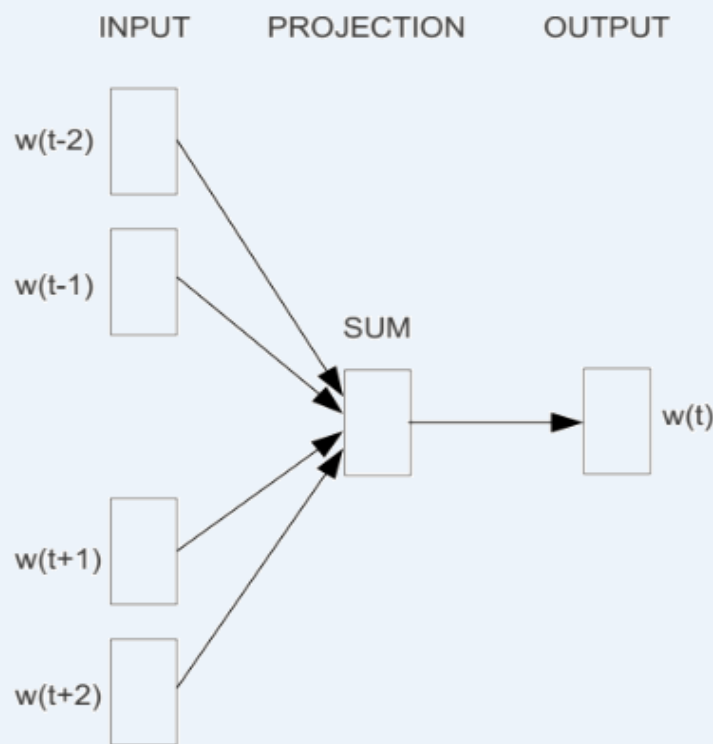
从上下文词（袋）预测中心词

训练损失函数：

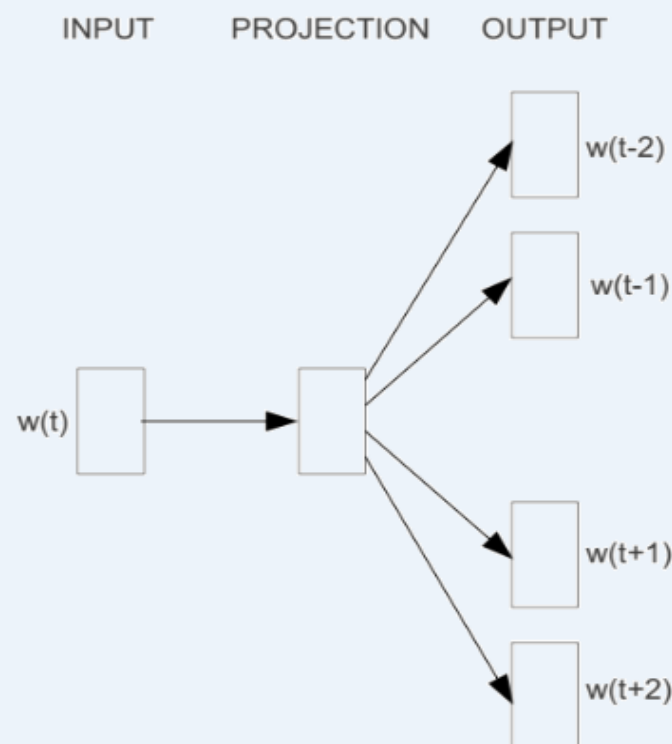
1. 朴素的(naïve)softmax（简单，但输出类别特别多时，计算开销大）
2. 使用如分层 softmax等优化的变体
3. 负采样

到目前为止，我们解释了朴素softmax

Word2Vec: Skip-grams和CBOW



CBOW



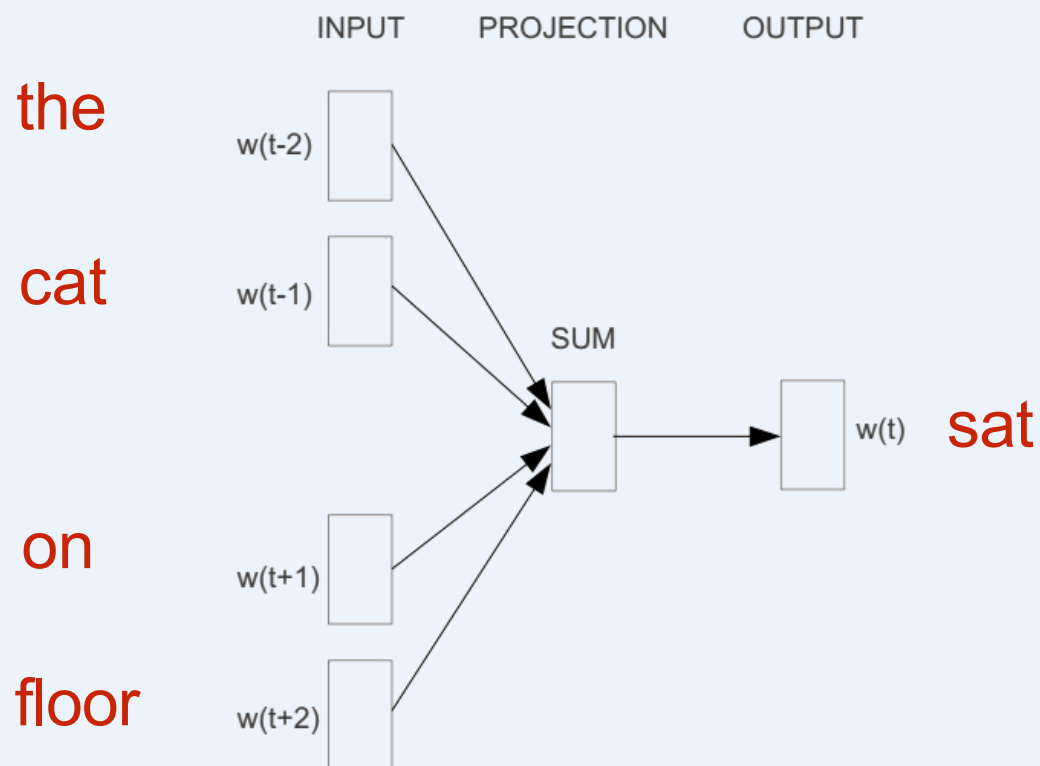
Skip-gram

- Continuous Bag of Words (CBOW): 使用窗口中的上下文单词来预测中心词
- Skip-grams: 使用中心词来预测窗口中的上下文单词

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

例子: “The cat sat on floor” (窗口大小为2)



*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

cat
(one-hot vector)
on

0
1
0
0
0
0
0
0
0
...
0
0
0
0
0
0
0
0
0
0
0
0
0
0
...

Hidden layer

Hidden layer nodes

Output layer

sat (one-hot vector)

0
0
0
0
0
0
0
0
0
1
...
0

*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

cat

on

$\mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}$

0
1
0
0
0
0
0
0
0
...
0
0
0
0
0
0
0
0
0
0
...

$\mathbf{W}_{|\mathcal{V}| \rightarrow |\mathcal{N}|}$

$\mathbf{W}_{|\mathcal{V}| \rightarrow |\mathcal{N}|}$

Hidden layer

$\hat{\mathbf{v}} \in \mathbb{R}^{|\mathcal{N}|}$

Output layer

sat

$\hat{\mathbf{y}} \in \mathbb{R}^{|\mathcal{V}|}$

0
0
0
0
0
0
0
0
0
0
1
...
0

$\mathbf{W}^0_{|\mathcal{N}| \rightarrow |\mathcal{V}|}$

*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

cat
on

0
1
0
0
0
0
0
0
0
...
0
0
0
0
0
0
0
0
0
0
0
0
...

$W_{|V| \rightarrow |N|}$

$W_{|V| \rightarrow |N|}$

Hidden layer

0
0
0
0
0
0
0
0

需要学习的参数

$W^0_{|N| \rightarrow |V|}$

Output layer

0
0
0
0
0
0
0
0
0
0
1
...
0

sat

*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

cat

on

$\mathbf{x} \in \mathbb{R}^{|\mathcal{V}|}$

$\mathbf{W} \mid \mathcal{V} \rightarrow \mathcal{N}$

$\mathbf{W} \mid \mathcal{V} \rightarrow \mathcal{N}$

Hidden layer

$\hat{\mathbf{v}} \in \mathbb{R}^{|\mathcal{N}|}$

Output layer

sat

$\hat{\mathbf{y}} \in \mathbb{R}^{|\mathcal{V}|}$

需要学习的参数

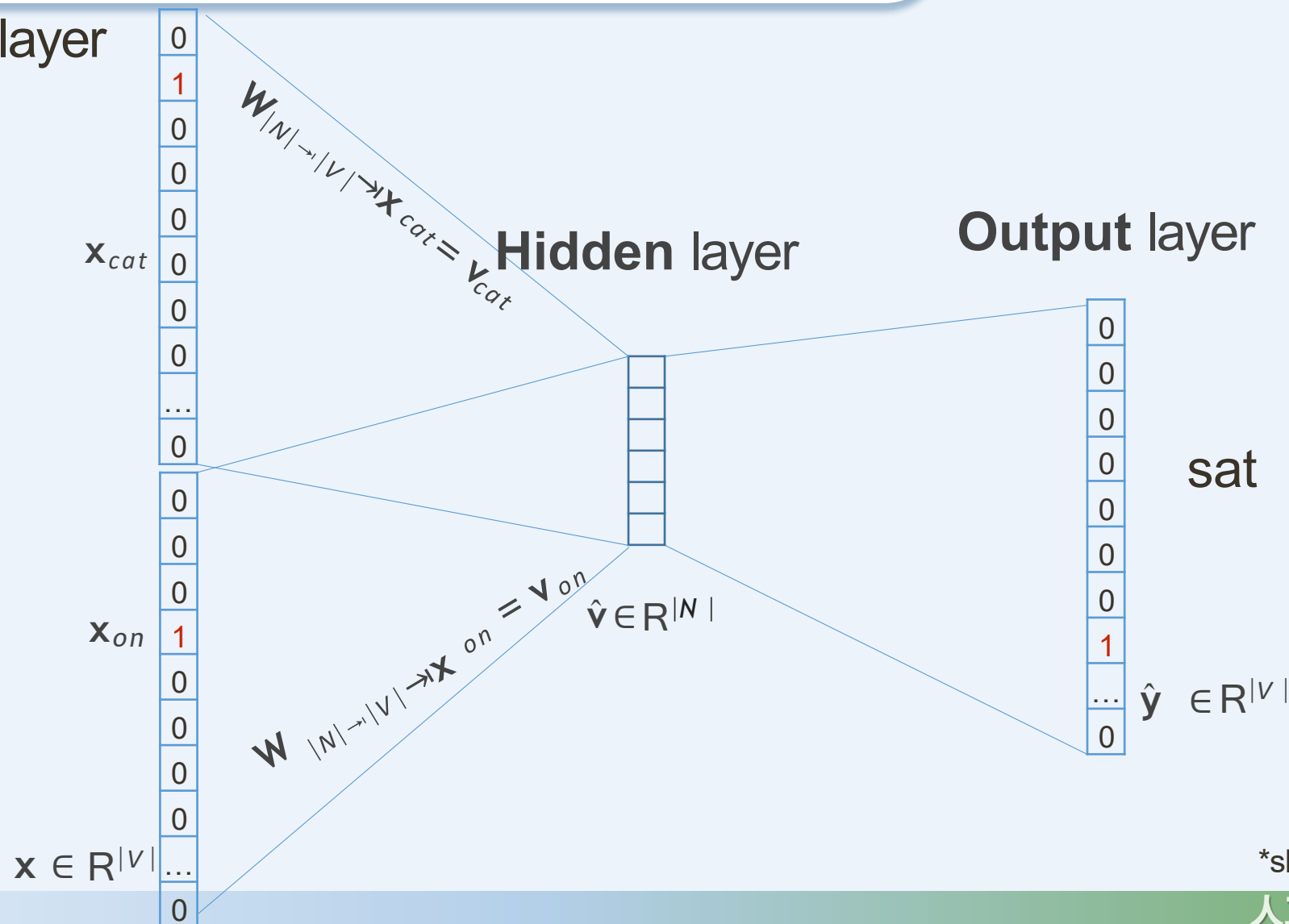
词向量的大小(e.g., 300)

*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

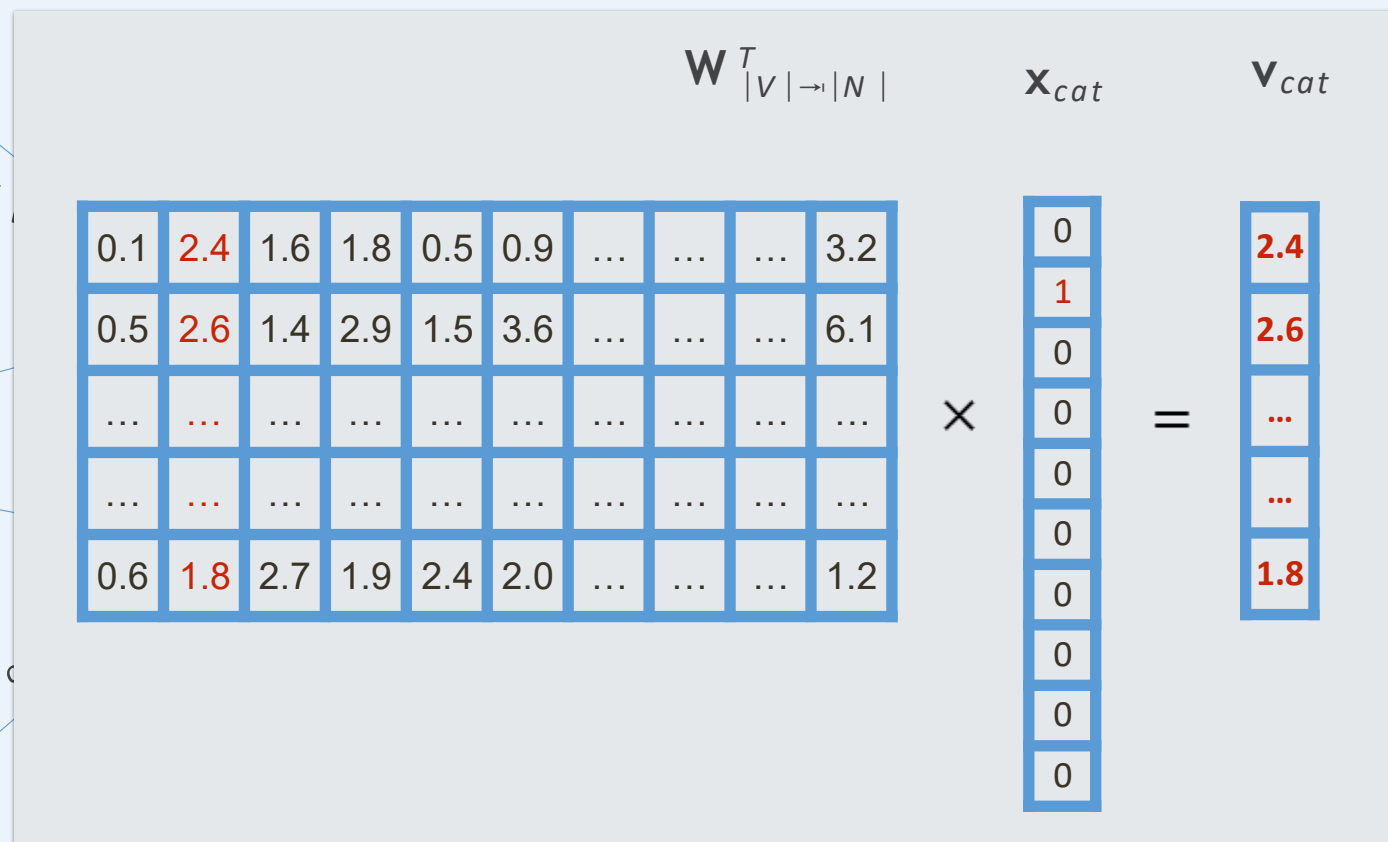
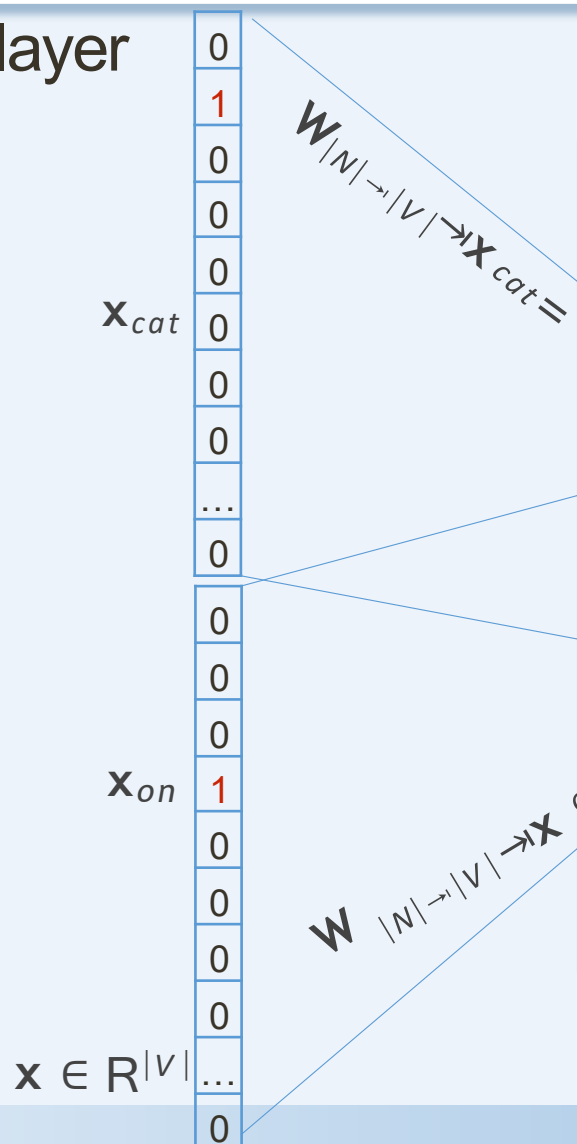


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

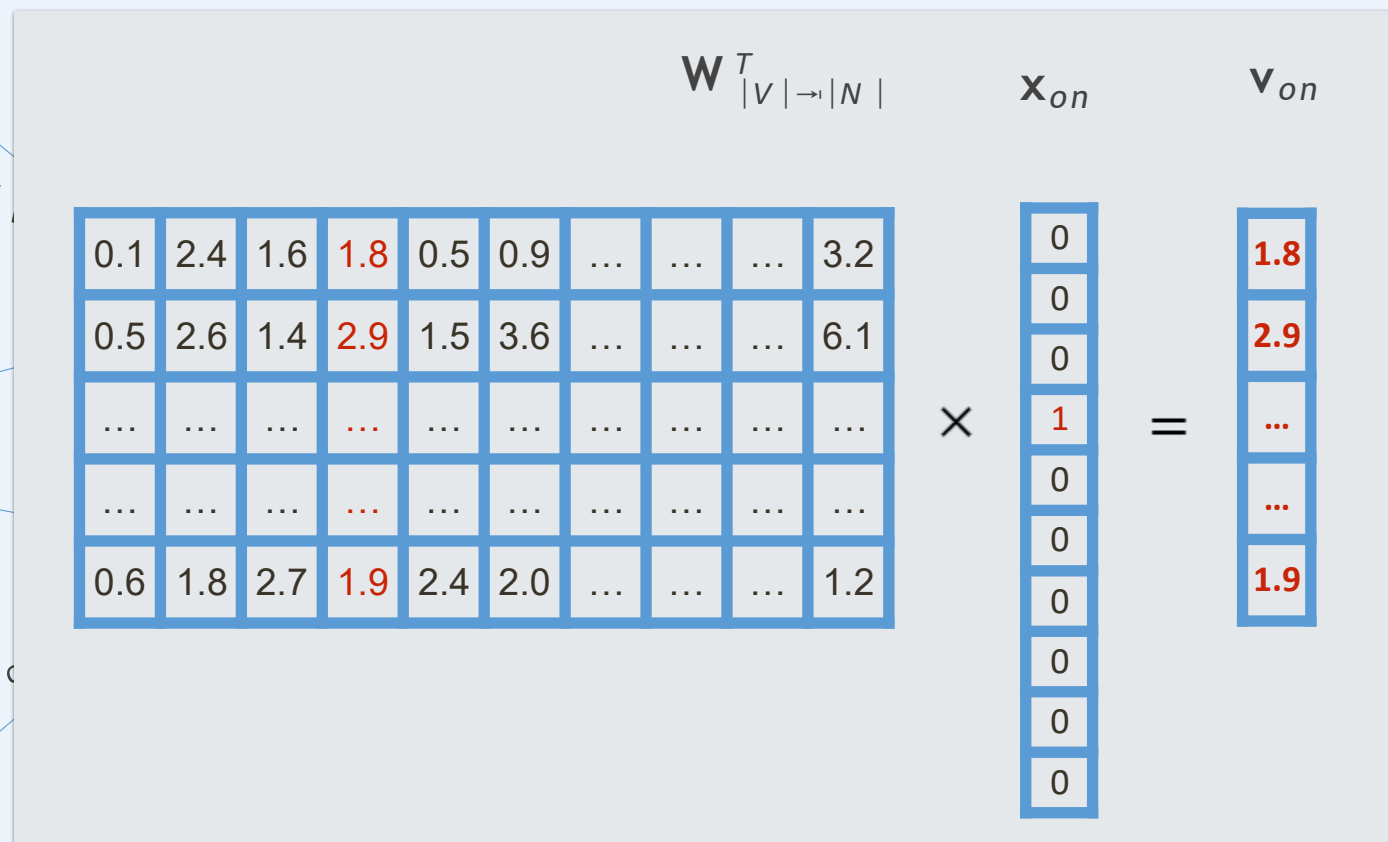
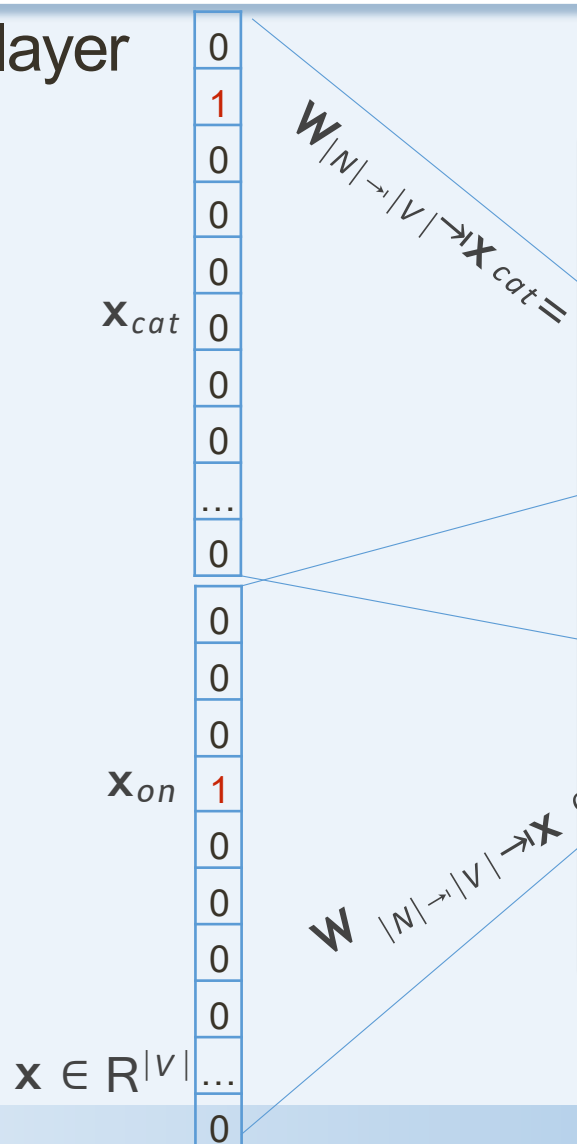


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

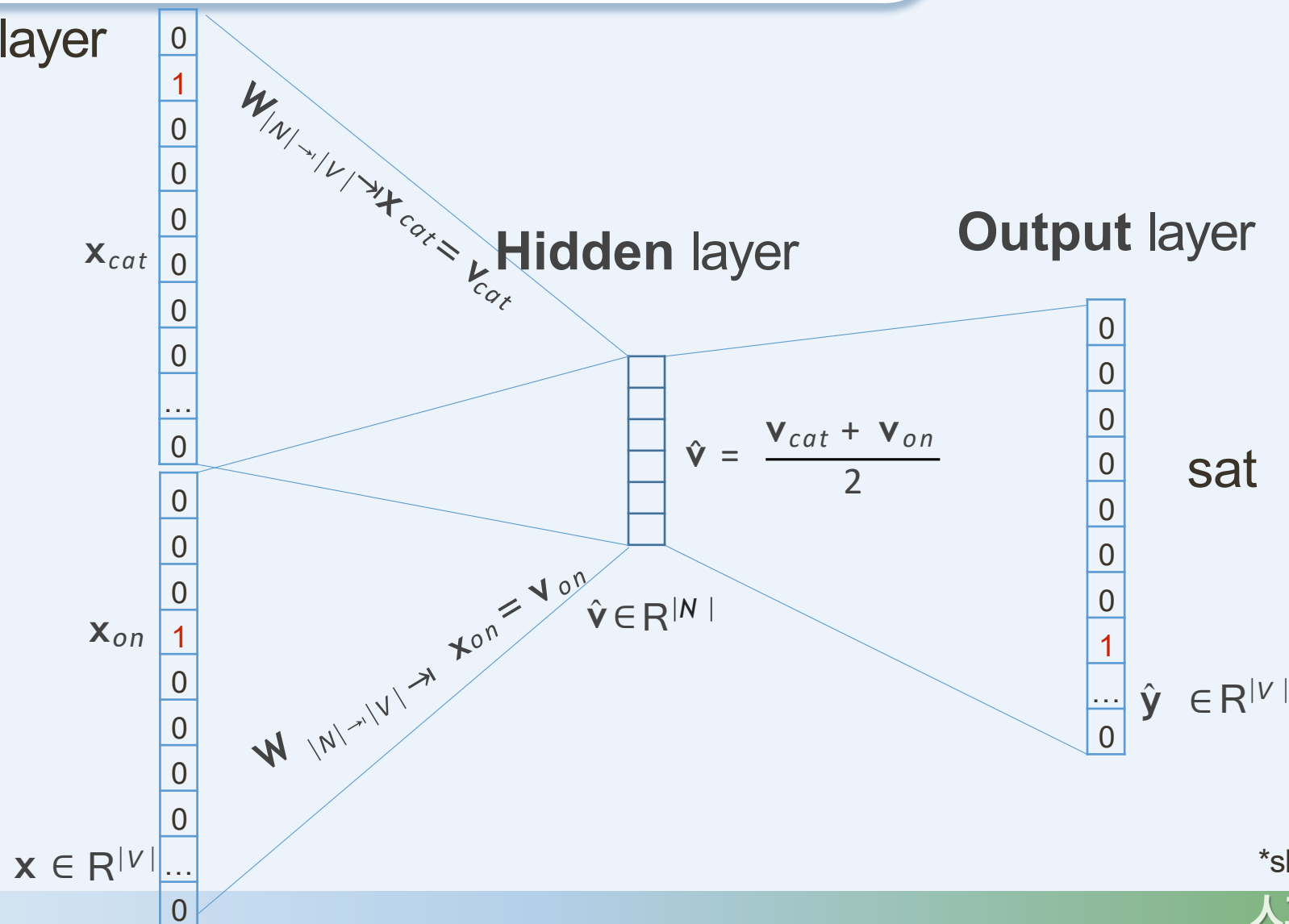


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

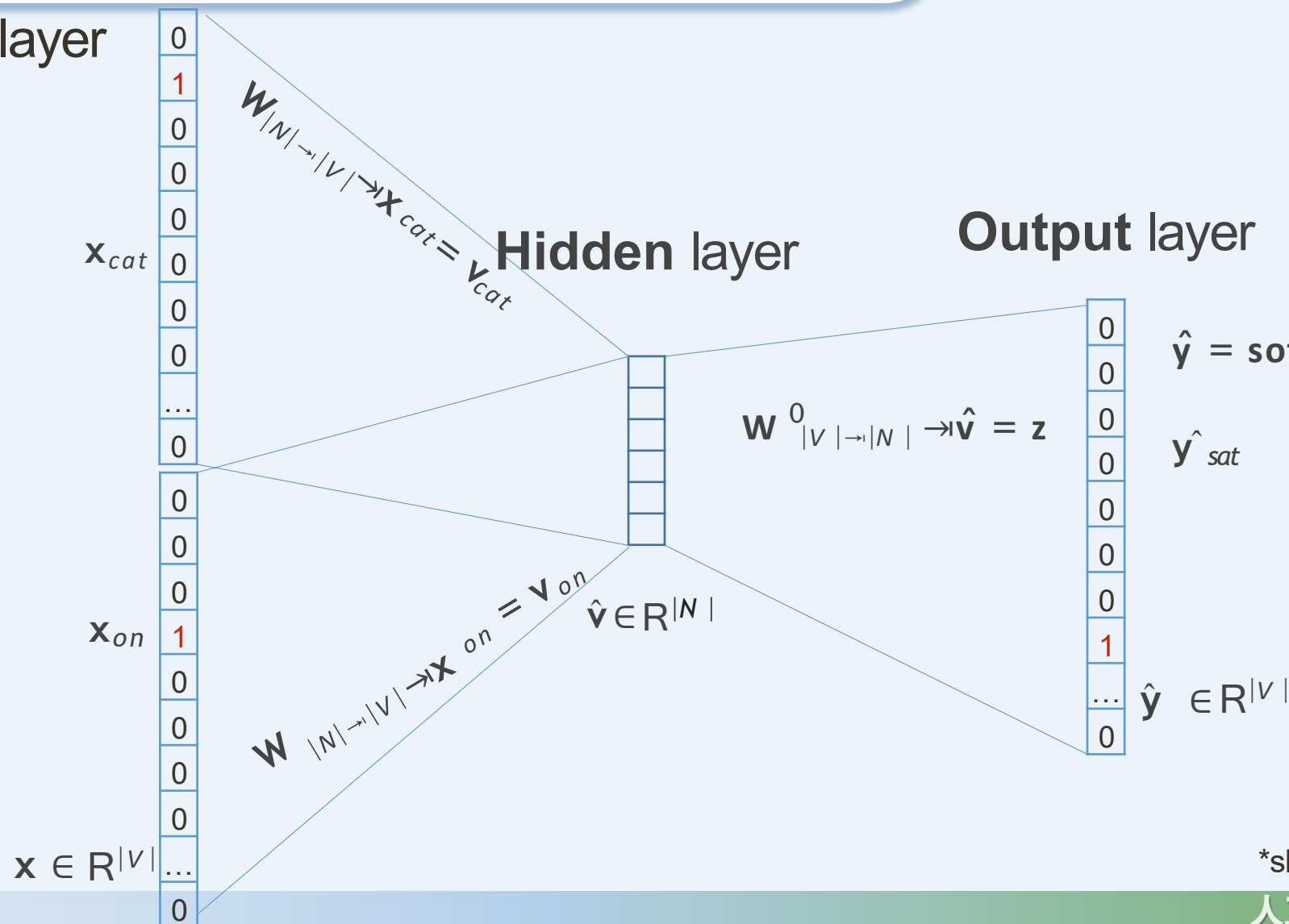


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

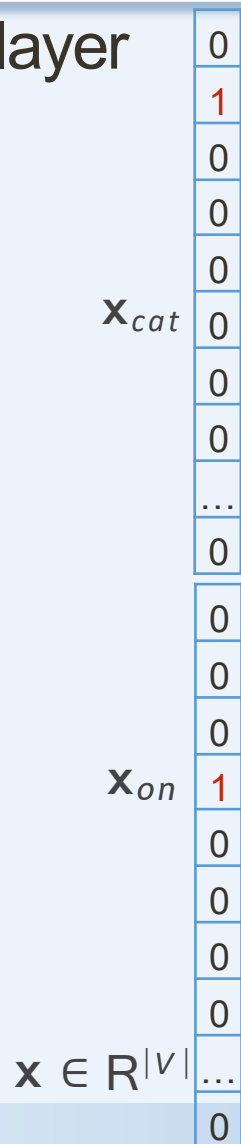


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer



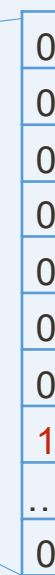
$$W_{|N| \rightarrow |V|} \rightarrow x_{cat} = v_{cat}$$

Hidden layer



$$W^0_{|V| \rightarrow |N|} \rightarrow \hat{v} = z$$

Output layer



$$\hat{y} = \text{softmax}(z)$$

$$\hat{y}_{sat}$$

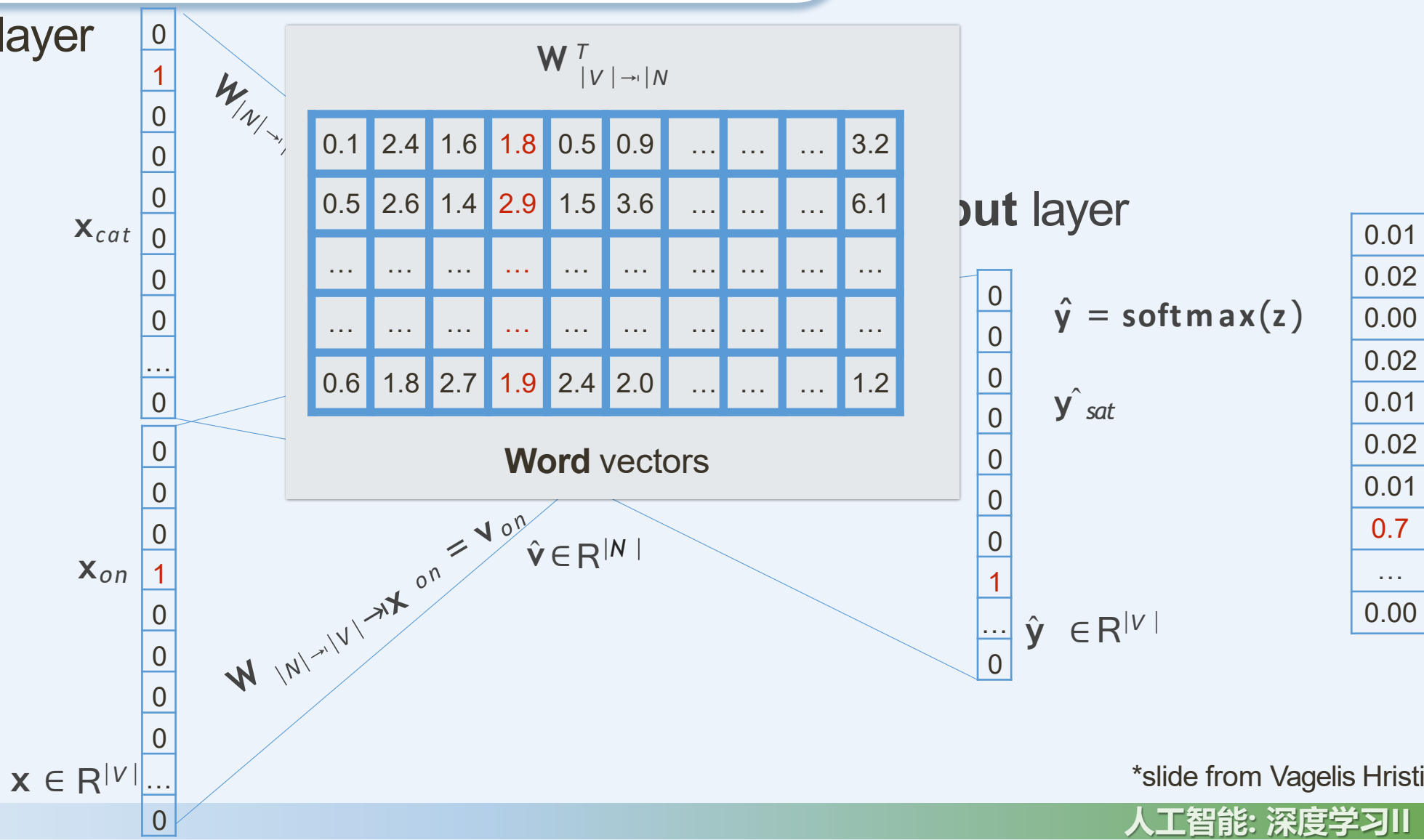
0.01
0.02
0.00
0.02
0.01
0.02
0.01
0.7
...
0.00

优化以接近one-hot编码

CBOW: Continuous Bag of Words

[Mikolov et al., 2013]

Input layer

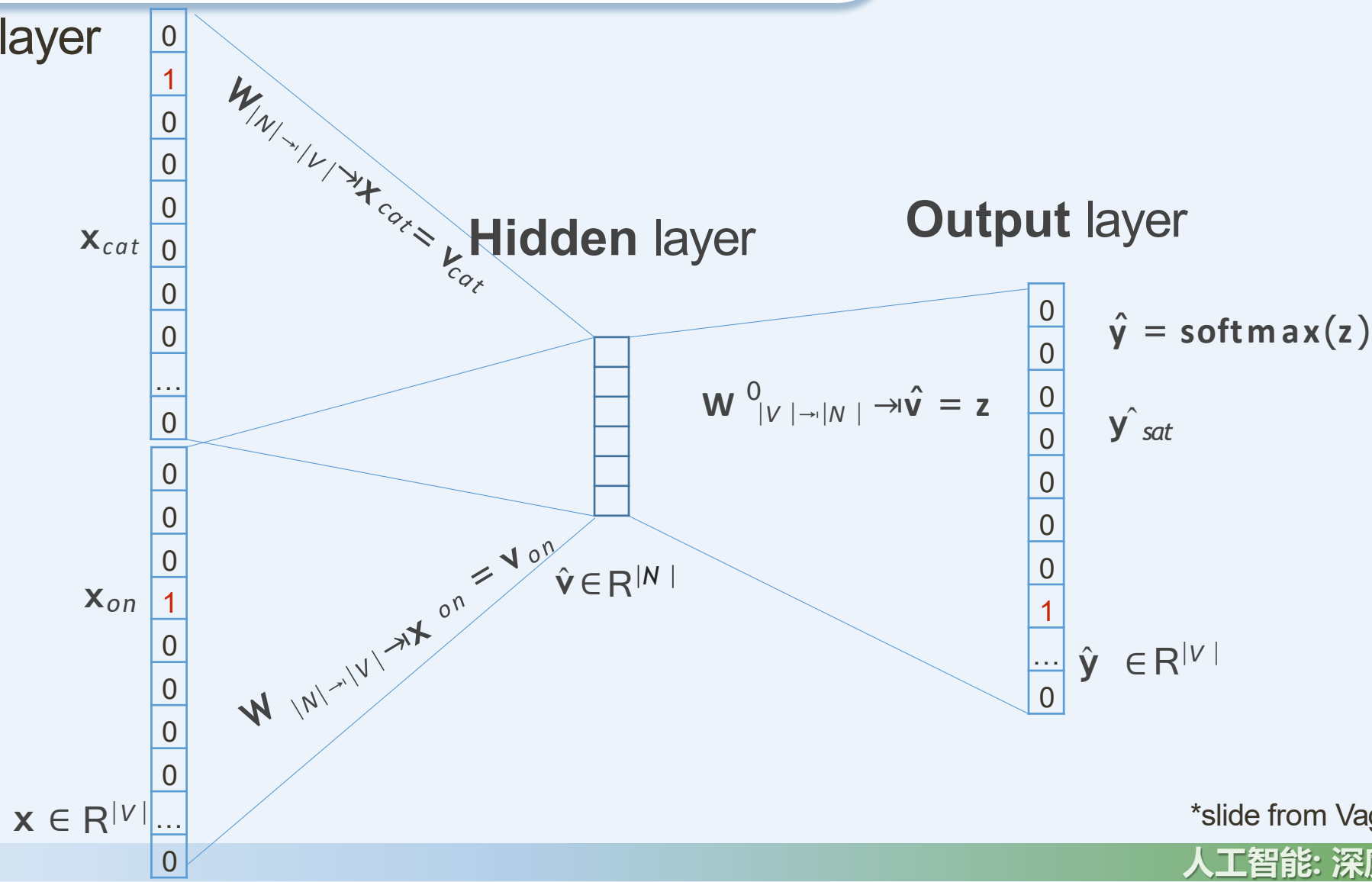


*slide from Vagelis Hristidis

CBOW: Continuous Bag of Words

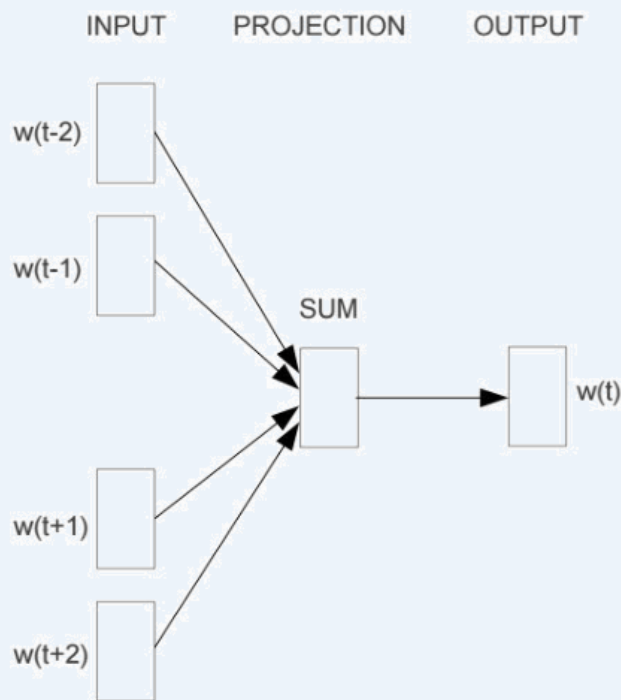
[Mikolov et al., 2013]

Input layer



*slide from Vagelis Hristidis

CBOW: 目标函数



$$L(\theta) = \prod_{t=1}^T P(w_t | \{w_{t+j}\}, -m \leq j \leq m, j \neq 0)$$

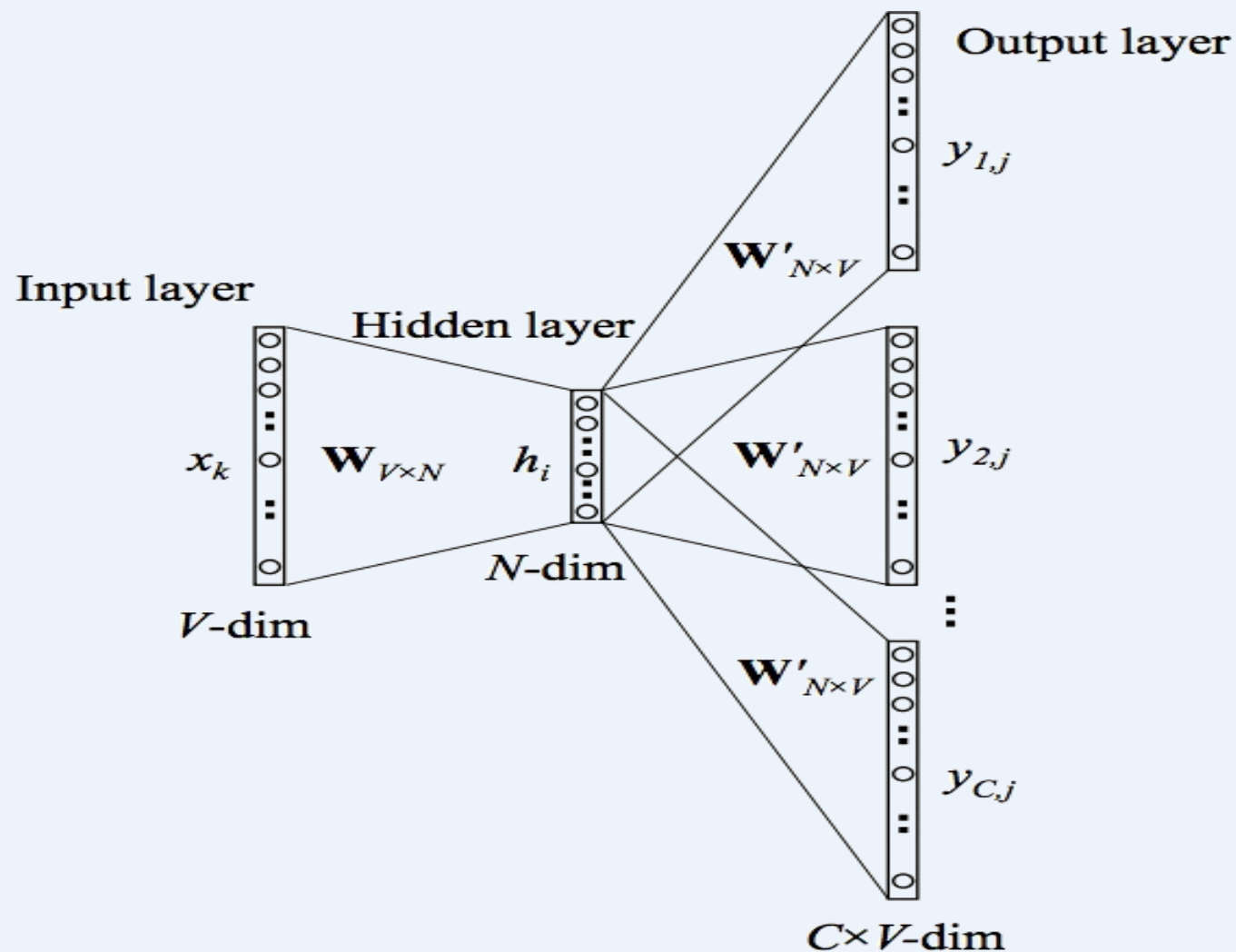
$$\bar{\mathbf{v}}_t = \frac{1}{2m} \sum_{-m \leq j \leq m, j \neq 0} \mathbf{v}_{t+j}$$

$$P(w_t | \{w_{t+j}\}) = \frac{\exp(\mathbf{u}_{w_t} \cdot \bar{\mathbf{v}}_t)}{\sum_{k \in V} \exp(\mathbf{u}_k \cdot \bar{\mathbf{v}}_t)}$$

- Continuous Bag of Words (CBOW): 使用窗口中的上下文单词来预测中心词

Skip-Gram模型

[Mikolov et al., 2013]



CBOW和Skip-Gram比较

— **CBOW**不适用于稀有单词，但通常只需要较少的数据来训练

例子：看到“两只[?]在抓老鼠”，请猜中间的词。

— **Skip-gram**更适用于稀有单词，但需要更多数据来训练模型

例子：给你一个词“[猫]”，请预测它周围经常会和哪些词一起出现。

Model	Vector Dimensionality	Training words	Accuracy [%]		
			Semantic	Syntactic	Total
Collobert-Weston NNLM	50	660M	9.3	12.3	11.0
Turian NNLM	50	37M	1.4	2.6	2.1
Turian NNLM	200	37M	1.4	2.2	1.8
Mnih NNLM	50	37M	1.8	9.1	5.8
Mnih NNLM	100	37M	3.3	13.2	8.8
Mikolov RNNLM	80	320M	4.9	18.4	12.7
Mikolov RNNLM	640	320M	8.6	36.5	24.6
Huang NNLM	50	990M	13.3	11.6	12.3
Our NNLM	20	6B	12.9	26.4	20.3
Our NNLM	50	6B	27.9	55.8	43.2
Our NNLM	100	6B	34.2	64.5	50.8
CBOW	300	783M	15.5	53.1	36.1
Skip-gram	300	783M	50.0	55.9	53.3



CBOW和Skip-Gram比较



Interesting Result: 词语类比

Mikolov et al. (2014)对线性关系进行测试:

a:b :: c:?



$$d = \arg \max_x \frac{(w_b - w_a + w_c)^T w_x}{\|w_b - w_a + w_c\|}$$

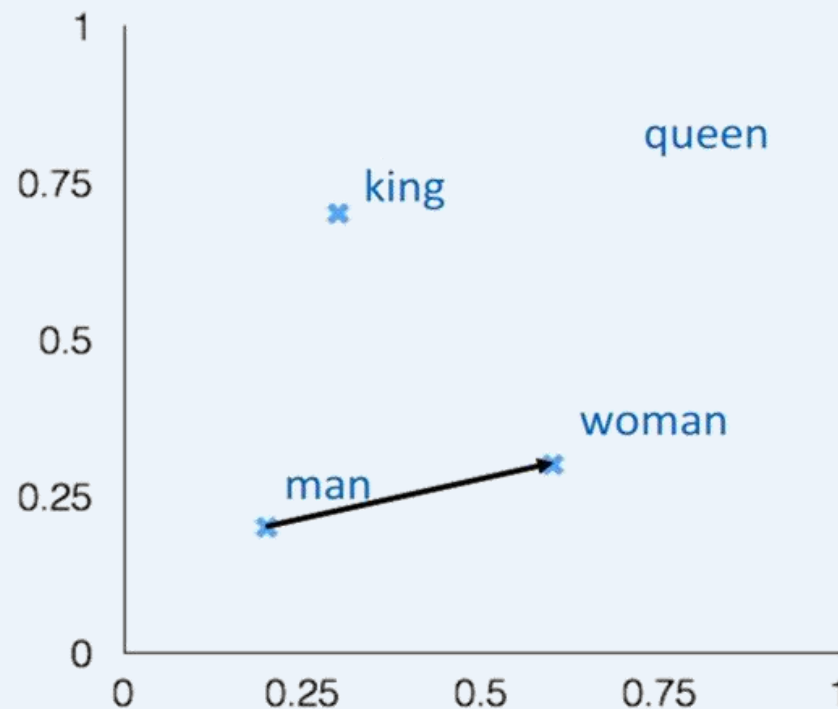
man:woman :: king:?

+ king [0.30 0.70]

- man [0.20 0.20]

+ woman [0.60 0.30]

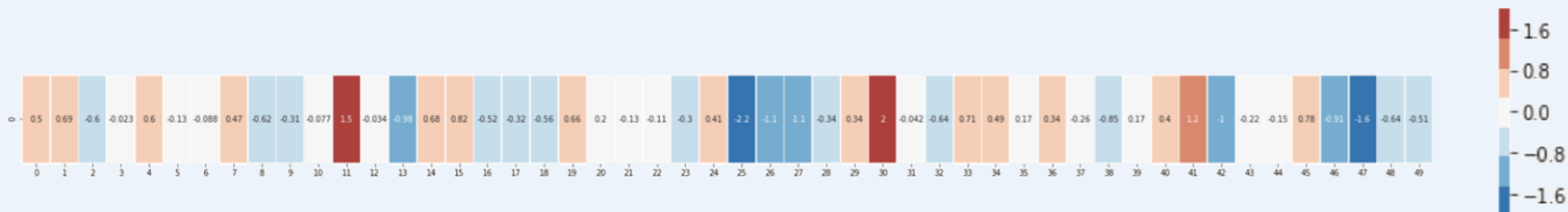
queen [0.70 0.80]



Interesting Result: 词语类比

这是单词“king”的词嵌入 (在Wiki百科上训练的GloVe vector):

[0.50451 , 0.68607 , -0.59517 , -0.022801, 0.60046 , -0.13498 , -0.08813 , 0.47377 , -0.61798 ,
-0.31012 , -0.076666, 1.493 , -0.034189, -0.98173 , 0.68229 , 0.81722 , -0.51874 , -0.31503 , -
0.55809 , 0.66421 , 0.1961 , -0.13495 , -0.11476 , -0.30344 , 0.41177 , -2.223 , -1.0756 , -
1.0783 , -0.34354 , 0.33505 , 1.9927 , -0.04234 , -0.64319 , 0.71125 , 0.49159 , 0.16754 ,
0.34344 , -0.25663 , -0.8523 , 0.1661 , 0.40102 , 1.1685 , -1.0137 , -0.21585 , -0.15155 ,
0.78321 , -0.91241 , -1.6106 , -0.64426 , -0.51042]



Interesting Result: 词语类比

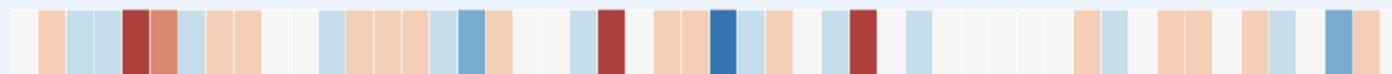
“king”



“Man”



“Woman”

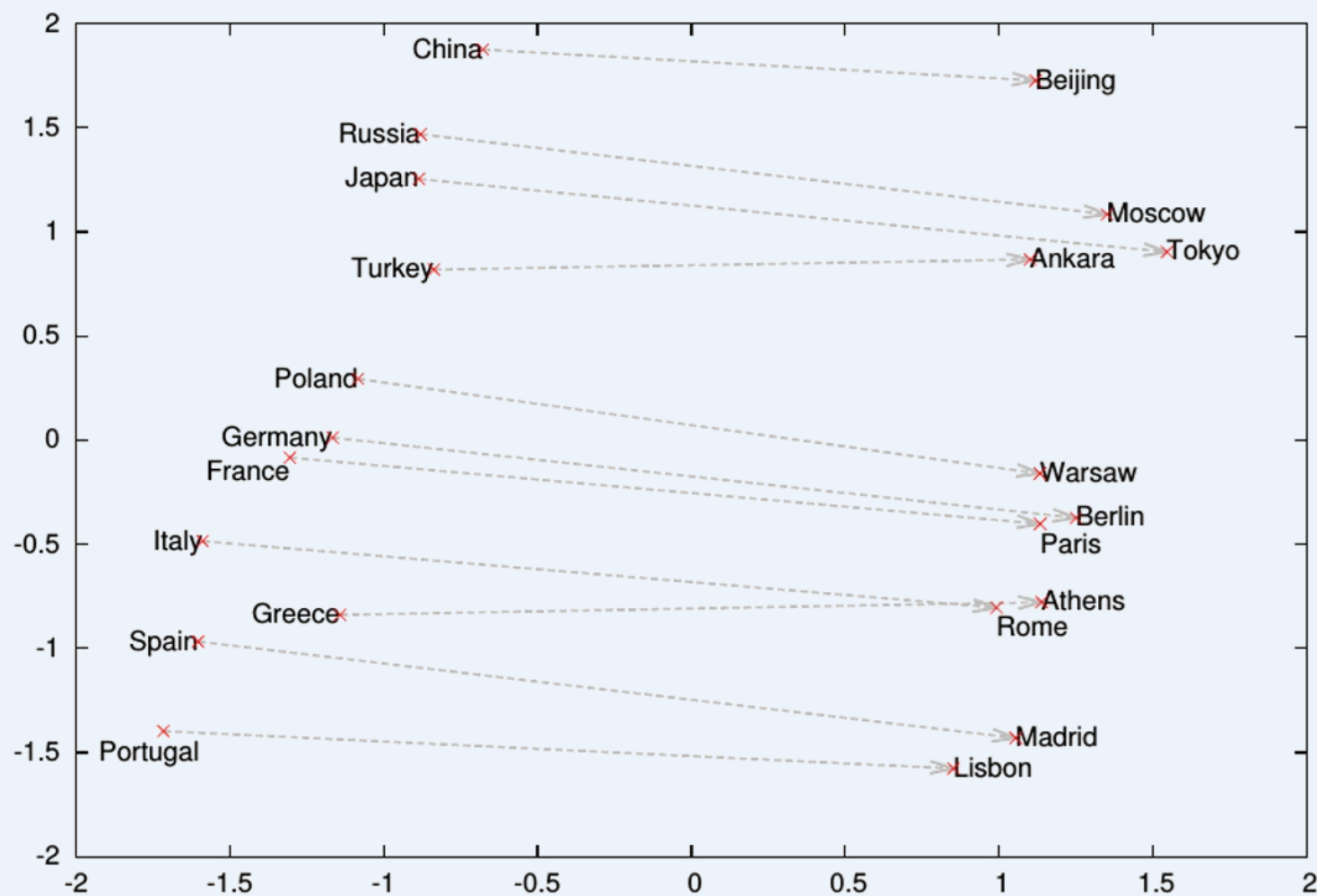


king - man + woman $\approx$ queen





Interesting Result: 词语类比



其他模型

Word2vec 词向量模型是将文字依照语言惯例转换成对应的向量，其训练出来的词向量会有盲点：

- 在同性质文件中出现，但是文句距离太远的字无法训练到

如果两个词在同一篇行业报告里频繁出现（说明它们高度相关），但在具体的句子里总是隔得很远（超过了窗口大小），Word2Vec的窗口就永远无法同时框住它们，从而无法捕捉到这种宏观的全局关联。

- 有类似字首 / 字尾的词向量，无法在词向量中表示出来

例如 reading、reader 和 readable，虽然长得很像且含义相近，但 Word2Vec 会把它们当成三个互不相干的独立个体来从头学习

因此有了 **Glove** 与 **FastText** 两种词向量方式依序解决以上问题。

$w \in \mathbb{R}^d$ are word vectors

probe word

$$F(w_i, w_j, \tilde{w}_k) = \frac{P_{ik}}{P_{jk}}$$

co-relations between the word w_i and w_j

co-occurrence probabilities for the word w_j and w_k

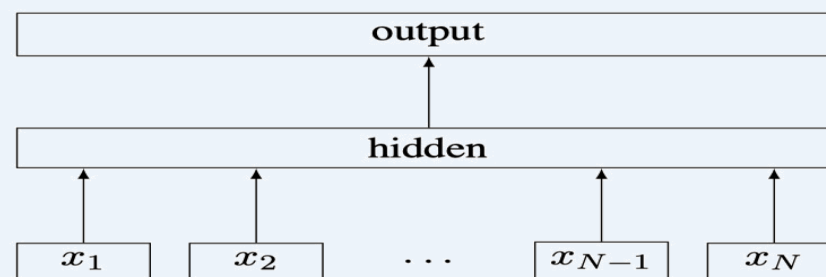


Figure 1: Model architecture of fastText for a sentence with N ngram features $x_1, \dots, x_N$. The features are embedded and averaged to form the hidden variable.

Glove

- GloVe(Global Vectors for Word Representation全局向量的词嵌入)是一个全局对数双线性回归模型。

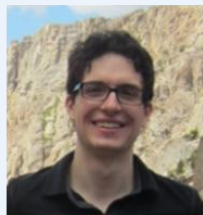
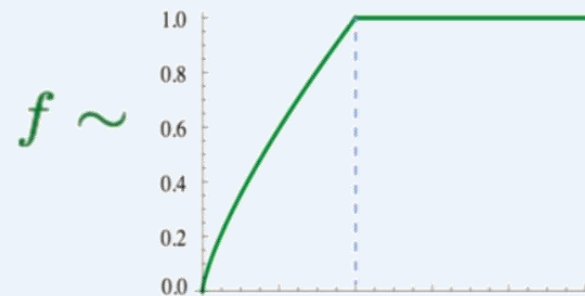
- **目标：**得到单词的词向量，让其尽可能的包含语义与语法信息。
- 输入是语料库(没错，不需要去构建训练集)，输出是词向量。
- GloVe模型的思路是：从语料库中统计共现矩阵，然后根据共现矩阵与GloVe模型来学习词向量。

GloVe: **Global Vectors**

- Let's take the global co-occurrence statistics: $X_{i,j}$

$$J = \sum_{i,j=1}^V f(X_{ij}) (w_i^T \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij})^2$$

- Training faster
- Scalable to very large corpora



(Pennington et al, 2014): GloVe: Global Vectors for Word Representation

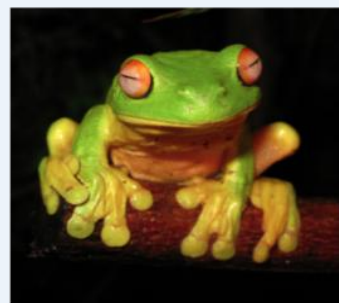
Glove

GloVe: **G**lobal **V**ectors

- GloVe(Global Vectors for Word Representation全局向量的词嵌入)是一个全局对数双线性回归模型。
- **目标：**得到单词的词向量，让其尽可能的包含语义与语法信息。
- 输入是语料库(没错，不需要去构建训练集)，输出是词向量。
- GloVe模型的思路是：从语料库中统计共现矩阵，然后根据共现矩阵与GloVe模型来学习词向量。

Nearest words to
frog:

1. frogs
2. toad
3. litoria
4. leptodactylidae
5. rana
6. lizard
7. eleutherodactylus



litoria



leptodactylidae



rana



eleutherodactylus

(Pennington et al, 2014): GloVe: Global Vectors for Word Representation

FastText

FastText: Sub-Word Embeddings

FastText是Facebook在2016年所提出的方法。其实，整个模型架构并没有特别创新的地方，和CBOW模型非常地像。其创新的地方在于：**子词嵌入的引入**。

- Similar as Skip-gram, but break words into n-grams with $n = 3$ to 6

where: 3-grams: <wh, whe, her, ere, re>

4-grams: <whe, wher, here, ere>

5-grams: <wher, where, here>

6-grams: <where, where>

- Replace $\mathbf{u}_i \cdot \mathbf{v}_j$ by $\sum_{g \in n\text{-grams}(w_i)} \mathbf{u}_g \cdot \mathbf{v}_j$

- More to come! Contextualized word embeddings



(Bojanowski et al, 2017): Enriching Word Vectors with Subword Information

Take-Away

Trained word embeddings available

- word2vec: <https://code.google.com/archive/p/word2vec/>
- GloVe: <https://nlp.stanford.edu/projects/glove/>
- FastText: <https://fasttext.cc/>

Download pre-trained word vectors

- Pre-trained word vectors. This data is made available under the [Public Domain Dedication and License](http://www.opendatacommons.org/licenses/pddl/1.0/) v1.0 whose full text can be found at: <http://www.opendatacommons.org/licenses/pddl/1.0/>.
 - [Wikipedia 2014](#) + [Gigaword 5](#) (6B tokens, 400K vocab, uncased, 50d, 100d, 200d, & 300d vectors, 822 MB download): [glove.6B.zip](#)
 - Common Crawl (42B tokens, 1.9M vocab, uncased, 300d vectors, 1.75 GB download): [glove.42B.300d.zip](#)
 - Common Crawl (840B tokens, 2.2M vocab, cased, 300d vectors, 2.03 GB download): [glove.840B.300d.zip](#)
 - Twitter (2B tweets, 27B tokens, 1.2M vocab, uncased, 25d, 50d, 100d, & 200d vectors, 1.42 GB download): [glove.twitter.27B.zip](#)
- Ruby [script](#) for preprocessing Twitter data

Differ in algorithms, text corpora, dimensions, cased/uncased...



机器翻译 (Machine Translation)

机器翻译(MT) 任务：将一种语言(**源语言**)中的句子 x 翻译为另一种语言(**目标语言**)中的句子 y

x : *L'homme est né libre, et partout il est dans les fers*

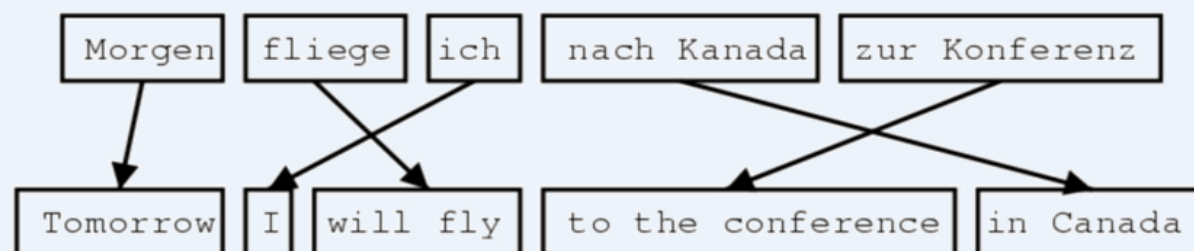


y : *Man is born free, but everywhere he is in chains*

– Rousseau

机器翻译 (Machine Translation)

翻译中所要做的事，对模型来说并不简单



1519年600名西班牙人在墨西哥登陆，去征服**几百万人口**的**阿兹特克帝国**，初次交锋他们损兵三分之二。

In 1519, six hundred Spaniards landed in Mexico to conquer **the Aztec Empire** **with a population of a few million**. They lost two thirds of their soldiers in the first clash.

translate.google.com (2009): 1519 600 Spaniards landed in Mexico, **millions of people to conquer the Aztec empire**, the first two-thirds of soldiers against their loss.

translate.google.com (2013): 1519 600 Spaniards landed in Mexico **to conquer the Aztec empire, hundreds of millions of people**, the initial confrontation loss of soldiers two-thirds.

translate.google.com (2015): 1519 600 Spaniards landed in Mexico, **millions of people to conquer the Aztec empire**, the first two-thirds of the loss of soldiers they clash.

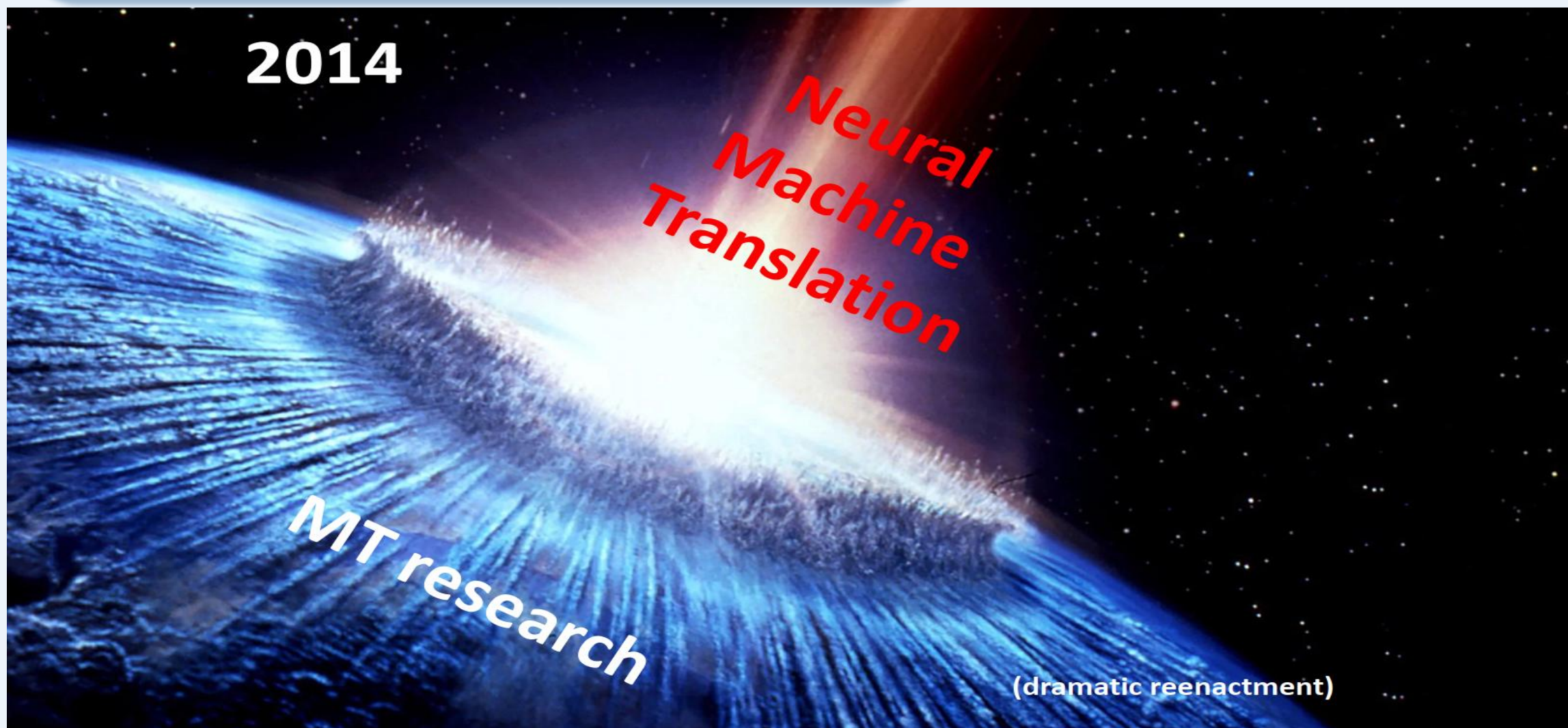


机器翻译 (Machine Translation)

1990s–2010s: 统计机器翻译 (Statistical Machine Translation)

- SMT曾经是庞大的研究领域
- 最优秀的系统及其复杂
 - 有数百个重要细节
- 系统中具有相当多单独设计的子组件
 - 需要大量的特征工程
 - 需要设计大量特征来捕获特定的语言现象
 - 需要翻译和维护额外的资源
 - 比如：等价短语(equivalent phrases)表
 - 需要大量人力维护
 - 对每两个语言都进行重复的工作

机器翻译 (Machine Translation)



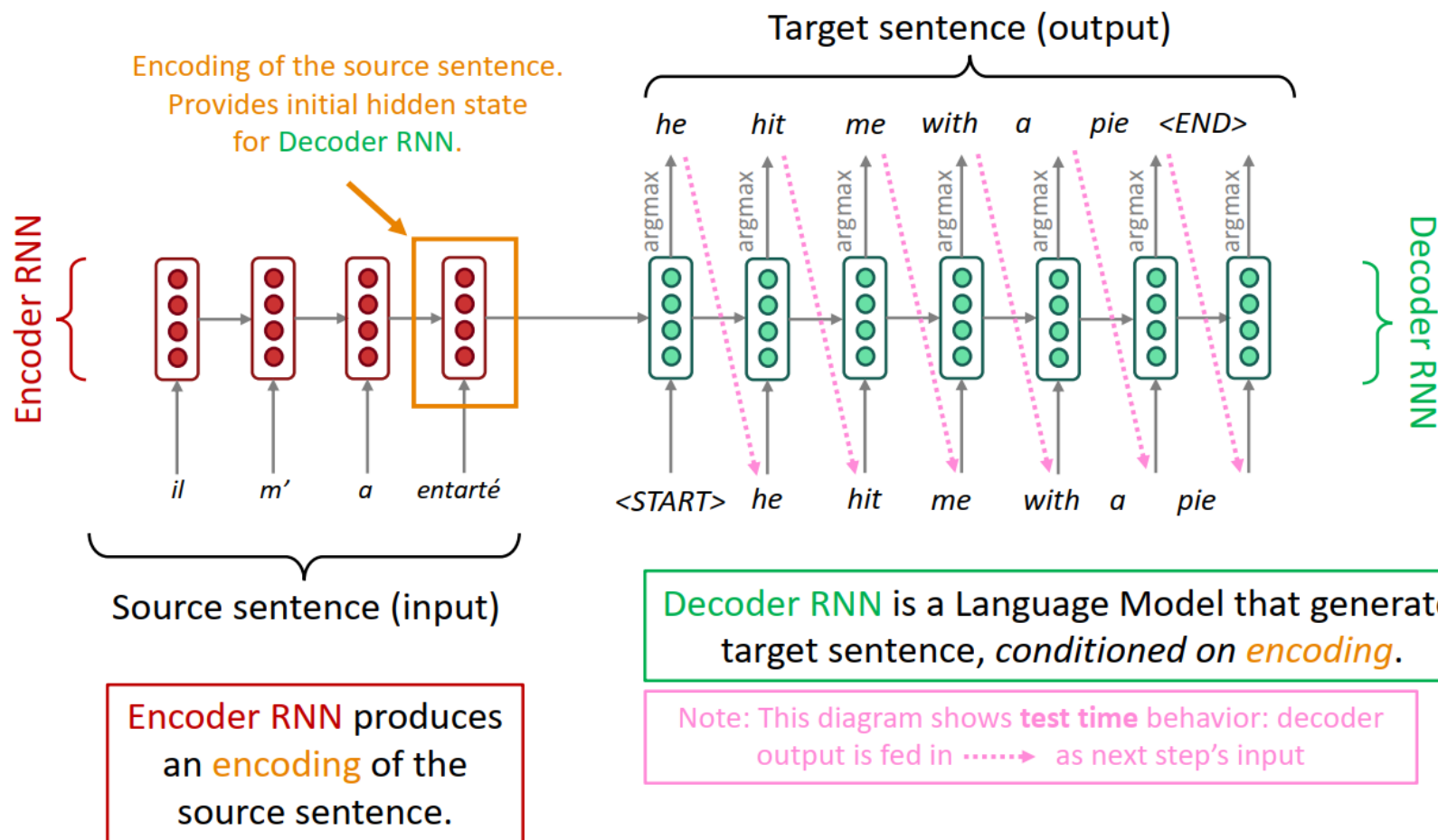


从SMT到NMT

- 神经机器翻译(Neural Machine Translation, NMT) 是一种用单个端到端神经网络进行机器翻译的方式
- 这种神经网络架构称为序列到序列模型 (即seq2seq), 其包括2个RNN

神经机器翻译NMT

The sequence-to-sequence model





Seq2Seq模型的通用性

- 这里的一般概念是个**编码器-解码器(encoder-decoder)**模型
 - 一个神经网络接受输入并生成神经表征
 - 另一个神经网络根据该神经表示生成输出
 - 如果输入和输出是序列，我们称其为 seq2seq 模型
- Seq2Seq不止是在MT上好用
- 许多 NLP 任务可以表述为sequence-to-sequence:
 - **文本总结** (长文本 → 短文本)
 - **对话** (先前的话语 → 接下来的话语)
 - **解析** (输入文本 → 以序列的形式输出解析)
 - **代码生成** (自然语言 → Python 代码)



Seq2Seq模型的通用性

代码生成 (NL2Sql 大模型)

👤 请帮我写一个SQL，查询出课程表中每一个学生的平均成绩



当然可以！ 以下是一个例子：

```
SELECT student_id, AVG(score) AS avg_score
FROM students
GROUP BY student_id;
```



这个 SQL 语句将返回每个学生的平均分数。 `student_id` 是课程表中的主键， `avg_score` 为每个学生的平均分数。我们使用了 `AVG()` 函数来计算总分除以人数得到的平均分数。注意：上面的例子中假设 `students` 包含一个列名为 `score` 的学生成绩列。如果实际表中有多个成绩列，则需要修改语句中的相应部分以适应您的实际情况。



数据集	公开年份	Query数量	数据库数	涉及领域	单轮/多轮	语言
ATIS	1994	5280	1	单领域	单轮	英文
GeoQuery	2001	877	1	单领域	单轮	英文
Restaurants	2003	378	1	单领域	单轮	英文
Academic	2014	196	1	单领域	单轮	英文
Scholar	2017	817	1	单领域	单轮	英文
IMDB	2017	131	1	单领域	单轮	英文
Yelp	2017	128	1	单领域	单轮	英文
WikiSQL	2017	80657	26521	多领域	单轮	英文
Advising	2018	3898	1	单领域	单轮	英文
Spider	2018	10181	200	多领域	单轮	英文
NL2SQL	2019	49974	5291	多领域	单轮	中文
CSpider	2019	9691	166	多领域	单轮	中文
SParC	2019	4298	200	多领域	多轮	英文
CoSQL	2019	3007	200	多领域	多轮	英文
DuSQL	2020	23797	200	多领域	单轮	中文
CHASE	2021	17940	280	多领域	多轮	中文



条件语言模型

- Seq2Seq模型是**条件语言模型**的一个例子
 - **语言模型**：因为解码器预测目标句子 y 的下一个单词
 - **条件**：因为它的预测也以源句子 x 为条件

- NMT directly calculates $P(y|x)$:

$$P(y|x) = P(y_1|x) P(y_2|y_1, x) P(y_3|y_1, y_2, x) \dots P(y_T|y_1, \dots, y_{T-1}, x)$$

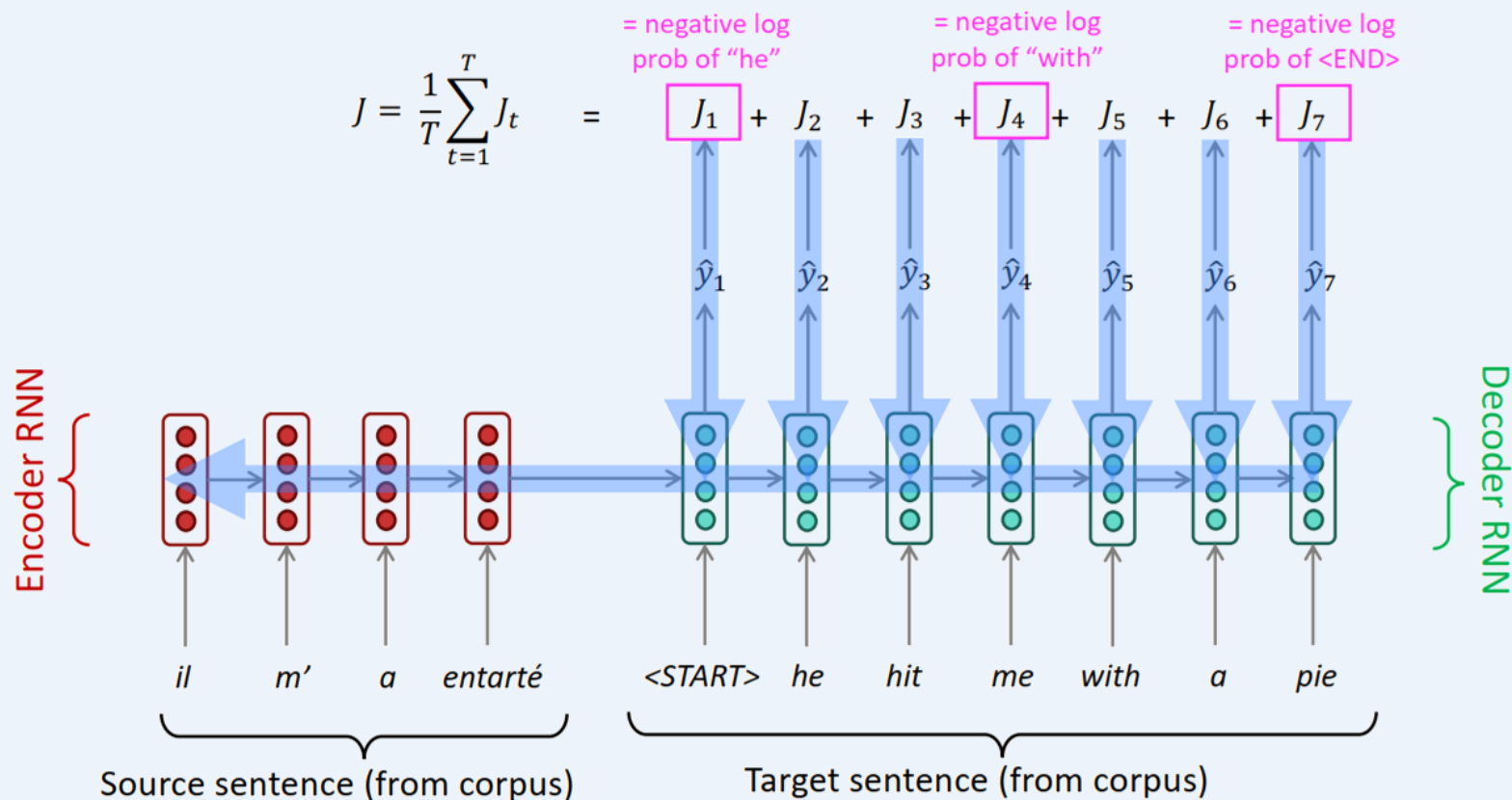
Probability of next target word, given
target words so far and source sentence x

- **问题**：如何训练NMT系统?
- **(简单的) 答案**：获取一个大的并行语料库...

人类已经翻译好的、高质量的**双语句子对**（对齐的文-英数据集，比如“我爱你 和 I love you”）

- 但现在在“无监督 NMT”、数据增强等方面已经有了很好的工作

NMT

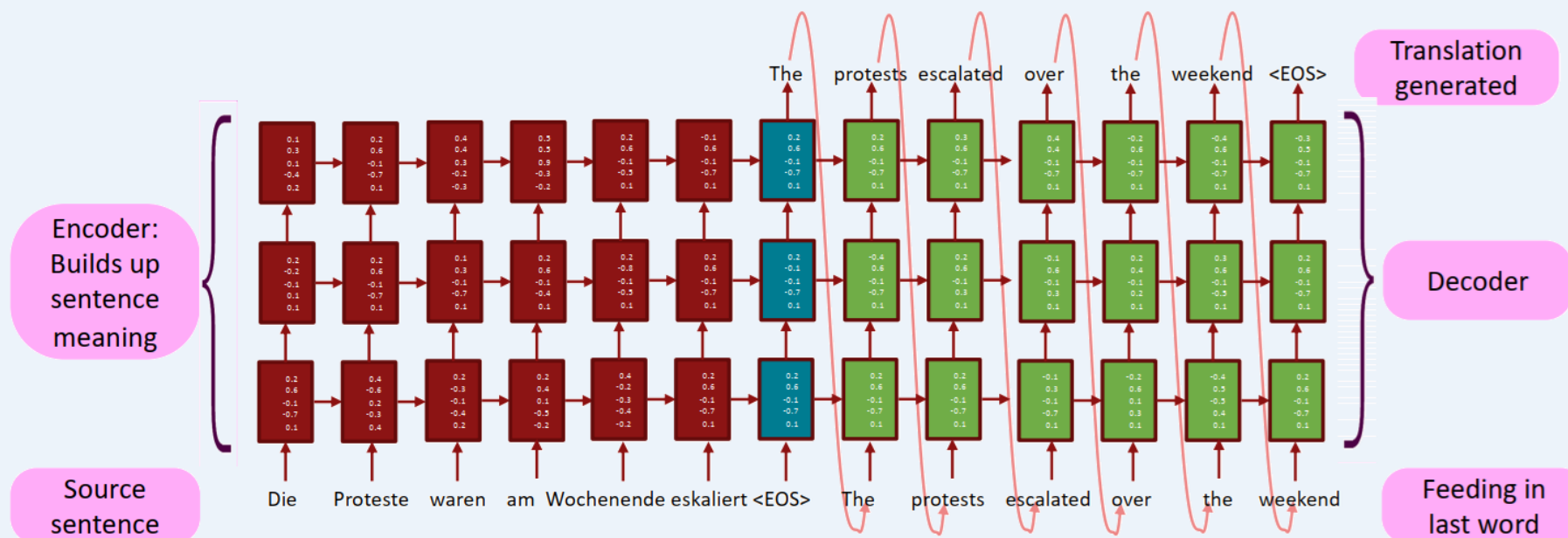


Seq2seq is optimized as a **single system**. Backpropagation operates "end-to-end".

多层深度编码器-解码器机器翻译网络

[Sutskever et al. 2014; Luong et al. 2015]

The hidden states from RNN layer i are the inputs to RNN layer $i+1$



Conditioning =
Bottleneck

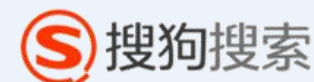


神经机器翻译NMT的成功

NMT: NLP深度学习的首个巨大成功

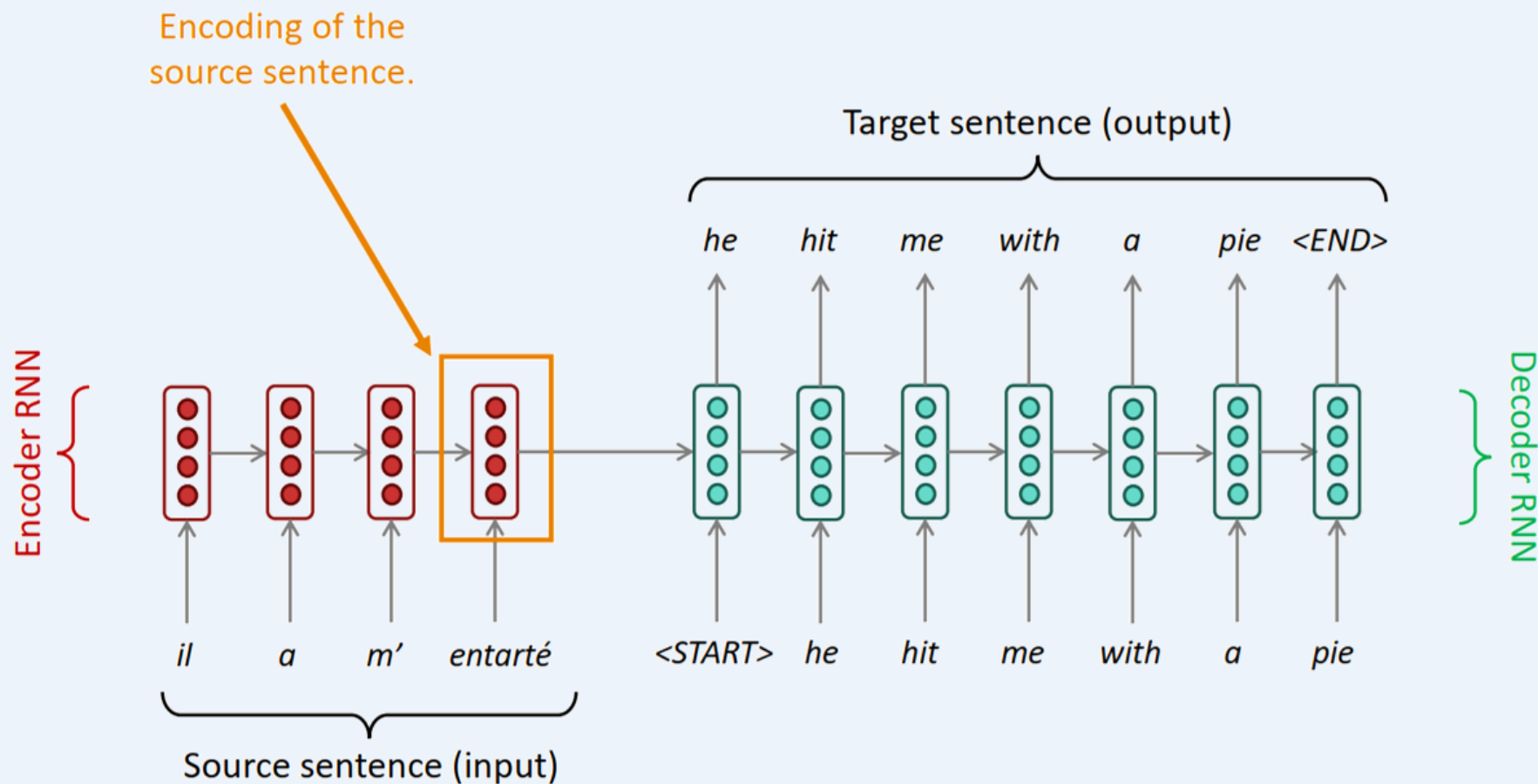
NMT从2014年的边缘研究to变成2016年的领先标准方法

- 2014: 第一篇seq2seq论文发表 [Sutskever et al. 2014]
- 2016: 谷歌翻译从SMT转而采用NMT – 且到2018年已全面替代
 - <https://www.nytimes.com/2016/12/14/magazine/the-great-ai-awakening.html>



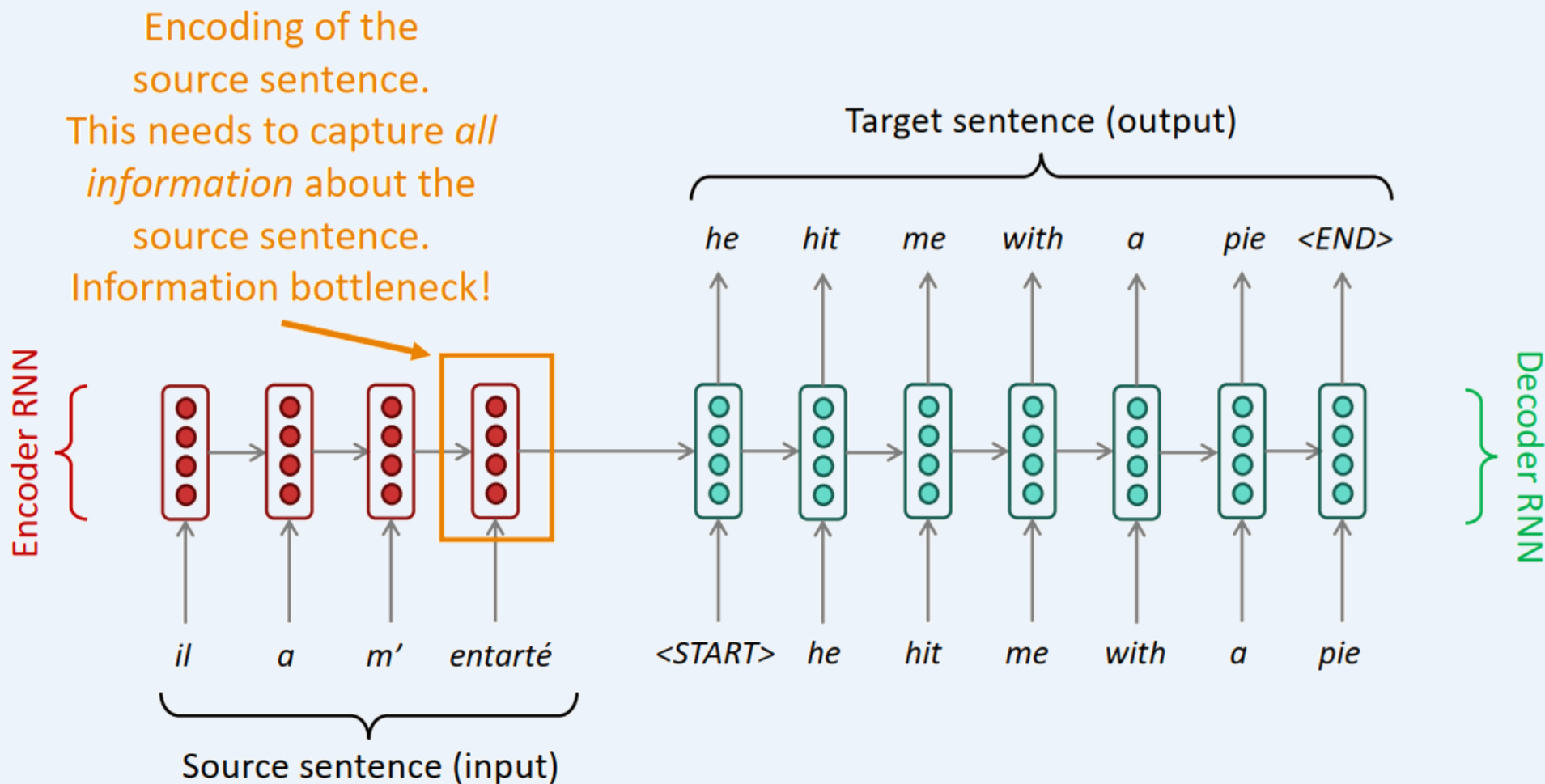
- Amazing!
 - 由上百工程师数年构建的SMT系统，被由 工程师小组仅花费数月构建的NMT系统超越

Seq2Seq的瓶颈问题



Problems with this architecture?

Seq2Seq的瓶颈问题



注意力(Attention)

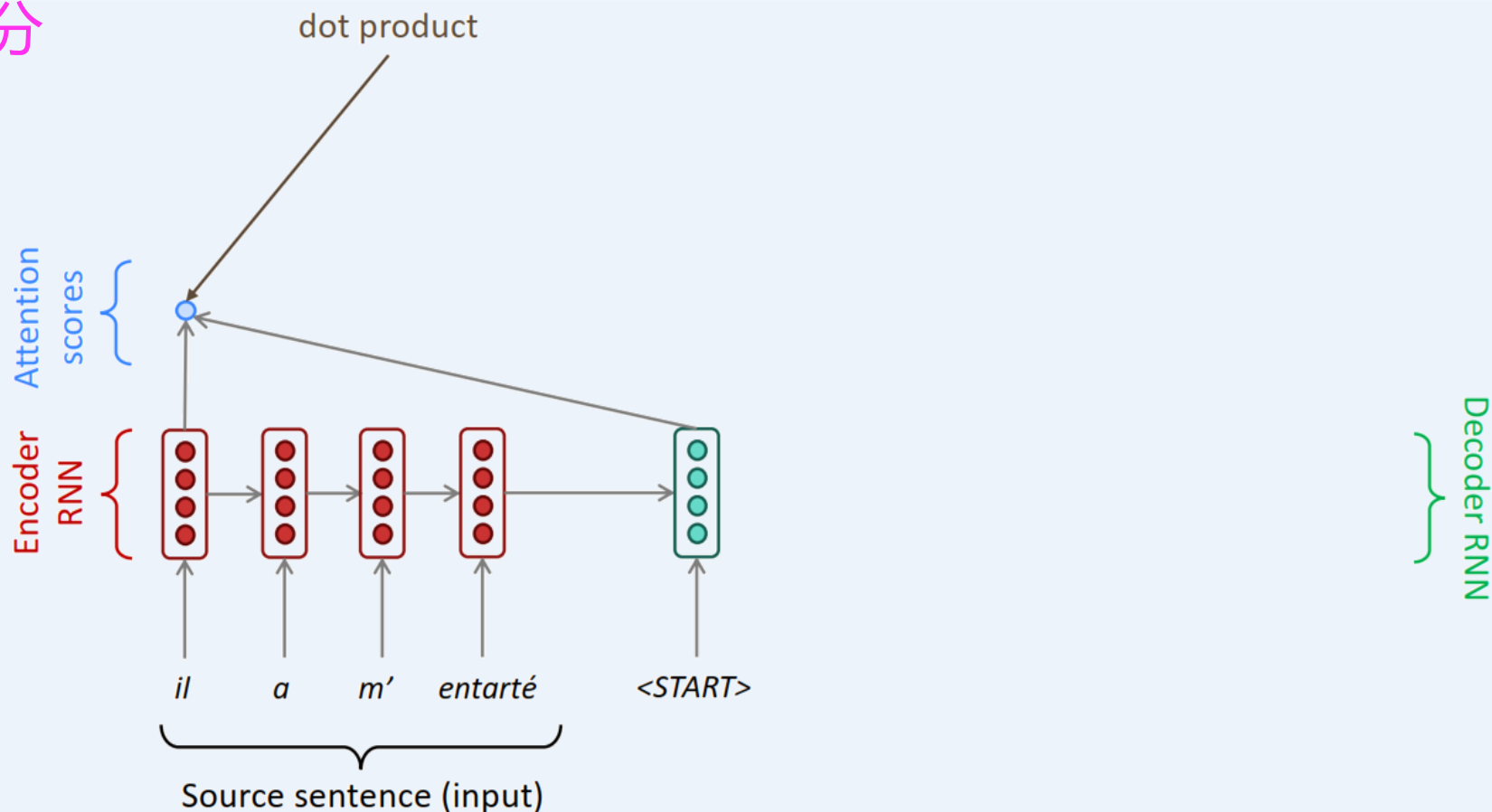
- 注意力提供了解决该瓶颈问题的一个方式
- **核心思想**：在解码器的每一步，建立直接连接到编码器，聚焦于源序列的特定部分



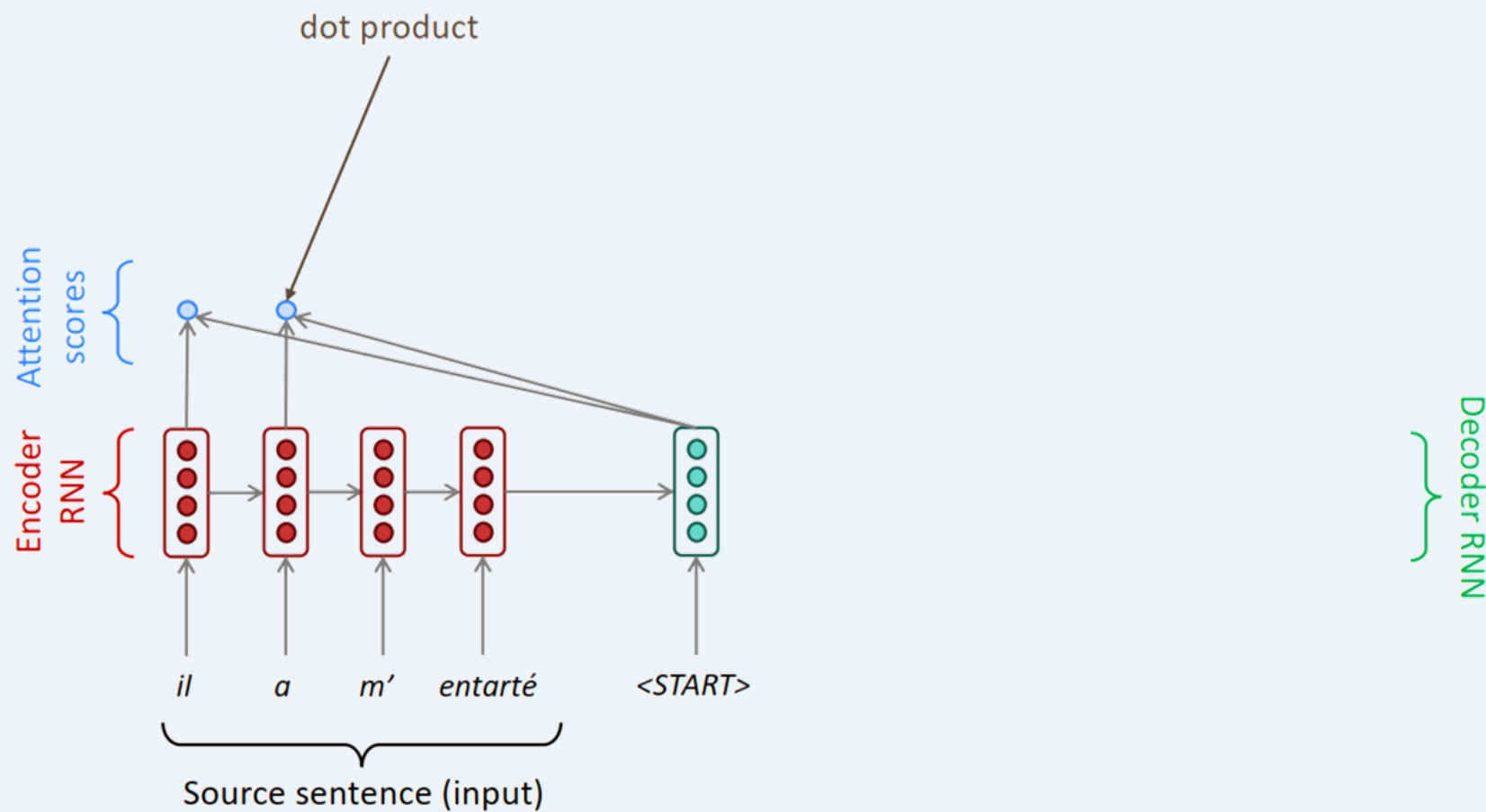
- 首先，我们通过图表展示，再形式化表示

Seq2Seq和注意力

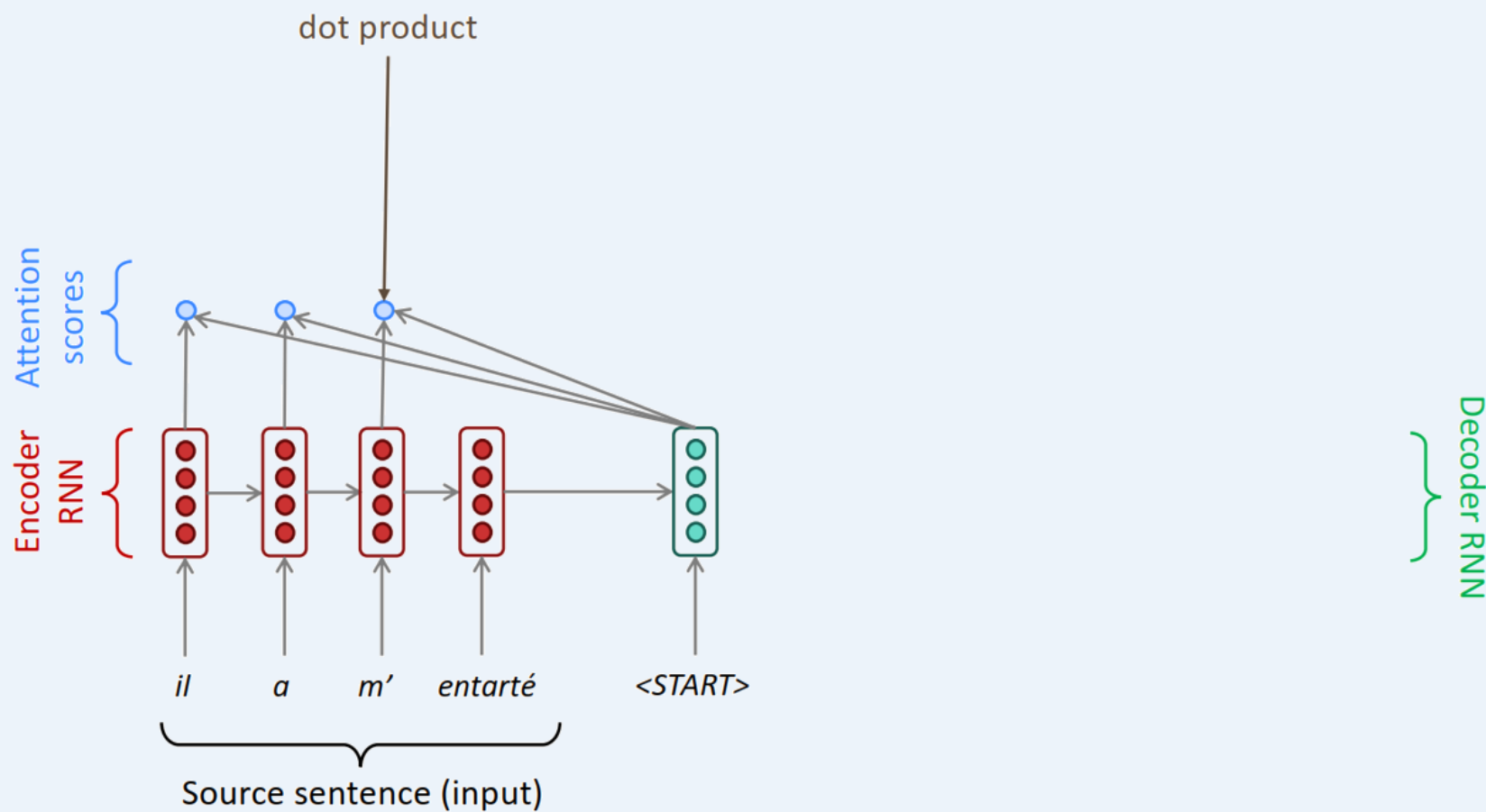
- **核心思想**：在解码器的每一步，建立直接连接到编码器，聚焦于源序列的特定部分



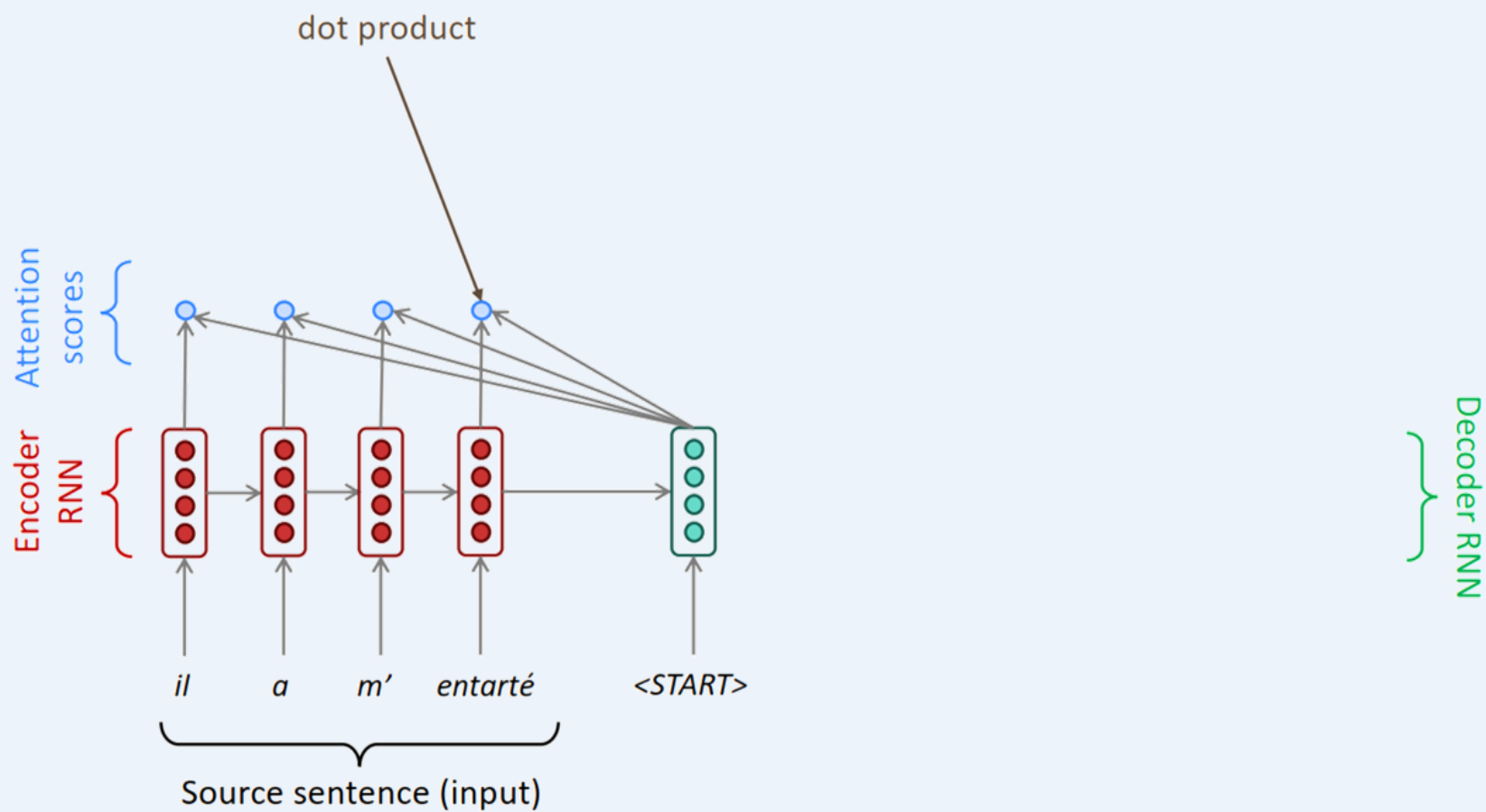
Seq2Seq和注意力



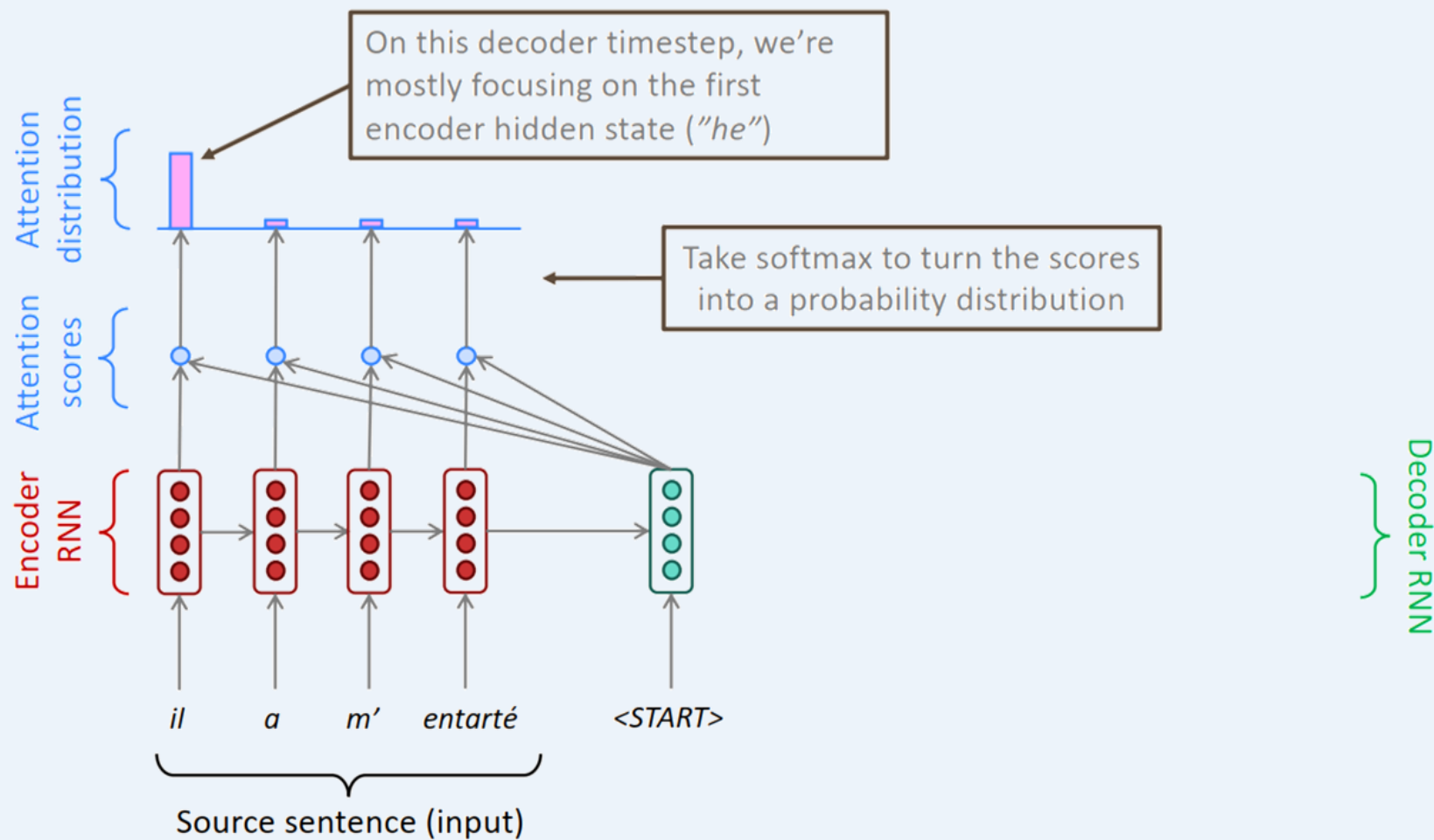
Seq2Seq和注意力



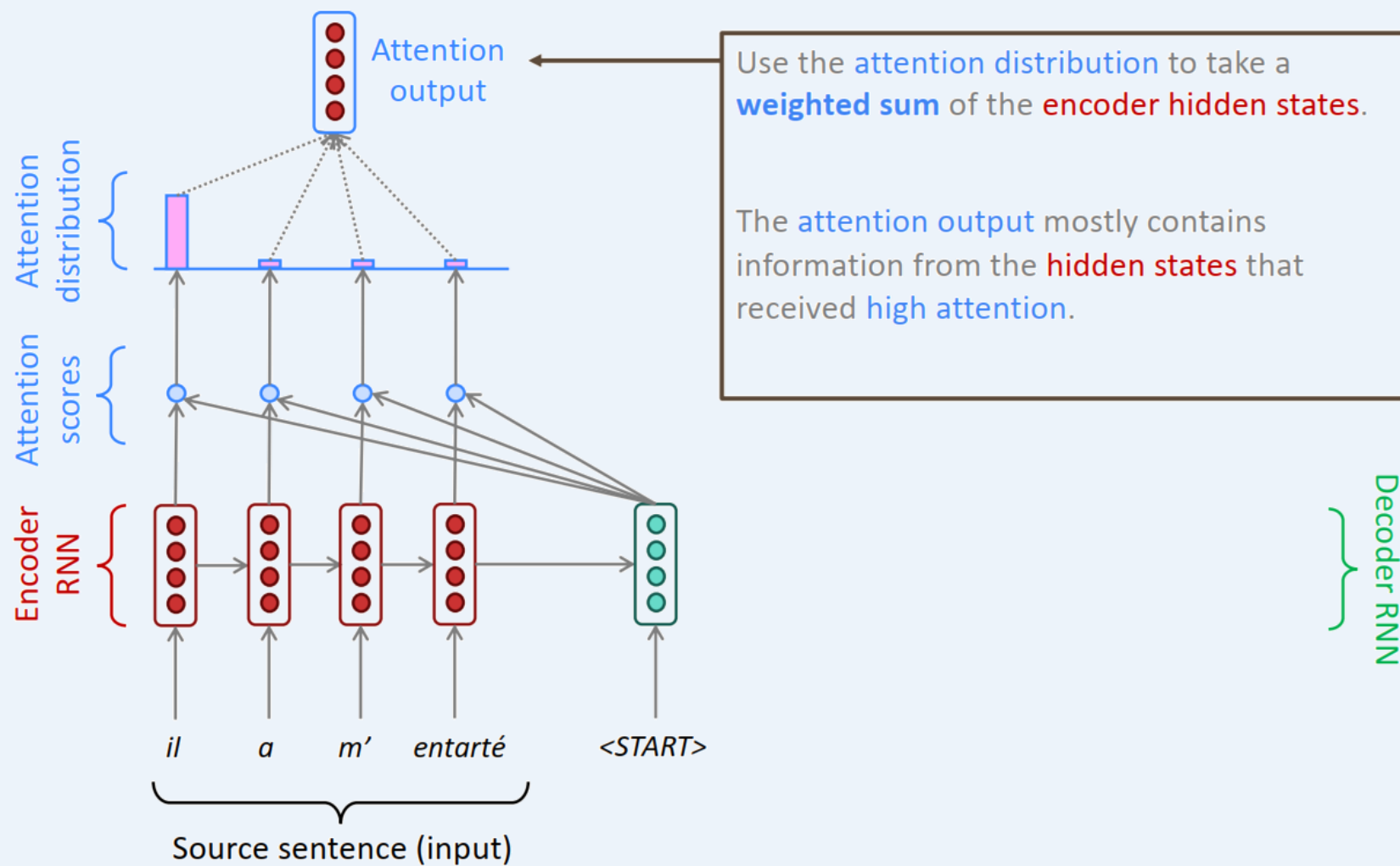
Seq2Seq和注意力



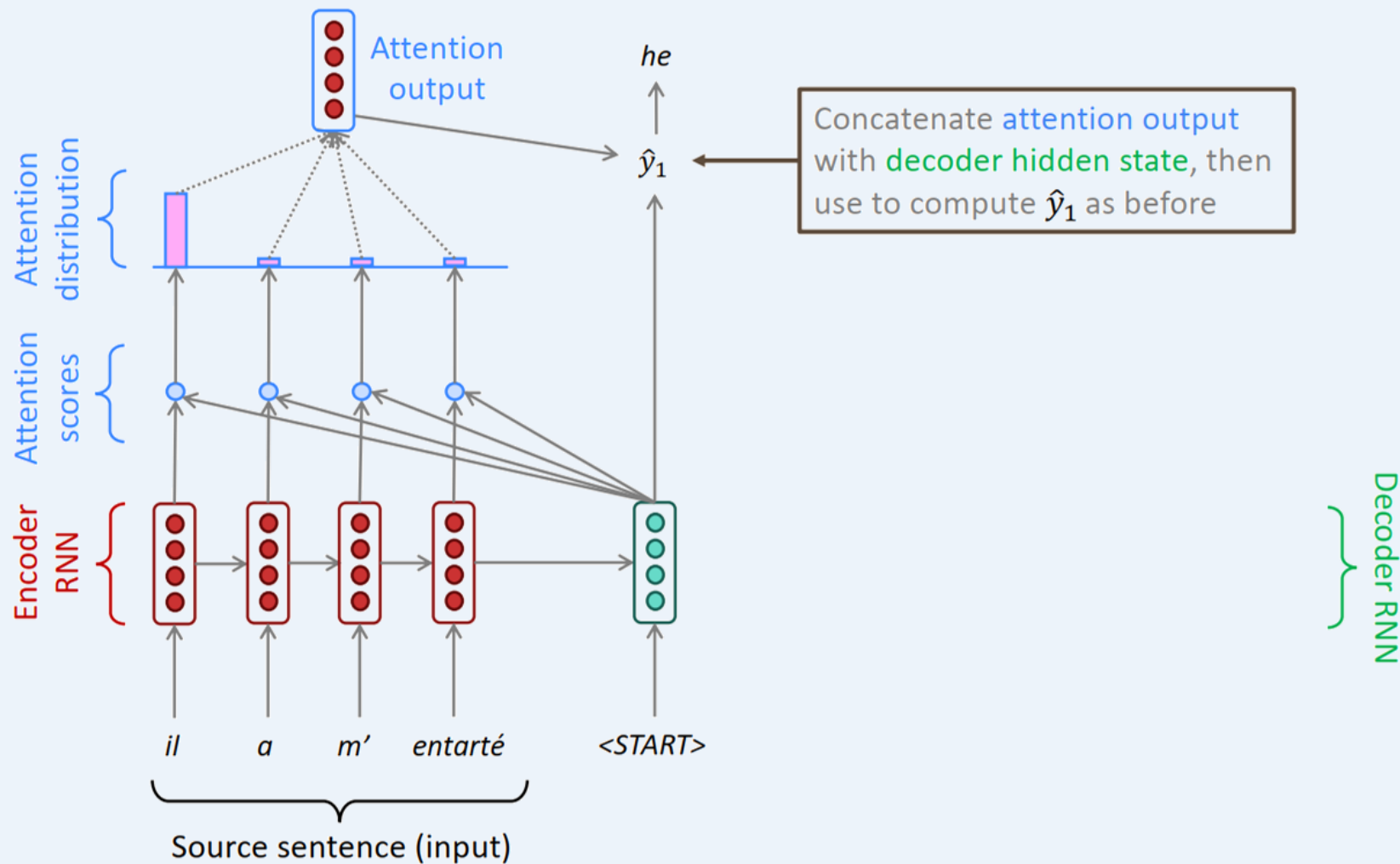
Seq2Seq和注意力



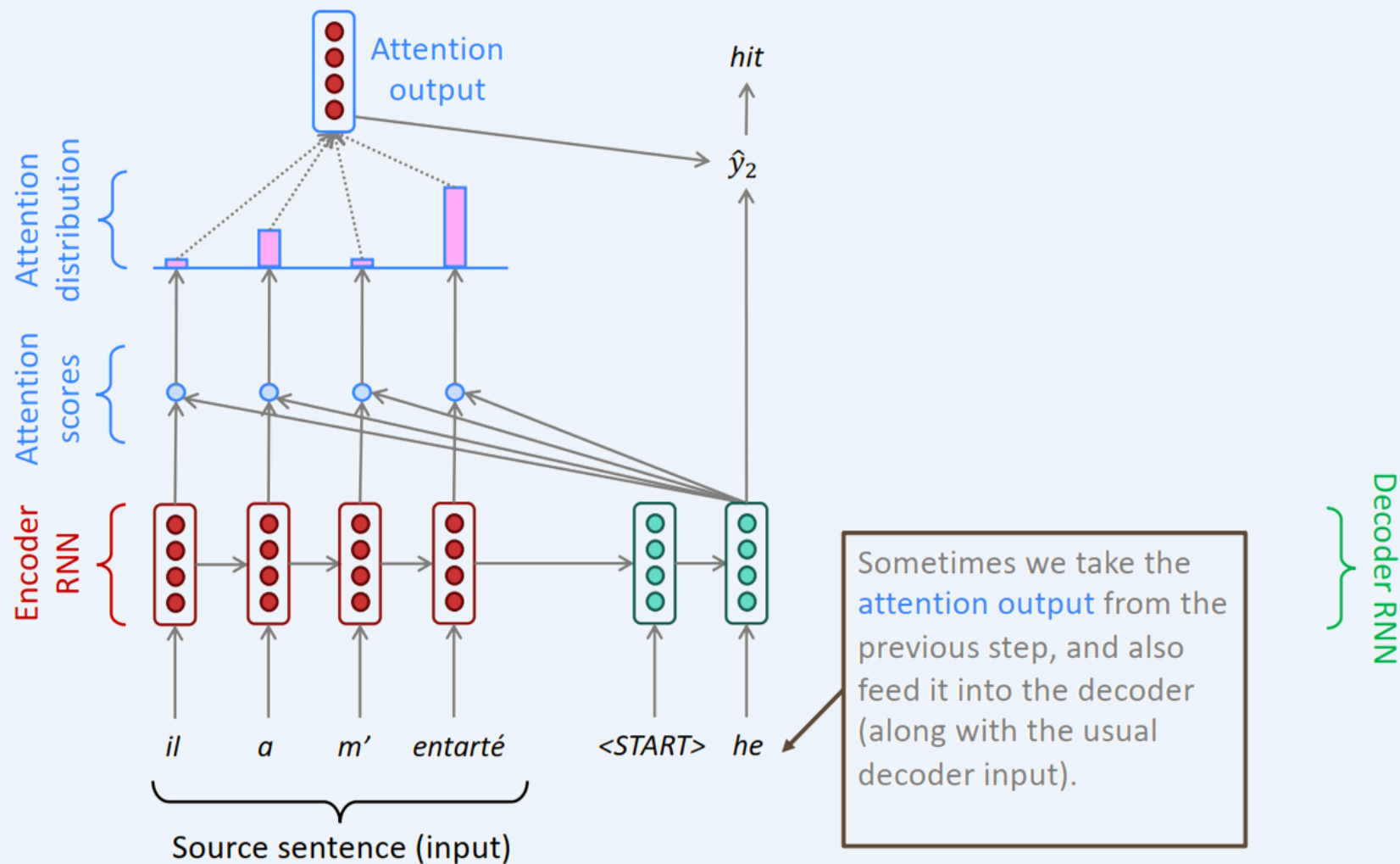
Seq2Seq和注意力



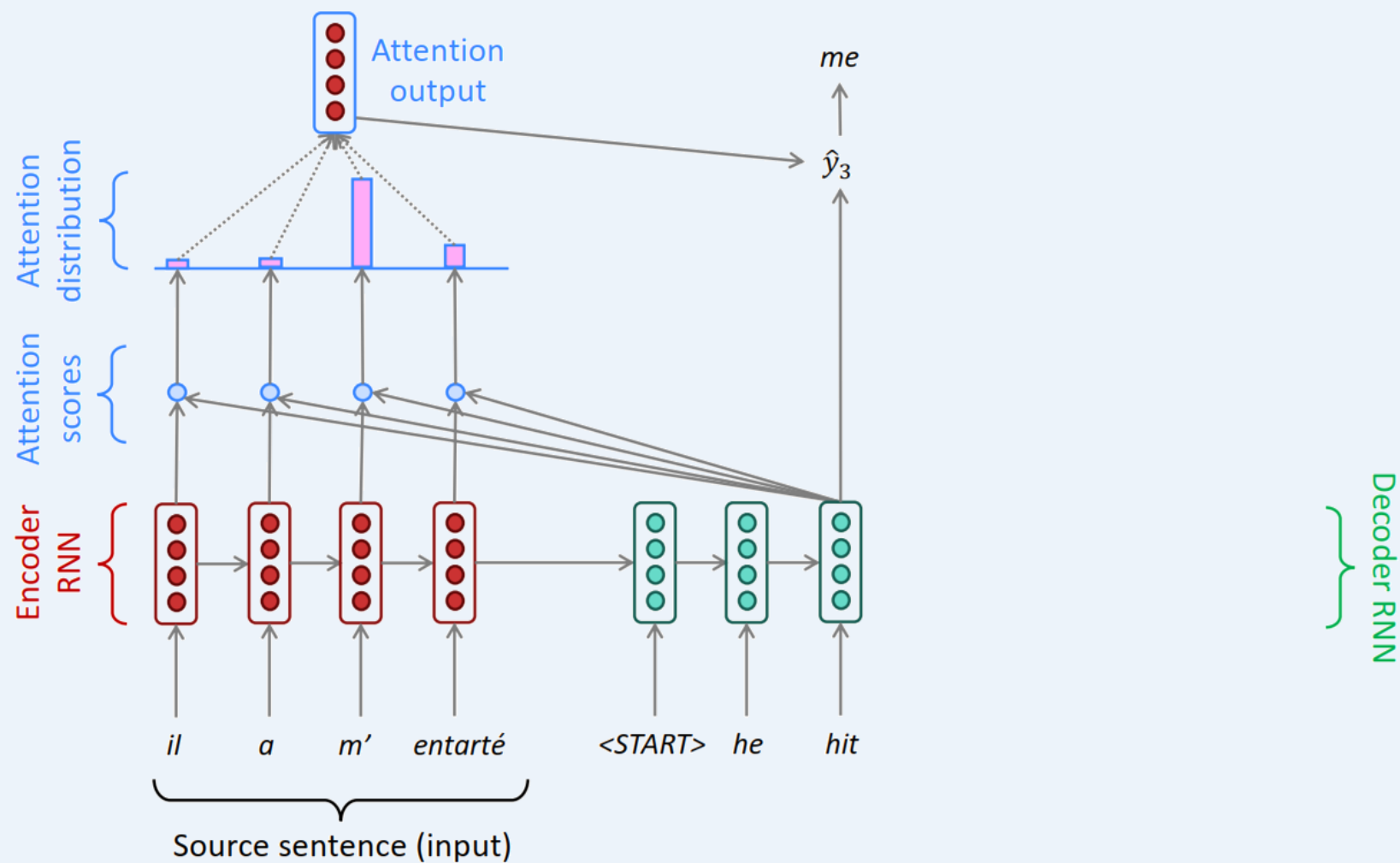
Seq2Seq和注意力



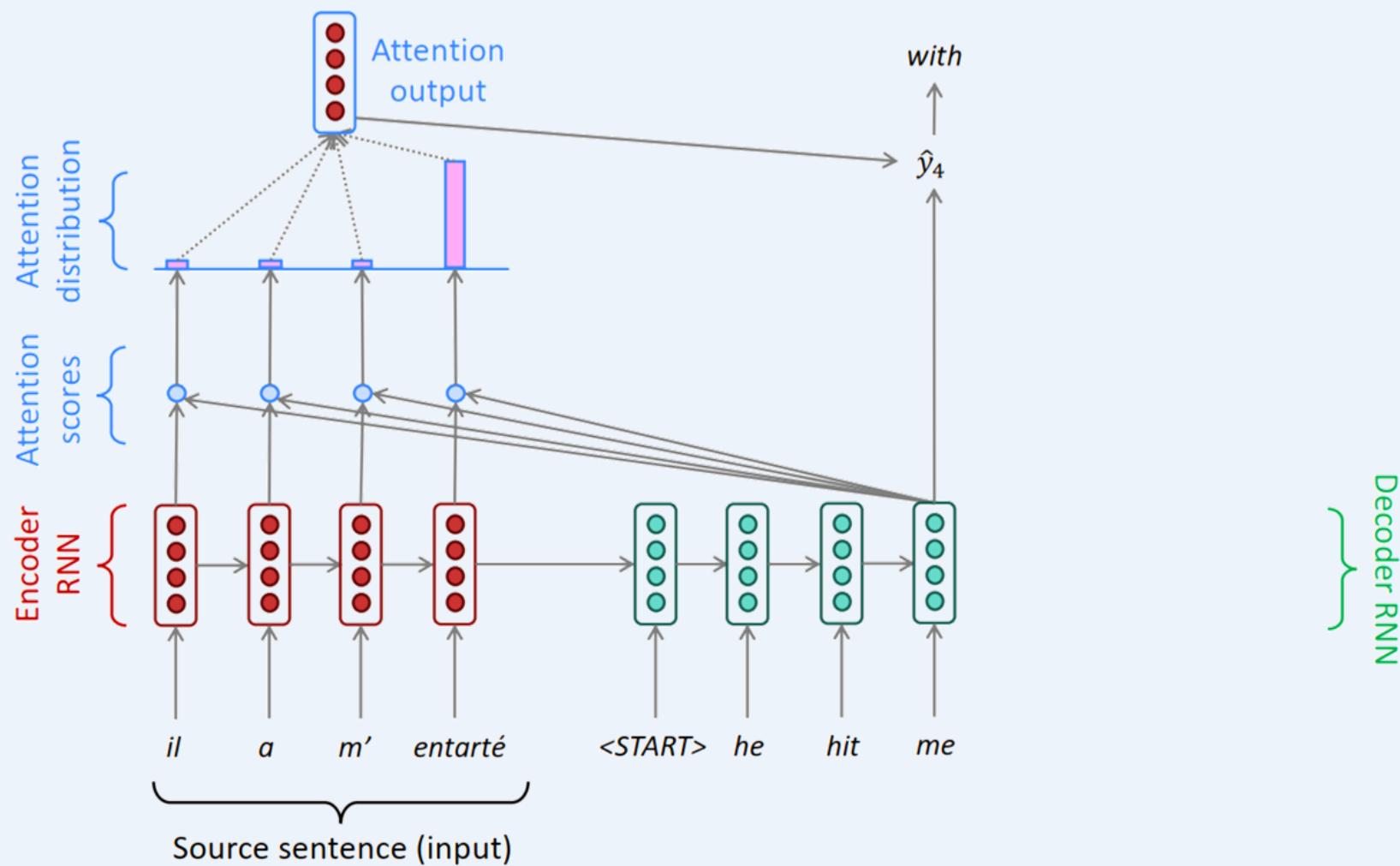
Seq2Seq和注意力



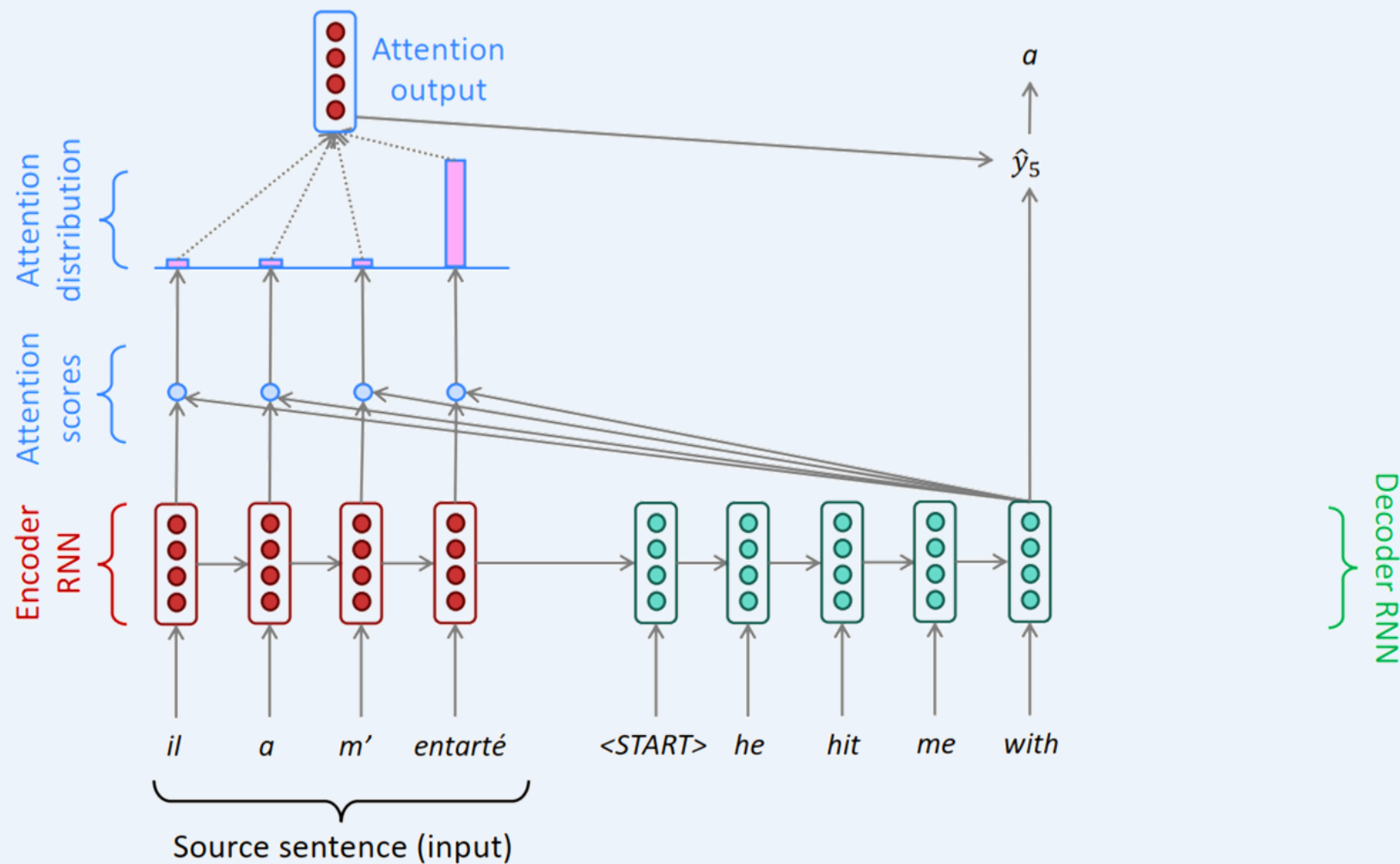
Seq2Seq和注意力



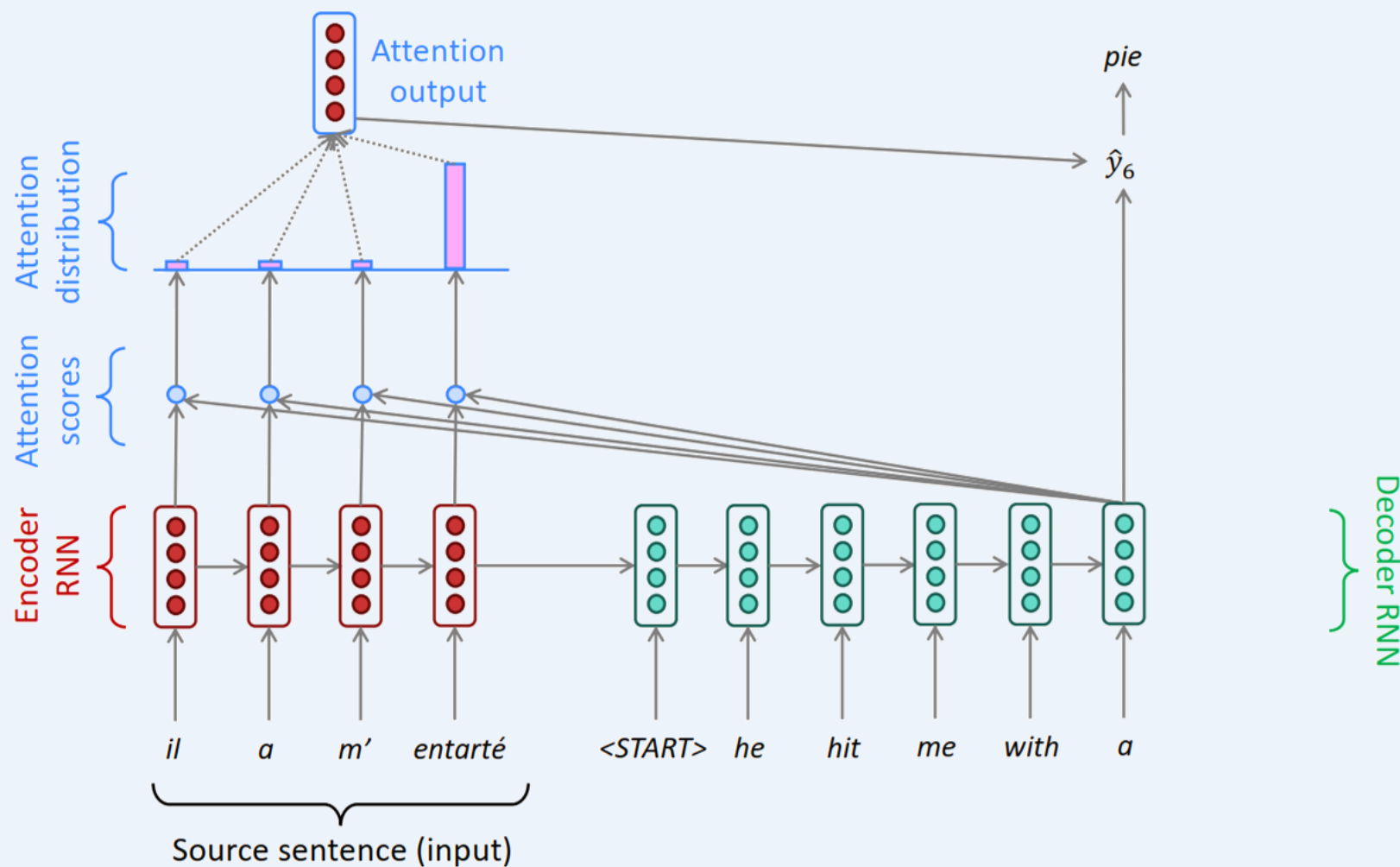
Seq2Seq和注意力



Seq2Seq和注意力



Seq2Seq和注意力



Seq2Seq和注意力

- 编码器隐藏状态: $\mathbf{h}_1, \dots, \mathbf{h}_N \in \mathbb{R}^h$
- 在时间步 t , 有解码器隐藏状态 $\mathbf{s}_t \in \mathbb{R}^h$
- 对这一步计算注意力分数 $\mathbf{e}^t$:

$$\mathbf{e}^t = [\mathbf{s}_t^T \mathbf{h}_1, \dots, \mathbf{s}_t^T \mathbf{h}_N] \in \mathbb{R}^N$$

- 对这一步用softmax来得到注意力分布 α^t (这是个概率分布且和为1):

$$\alpha^t = \text{softmax}(\mathbf{e}^t) \in \mathbb{R}^N$$

- 使用 α^t , 对编码器隐藏状态进行加权求和, 得到注意力输出 $\mathbf{a}_t$:

$$\mathbf{a}_t = \sum_{i=1}^N \alpha_i^t \mathbf{h}_i \in \mathbb{R}^h$$

- 最后将注意力输出 $\mathbf{a}_t$ 和解码器隐藏状态 $\mathbf{s}_t$ 拼接, 并和非注意力Seq2Seq模型一样继续:

$$[\mathbf{a}_t; \mathbf{s}_t] \in \mathbb{R}^{2h}$$

Seq2Seq和注意力

注意力的优势

- 注意力能够显著提升NMT的性能
 - 允许解码器专注于源句子的某些部分
- 注意 • 可以在翻译时回顾源句子，而无需记住所有内容
- 注意力解决了瓶颈问题
 - 注意力允许解码器直接查看源句子，绕开了瓶颈问题
- 注意力有助于解决梯度消失问题
 - 提供了捷径到更远的状态
- 注意力提供了一些可解释性
 - 可以通过检查注意力分布看到解码器关注的内容
 - 可以直接获得(软)对齐
 - 因为从未明确训练过对齐系统，所以这是很有用的
 - 网络只是自己学会了对齐

注意力的通用性

注意力是一种通用的深度学习技术

- 我们已经看到，注意力是改进机器翻译的序列到序列模型的好方法。
- 但是：注意力在**很多架构**中都能够使用
(不仅仅是seq2seq) 和**很多任务** (不只是MT)

- **对注意力更一般的定义：**

- 给定一个向量集合 *values*， 以及一个向量 *query*， **注意力**在*query*上， 计算*values*加权和的技术

- 有时称为**query对values的注意力聚焦**。
- 例如， 在 seq2seq+attention 模型中， 每个解码器隐藏状态（query）
关注所有编码器隐藏状态（values）

注意力的通用性

注意力是一种通用的深度学习技术

- **对注意力更一般的定义:**

- 给定一个向量集合 *values*, 以及一个向量 *query*, **注意力**在*query*上, 计算*values*加权和的技术

直观上:

- 加权和运算本质是对*values*向量信息的**选择性摘要**, 由*query*向量动态决定关注焦点
- 注意力机制实现了**任意规模输入→定长表征**的转换(*values*向量), 其信息压缩过程始终以*query*向量为条件

核心结论:

- 注意力已成为深度学习模型中强大、灵活、普适的指路牌与记忆操作范式 (2010年后源自神经机器翻译的重大创新)

Transformer

Transformers: Is Attention All We Need? (2017)

- 我们知道，注意力可以显著提高循环神经网络的性能。
- 今天，我们将更进一步，问：Is All Attention We Need?
- **剧透**：还差一点！

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Łukasz Kaiser*
Google Brain
lukaszkaiser@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

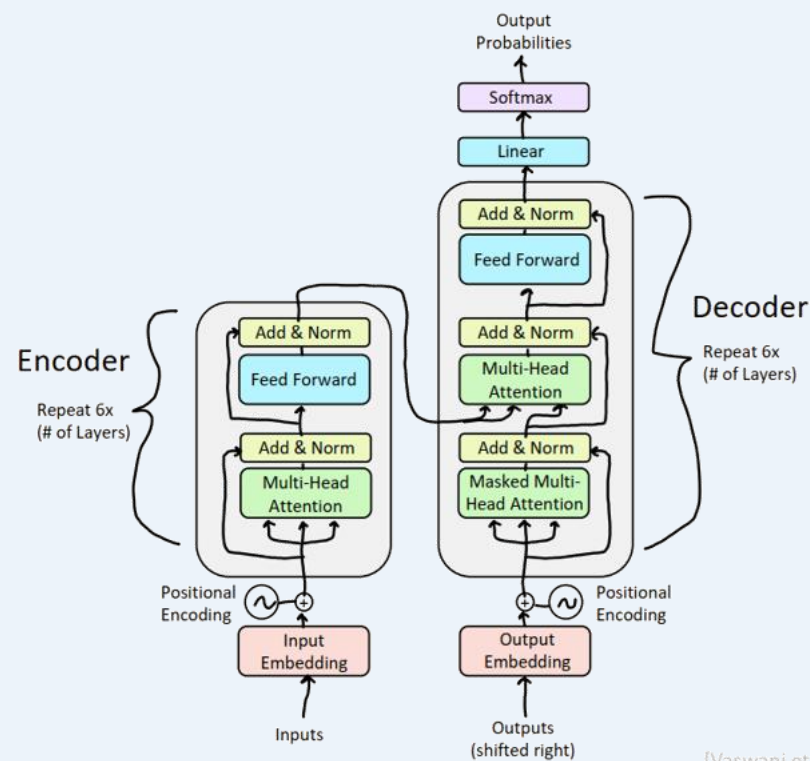
Transformer

Transformers是NLP领域的重大革新

- 几乎支撑了现如今所有最先进NLP模型的神经网络架构



Courtesy of Paramount Pictures



[Vaswani et al., 2017]

Transformer的表现：机器翻译

First, Machine Translation results from the original Transformers paper!

Model	BLEU		Training Cost (FLOPs)	
	EN-DE	EN-FR	EN-DE	EN-FR
ByteNet [18]	23.75			
Deep-Att + PosUnk [39]		39.2		$1.0 \cdot 10^{20}$
GNMT + RL [38]	24.6	39.92	$2.3 \cdot 10^{19}$	$1.4 \cdot 10^{20}$
ConvS2S [9]	25.16	40.46	$9.6 \cdot 10^{18}$	$1.5 \cdot 10^{20}$
MoE [32]	26.03	40.56	$2.0 \cdot 10^{19}$	$1.2 \cdot 10^{20}$
Deep-Att + PosUnk Ensemble [39]		40.4		$8.0 \cdot 10^{20}$
GNMT + RL Ensemble [38]	26.30	41.16	$1.8 \cdot 10^{20}$	$1.1 \cdot 10^{21}$
ConvS2S Ensemble [9]	26.36	41.29	$7.7 \cdot 10^{19}$	$1.2 \cdot 10^{21}$
Transformer (base model)	27.3	38.1	$3.3 \cdot 10^{18}$	
Transformer (big)	28.4	41.8	$2.3 \cdot 10^{19}$	

[Test sets: WMT 2014 English-German and English-French]

[Vaswani et al., 2017]

Transformer的表现：大预言模型崛起

LMSYS Chatbot Arena（大模型竞技场）是由大型模型系统组织（LMSYS Org，由加州大学伯克利分校等机构的研究人员组成）开发的一个开源的大语言模型（LLM）盲测众测平台。

Today, Transformer-based models dominate LMSYS Chatbot Arena Leaderboard!

Rank	Model	Arena Elo	95% CI	Votes	Organization	License	Knowledge Cutoff
1	GPT-4-Turbo-2024-04-09	1258	+4/-4	26444	OpenAI	Proprietary	2023/12
1	GPT-4-1106-preview	1253	+3/-3	68353	OpenAI	Proprietary	2023/4
1	Claude 3 Opus	1251	+3/-3	71500	Anthropic	Proprietary	2023/8
2	Gemini 1.5 Pro API-0409-Preview	1249	+4/-5	22211	Google	Proprietary	2023/11
3	GPT-4-0125-preview	1248	+2/-3	58959	OpenAI	Proprietary	2023/12
6	Meta Llama 3 70b Instruct	1213	+4/-6	15809	Meta	Llama 3 Community	2023/12
6	Bard (Gemini Pro)	1208	+7/-6	12435	Google	Proprietary	Online
7	Claude 3 Sonnet	1201	+4/-2	73414	Anthropic	Proprietary	2023/8



Gemini / Bard
(Google)



ChatGPT / GPT-4
(OpenAI)



Claude 3
(Anthropic)

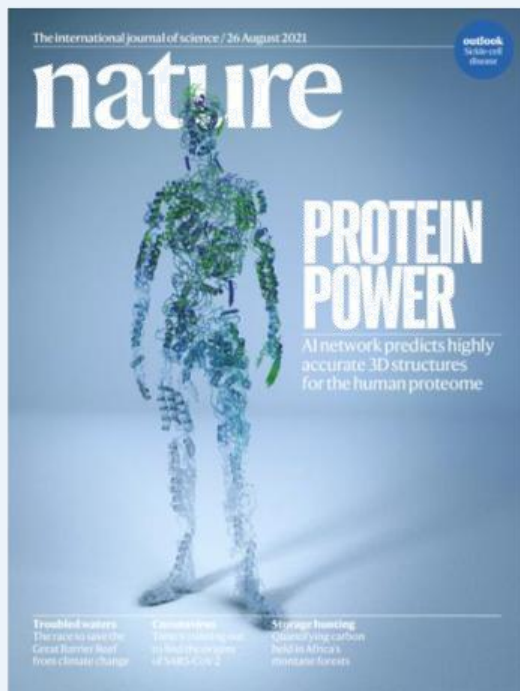


Llama 3
(Meta)

[Chiang et al., 2024]

Transformer的表现：NLP之外

Protein Folding



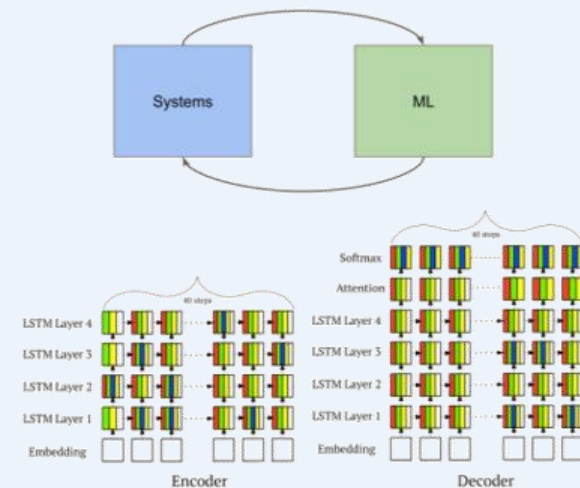
[Jumper et al. 2021] aka AlphaFold2!



Image Classification

[Dosovitskiy et al. 2020]: Vision Transformer (ViT) outperforms ResNet-based baselines with substantially less compute.

	Ours-JFT (ViT-H/14)	Ours-JFT (ViT-L/16)	Ours-I21k (ViT-L/16)	BiT-L (ResNet152x4)	Noisy Student (EfficientNet-L2)
ImageNet	88.55 ± 0.04	87.76 ± 0.03	85.30 ± 0.02	87.54 ± 0.02	88.4/88.5*
ImageNet Real.	90.72 ± 0.05	90.54 ± 0.03	88.62 ± 0.05	90.54	90.55
CIFAR-10	99.50 ± 0.06	99.42 ± 0.03	99.15 ± 0.03	99.37 ± 0.06	—
CIFAR-100	94.55 ± 0.04	93.90 ± 0.05	93.25 ± 0.05	93.51 ± 0.08	—
Oxford-IIIT Pets	97.56 ± 0.03	97.32 ± 0.11	94.67 ± 0.15	96.62 ± 0.23	—
Oxford Flowers-102	99.68 ± 0.02	99.74 ± 0.00	99.61 ± 0.02	99.63 ± 0.03	—
VTAB (19 tasks)	77.63 ± 0.23	76.28 ± 0.46	72.72 ± 0.21	76.29 ± 1.70	—
TPUv3-core-days	2.5k	0.68k	0.23k	9.9k	12.3k



ML for Systems

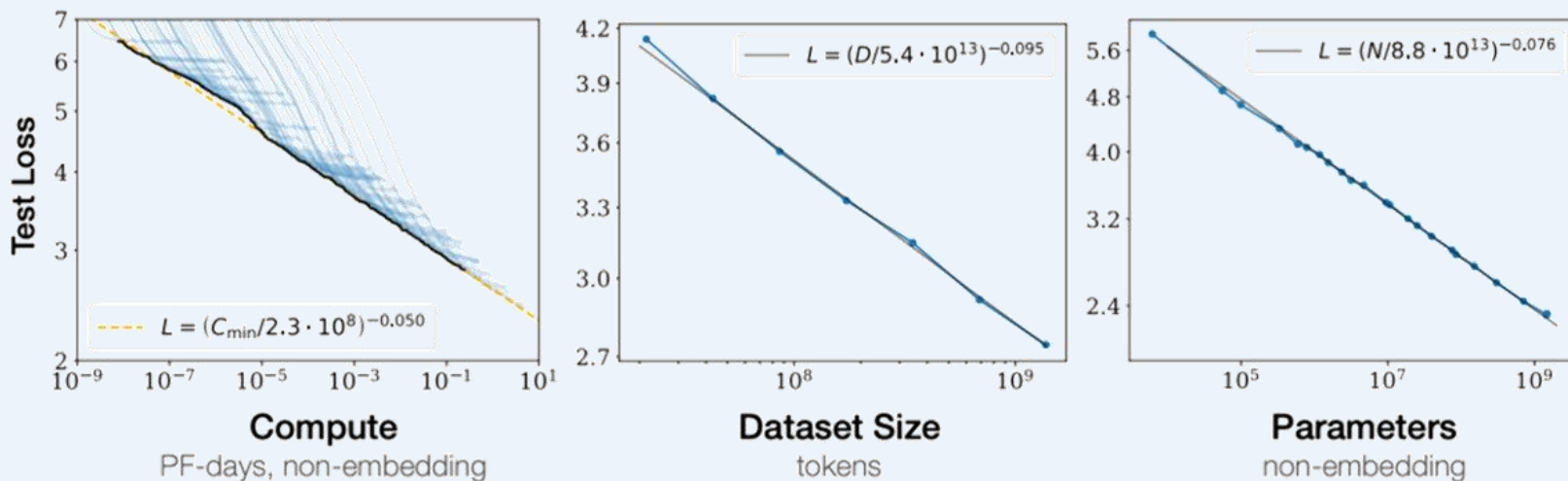
[Zhou et al. 2020]: A Transformer-based compiler model (GO-one) speeds up a Transformer model!

Model (#devices)	GO-one (s)	HP (s)	METIS (s)	HDP (s)	Run time speed up over HP / HDP	Search speed up over HDP
2-layer RNNLM (2)	0.173	0.192	0.355	0.191	9.9% / 9.4%	2.9x
4-layer RNNLM (4)	0.210	0.239	0.503	0.251	13.8% / 16.3%	1.76x
8-layer RNNLM (8)	0.320	0.332	OOM	0.764	3.8% / 58.1%	27.8x
2-layer GNNMT (2)	0.301	0.384	0.344	0.327	27.6% / 14.3%	30x
4-layer GNNMT (4)	0.350	0.469	0.466	0.432	34% / 23.4%	58.8x
8-layer GNNMT (8)	0.440	0.562	OOM	0.693	21.7% / 36.5%	7.35x
2-layer Transformer-XL (2)	0.223	0.268	0.37	0.262	20.1% / 17.4%	40x
4-layer Transformer-XL (4)	0.230	0.27	OOM	0.259	17.4% / 12.8%	26.7x
8-layer Transformer-XL (8)	0.350	0.46	OOM	0.425	23.9% / 16.7%	16.7x
Inception (2) b64	0.229	0.312	OOM	0.301	26.6% / 23.9%	13.5x
AmoebaNet (4)	0.423	0.731	OOM	0.498	42.1% / 29.3%	21.0x
2-stack 18-layer WaveNet (2)	0.394	0.44	0.426	0.418	26.1% / 6.1%	58.8x
4-stack 36-layer WaveNet (4)	0.317	0.376	OOM	0.354	18.6% / 11.7%	6.67x
GEOMEAN	0.659	0.988	OOM	0.721	50% / 9.4%	20x
	-	-	-	-	26.5% / 18.2%	15x

Transformer: 尺度定律(Scaling Law)

Transformer就是我们所需要的吗?

- 使用 Transformer, 随着我们同时增加模型大小、训练数据和计算资源, 语言建模性能可以顺利提高
- 这种幂律关系已经在多个数量级上被观察到, 而且没有放缓的迹象!
- 如果我们继续扩展这些模型 (不改变架构), 它们最终能否达到或超过人类水平的性能?



[Kaplan et al., 2020]

Transformer的动机

Transformer的作者在设计这个架构时有3个想法：

1. 最小化（至少别增加）每层的计算复杂度
2. 最小化任何一对单词之间的路径长度，以促进学习长距离依赖关系
3. 最大化可并行化的计算量

Transformer的动机

1. 每层的计算复杂度

当时，与我们目前了解的RNN模型相比，Transformer 的每层复杂度较低。When sequence length (n) $\ll$ representation dimension (d), complexity per layer is lower for a Transformer compared to the recurrent models we've learned about so far.

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

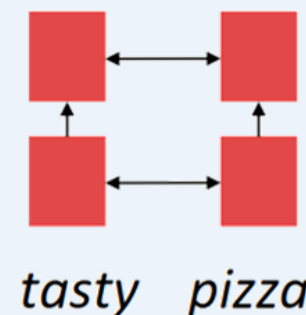
Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

Table 1 of the Transformer paper.

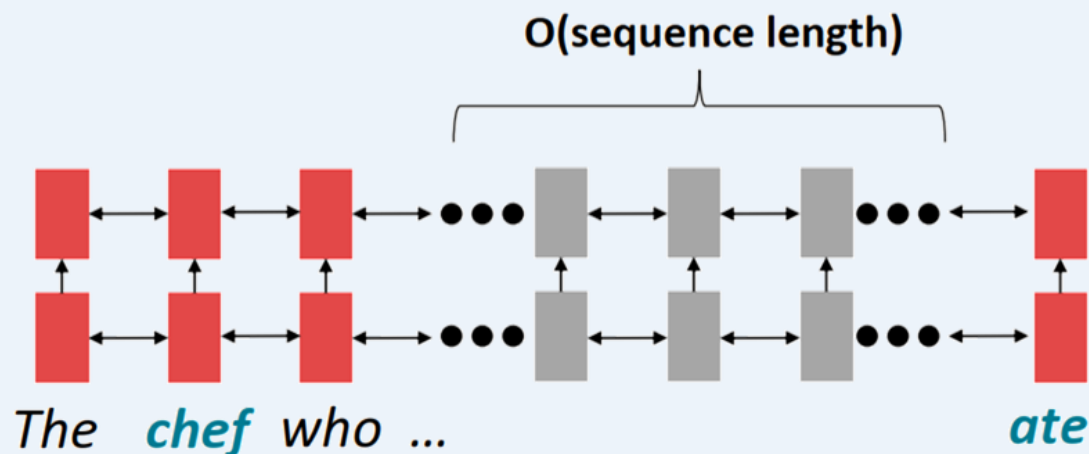
Transformer的动机

2. 最小化线互距离

- RNN是“从左到右”展开的
- 编码线性局部性，是一种有用的启发式方法
 - 相邻的单词通常会相互影响彼此的含义



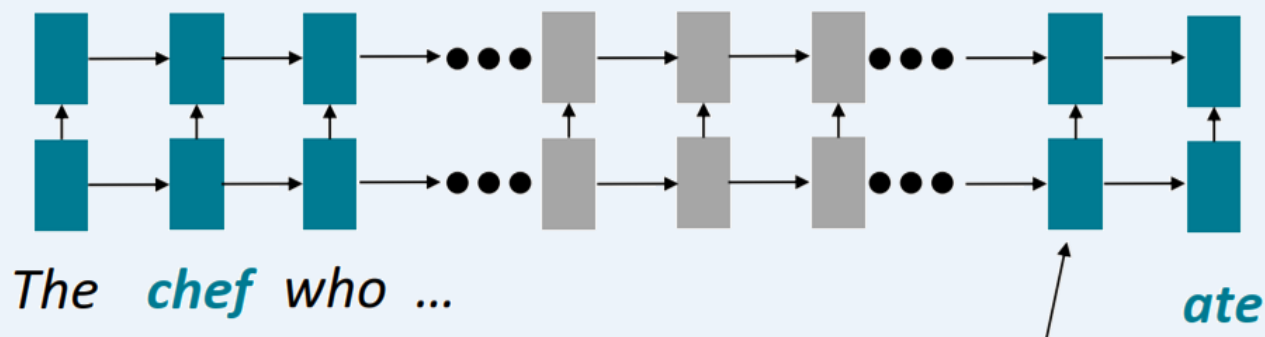
- **问题:** RNN需要 $O(\text{sequence length})$ 的步数来使相隔较远的单词交互



Transformer的动机

2. 最小化线互距离

- $O(\text{sequence length})$ 的步数来使相隔较远的单词交互，意味着：
- 难以学习到长距离依赖(因为梯度问题)
 - 单词的线性顺序是“内置”的，我们知道顺序结构并不能说明所有情况

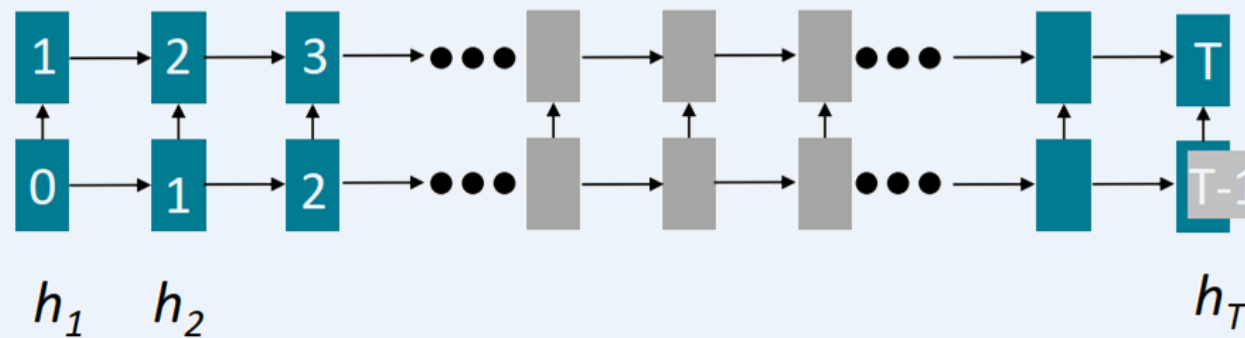


Info of *chef* has gone through $O(\text{sequence length})$ many layers!

Transformer的动机

3. 最大化可并行性

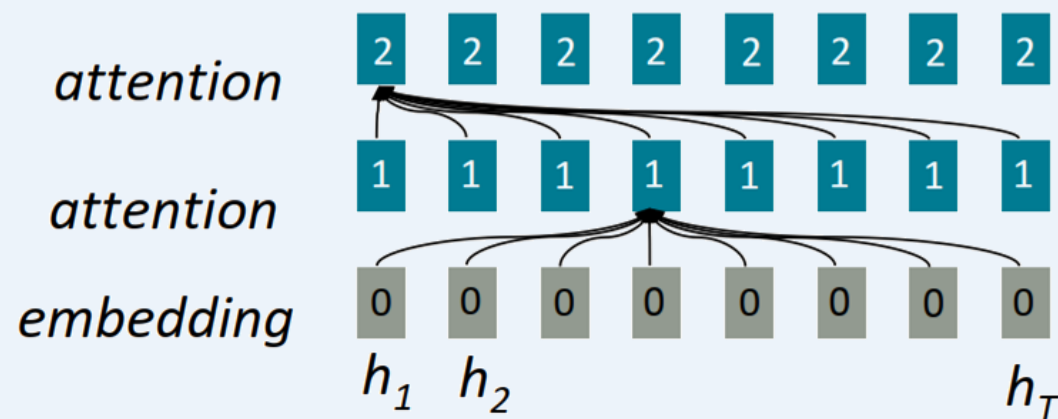
- 前向和后向传播有 $O(\text{seq length})$ 个无法并行化的操作
 - GPU (和 TPU) 允许一次执行许多独立计算
 - 但是, 在计算过去的 RNN 隐藏状态之前, 不能完全计算未来的 RNN 隐藏状态
 - 难以在非常大的数据集上进行训练
 - 随着序列长度的增加, 问题尤其严重, 因为由于内存限制, 无法再将更多样本组合成batch



Numbers indicate min # of steps before a state can be computed

Transformer与(自)注意力

- 概括地说，**注意力**将每个单词的表示形式视为一个query，用于访问和合并一组values中的信息。
 - 之前我们从递归的Seq2Seq模型中了解到了**解码器**对**编码器**的关注
 - **自注意力(Self-attention)**是**编码器-编码器** (或**解码器-解码器**) 的注意力，其中，每个单词都关注输入（或输出）中的其他单词

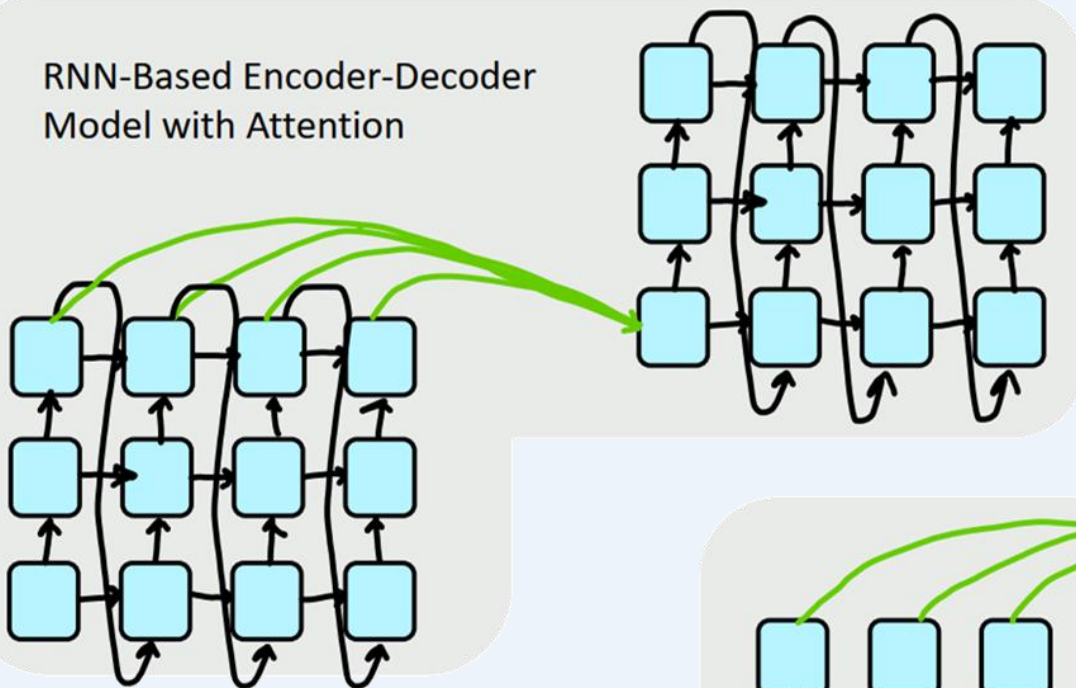


All words attend to all words in previous layer; most arrows here are omitted

计算依赖关系

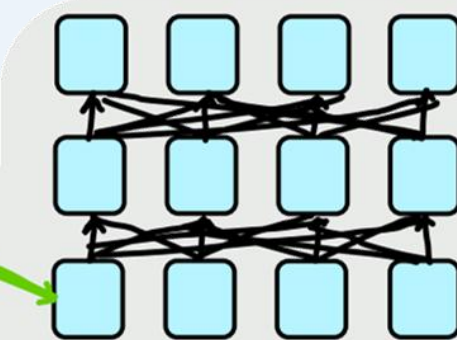
递归(recurrence)vs.注意力(attention)的计算依赖关系

RNN-Based Encoder-Decoder Model with Attention



Transformer的优点:

- 不可并行操作的数量并不会随着序列的长度上升
- 每个“单词”可以与其他交互, 因此最大交互距离为 $O(1)$ 。

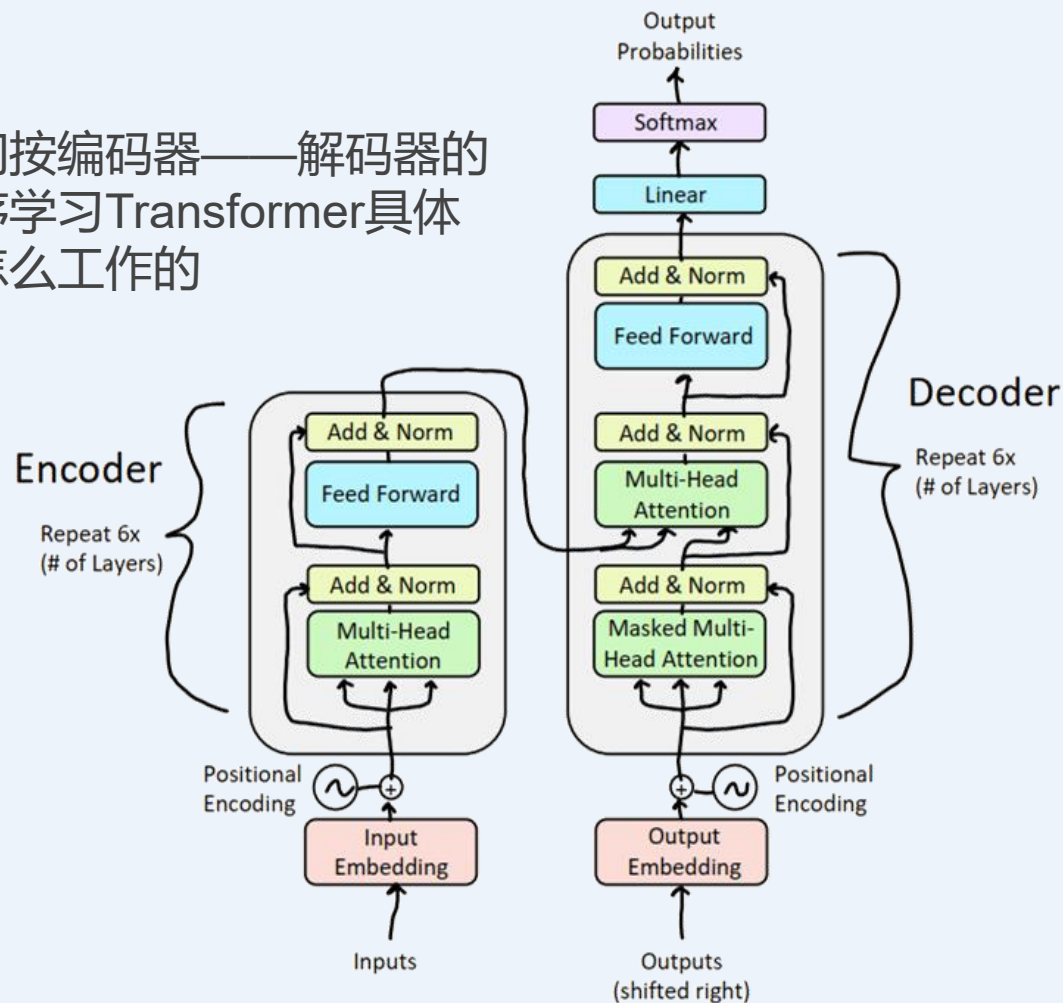


Transformer-Based Encoder-Decoder Model

Transformer编码器-解码器

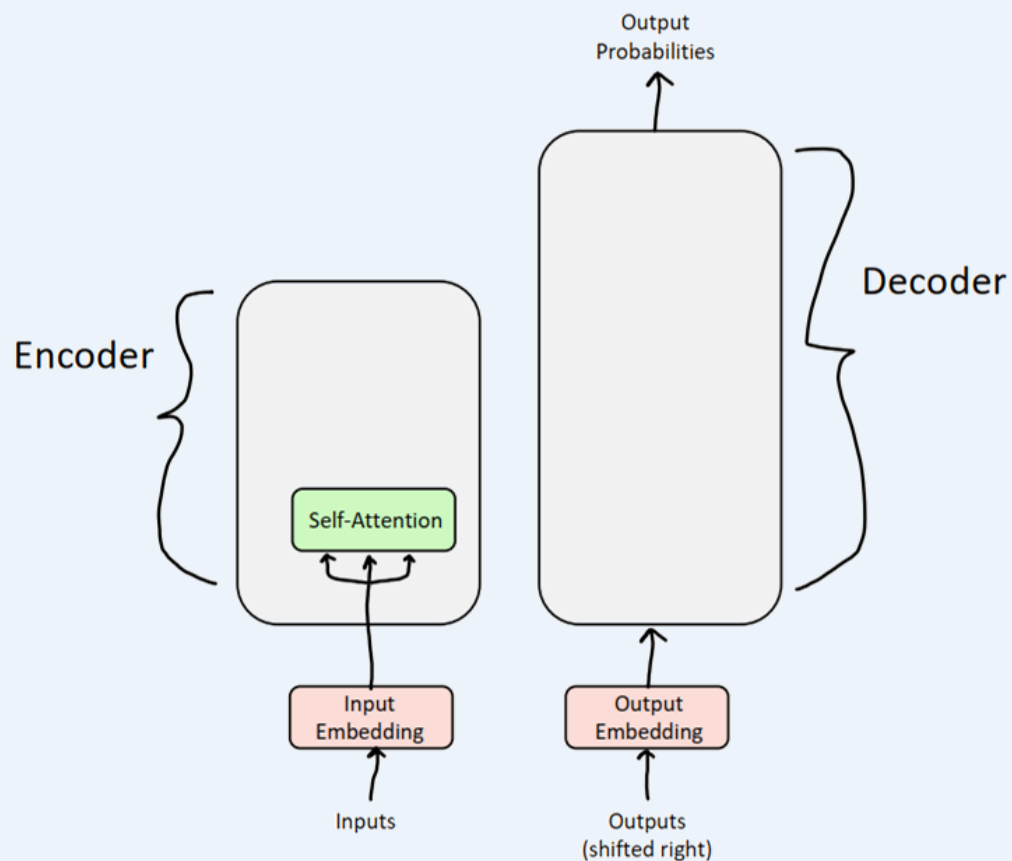
[Vaswani et al., 2017]

我们按编码器——解码器的顺序学习Transformer具体是怎么工作的



编码器：自注意力

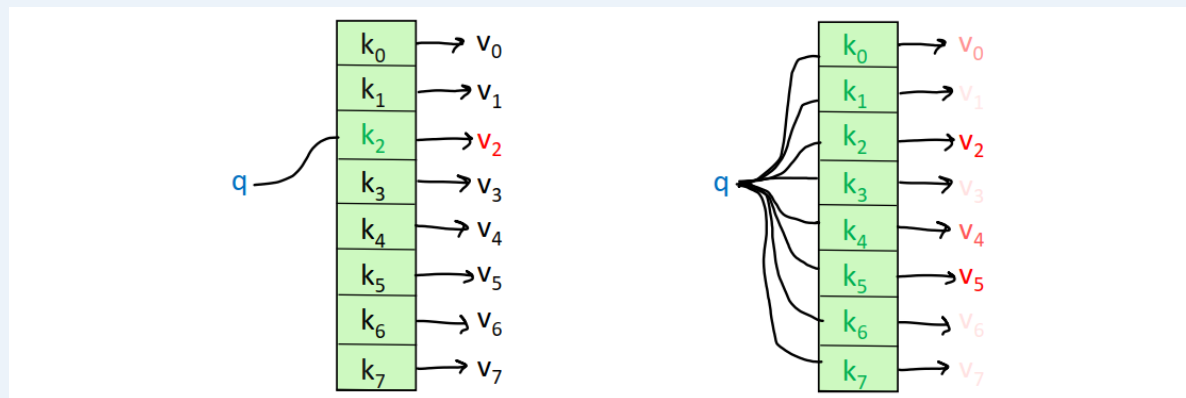
自注意力为Transformer的核心构建块



注意力机制：直观

把注意力当成一个“模糊的”或近似的哈希表：

- 为了查询一个 **value**，我们需要把 **query** 和表中的 **keys** 进行比较
- 在哈希表中(如左下图所示):
 - 每个 **query**(哈希) 映射到正好一个 **key-value** 对
- 在(自)注意力中(如右下图所示):
 - 每个 **query** 和每个 **key** 在不同程度上匹配
 - 以 **query-key** 的匹配程度为权重，返回 **values** 的和



Transformer编码器的自注意力

- 步骤1: 对每个单词 x_i , 计算它的**query**、**key**和**value**:

$$q_i = W^Q x_i \quad k_i = W^K x_i \quad v_i = W^V x_i$$

- 步骤2: 计算**query**和**key**之间的注意力分数:

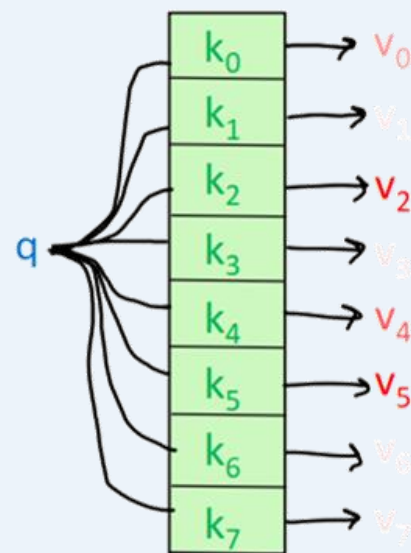
$$e_{ij} = q_i \cdot k_j$$

- 步骤3: 使用softmax对注意力分数进行归一化:

$$\alpha_{ij} = \text{softmax}(e_{ij}) = \frac{\exp(e_{ij})}{\sum_k \exp(e_{ik})}$$

- 步骤4: 计算**value**的加权和:

$$\text{output}_i = \sum_j \alpha_{ij} v_j$$



Transformer编码器的自注意力

向量化的表示形式

- 步骤1: 将词嵌入堆叠为 X , 计算**query**、**key**和**value**:

$$Q = XW^Q \quad K = XW^K \quad V = XW^V$$

- 步骤2: 计算**query**和**key**之间的注意力分数:

$$E = QK^T$$

- 步骤3: 使用softmax对注意力分数进行归一化:

$$Att = softmax(E)$$

- 步骤4: 计算**value**的加权和:

$$Output = AttV$$

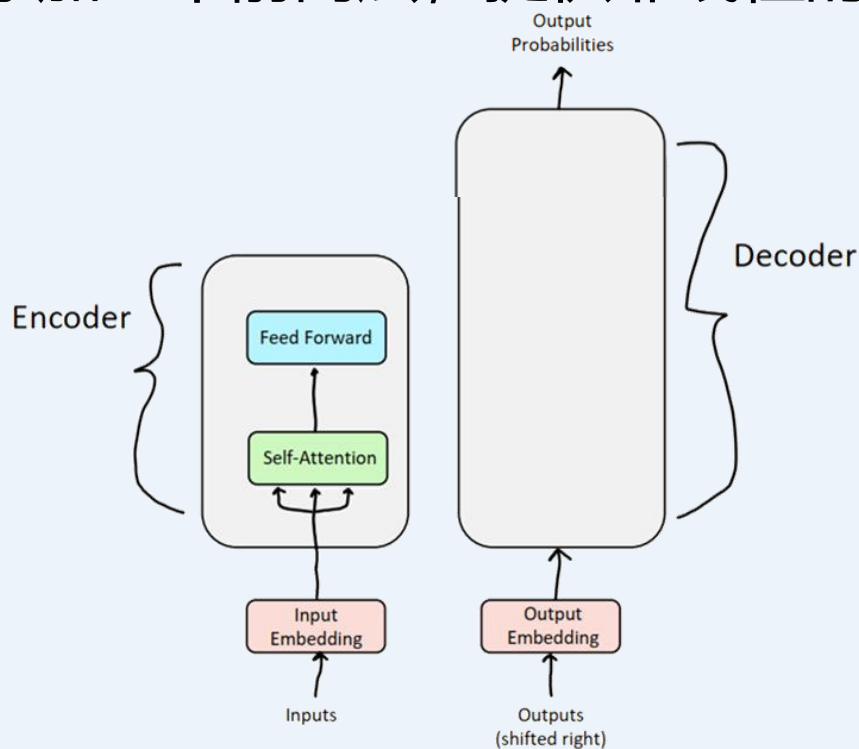
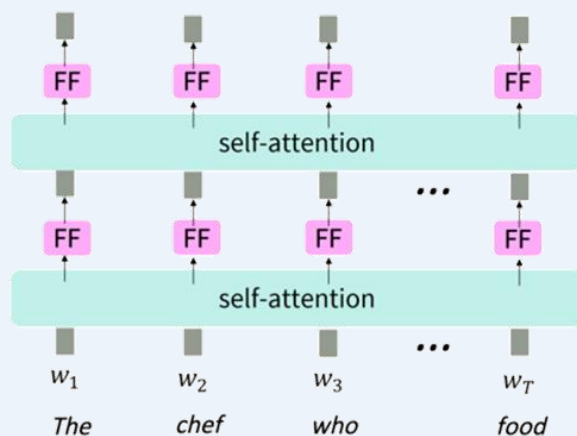
$$Output = softmax(QK^T) V$$

Transformer并非万能

- **问题:** 由于没有元素级非线性, 自注意力只是简单地对value向量进行重新平均。
- **简单修复:** 在注意力的输出上添加一个前向层, 提供非线性的激活作用 (以及额外的表达能力)

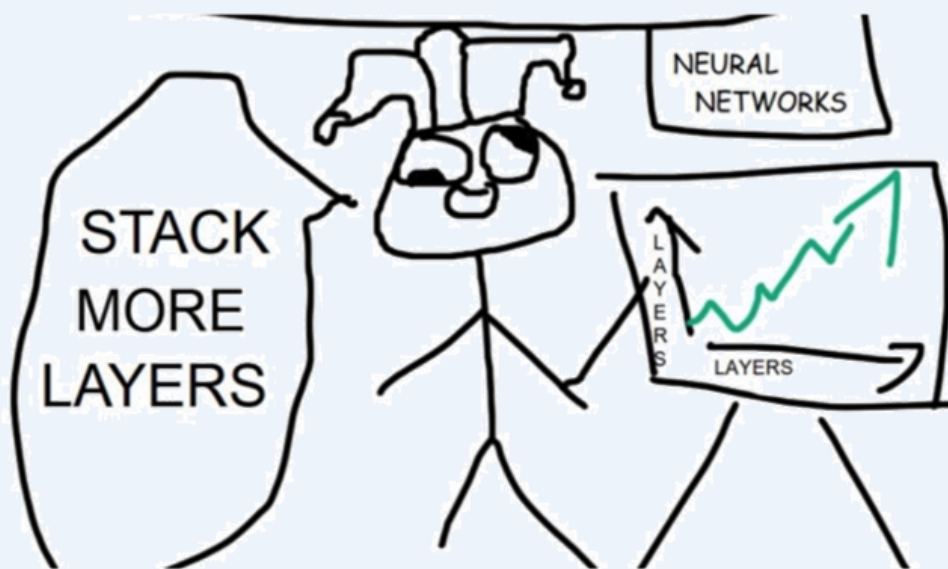
Equation for Feed Forward Layer

$$\begin{aligned} m_i &= \text{MLP}(\text{output}_i) \\ &= W_2 * \text{ReLU}(W_1 \times \text{output}_i + b_1) + b_2 \end{aligned}$$



Transformer: 训练Trick

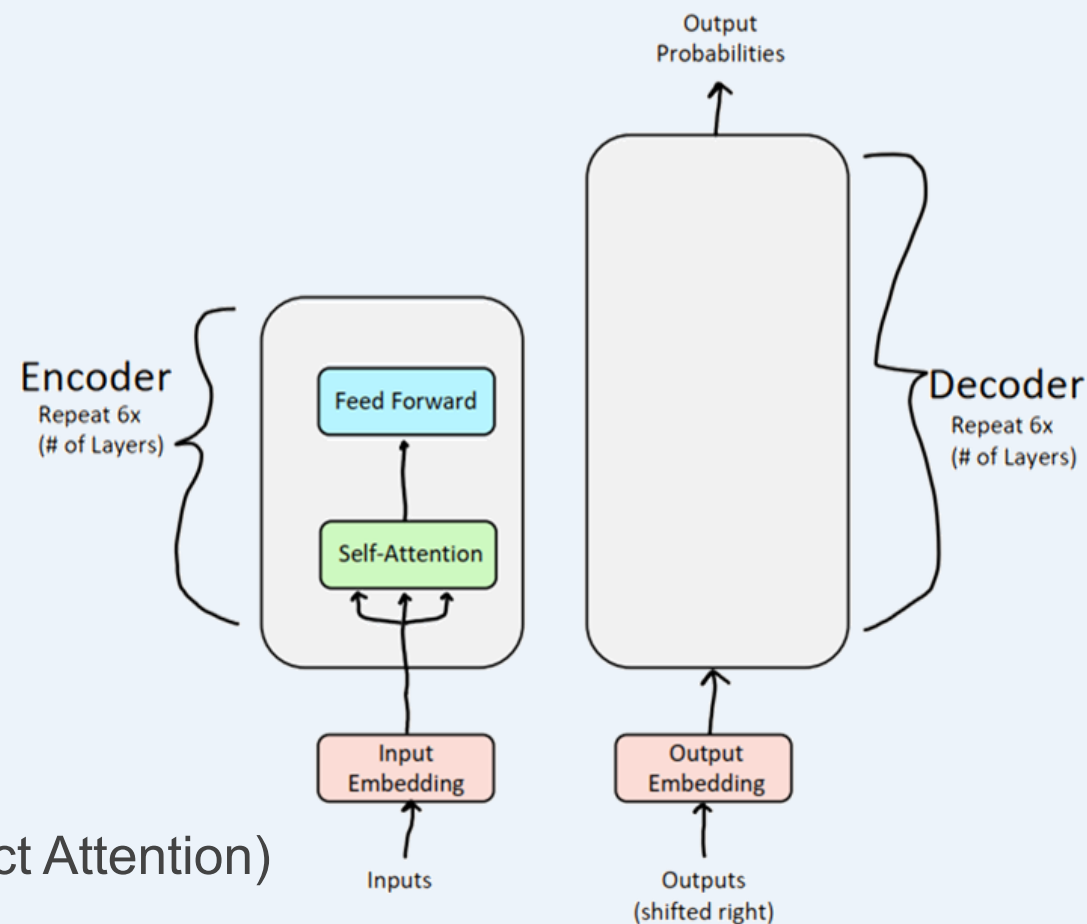
但怎么在深度的网络里面应用?



训练Trick #1: 残差连接 (Residual Connections)

训练Trick #2: 层归一化 (Layer Normalization)

训练Trick #3: 缩放点内积注意力 (Scaled Dot Product Attention)



Transformer: 训练Trick

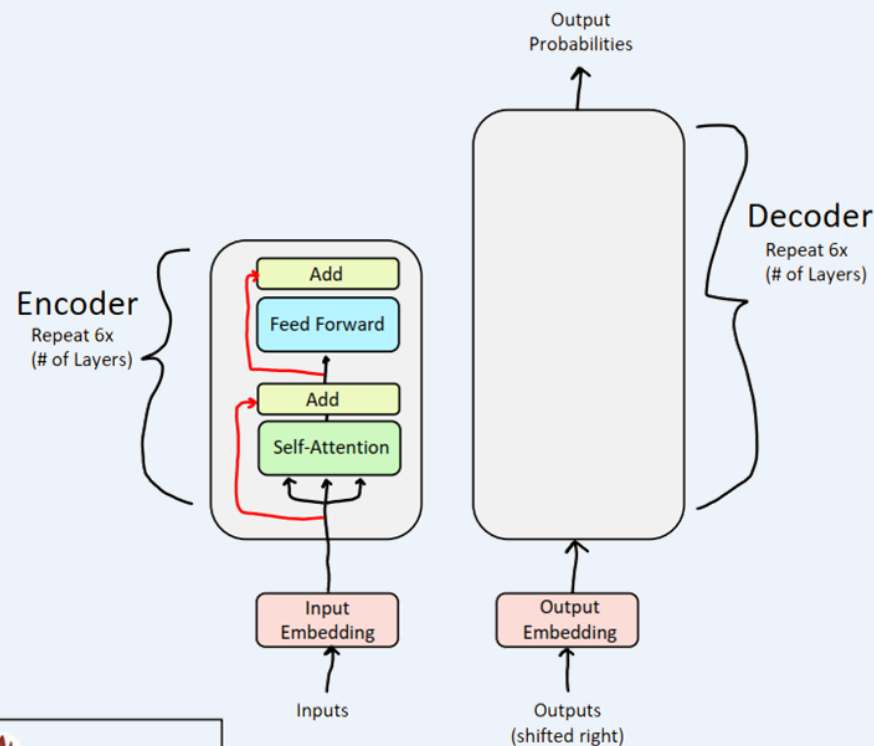
Trick #1: 残差连接 (Residual Connections)

- 残差连接是计算机视觉领域一个简单但强大的技术
- 深度网络其实在学习恒等函数上，出奇地差
- 因此，直接传播“未加工的”嵌入到下一层，会有很大的帮助

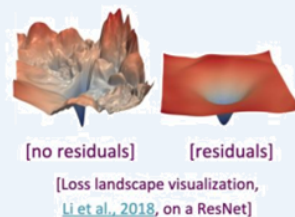
这能够避免网络“遗忘”或扭曲一些重要的信息，因为

$$x_l = F(x_{l-1}) + x_{l-1}$$

它被很多层处理



残差连接也被认为能够平滑损失地貌 (Loss Landscape), 使训练更加容易



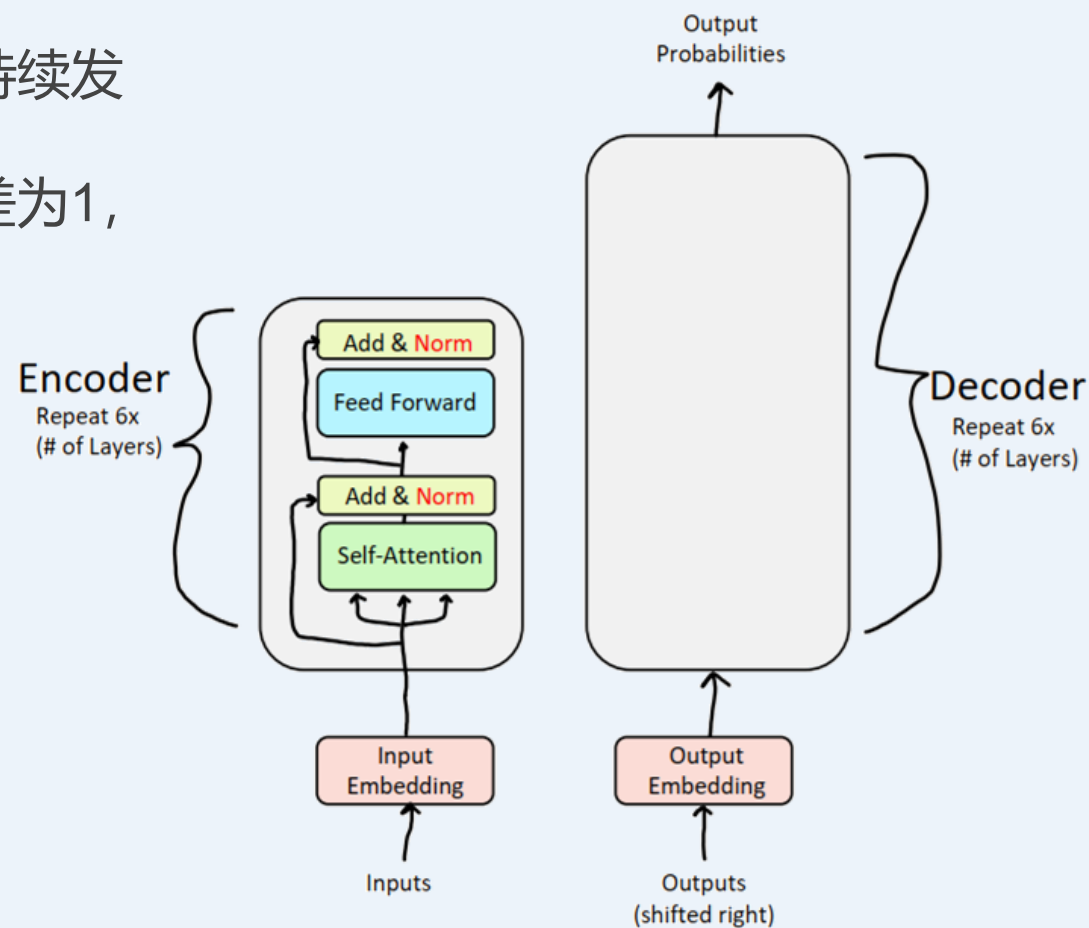
Transformer: 训练Trick

Trick #2: 层归一化(Layer Normalization)

- **问题**: 对于给定层, 由于它从更底层得到的输入持续发生变换, 参数难以训练
- **解决**: 在**每一层内**, 通过**归一化**到均值为和标准差为1, 减小方差

$$\text{Mean: } \mu^l = \frac{1}{H} \sum_{i=1}^H a_i^l \quad \text{Standard Deviation: } \sigma^l = \sqrt{\frac{1}{H} \sum_{i=1}^H (a_i^l - \mu^l)^2}$$

$$x^{\ell'} = \frac{x^{\ell} - \mu^{\ell}}{\sigma^{\ell} + \epsilon}$$



Transformer: 训练Trick

Trick #3: 缩放点积注意力(Scaled Dot Product Attention)

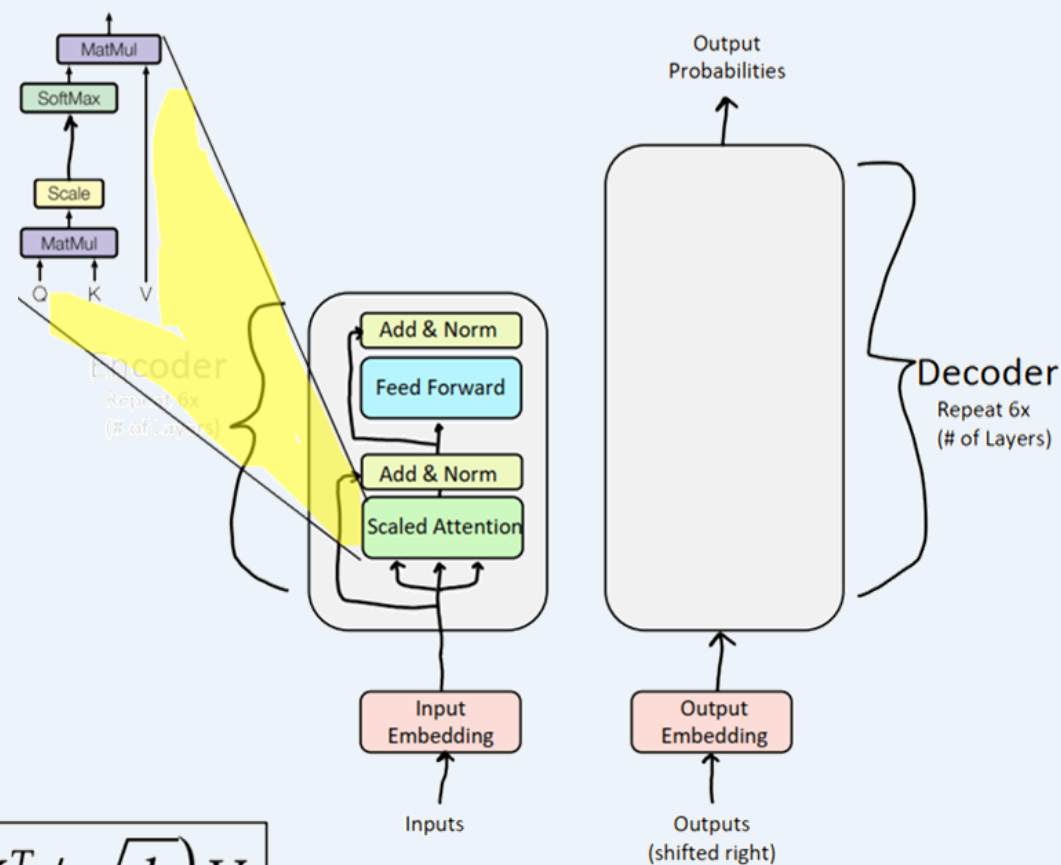
- 在层归一化之后, 向量元素的均值和方差分别为0和1
- 然而, 极端值的出现仍然会极大影响点积的结果, 因为它的方差随维度 d_k 缩放

回顾统计理论:

- 和的均值=均值的和= $d_k * 0 = 0$
- 和的方差=方差的和= $d_k * 1 = d_k$
- 为使方差为1, 只需要简单地除以 $\sqrt{d_k}$

更新之后的自注意力公式:

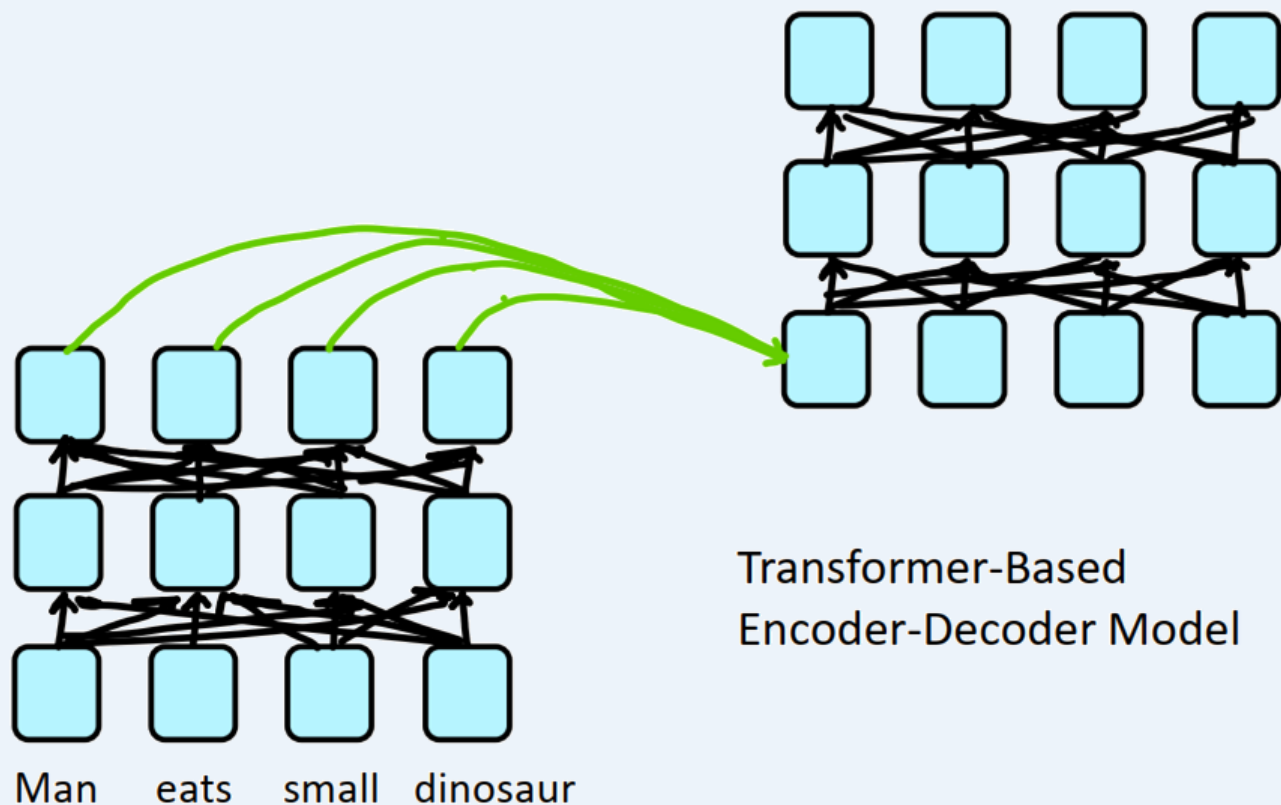
$$Output = softmax(QK^T / \sqrt{d_k}) V$$



Transformer: 主要问题

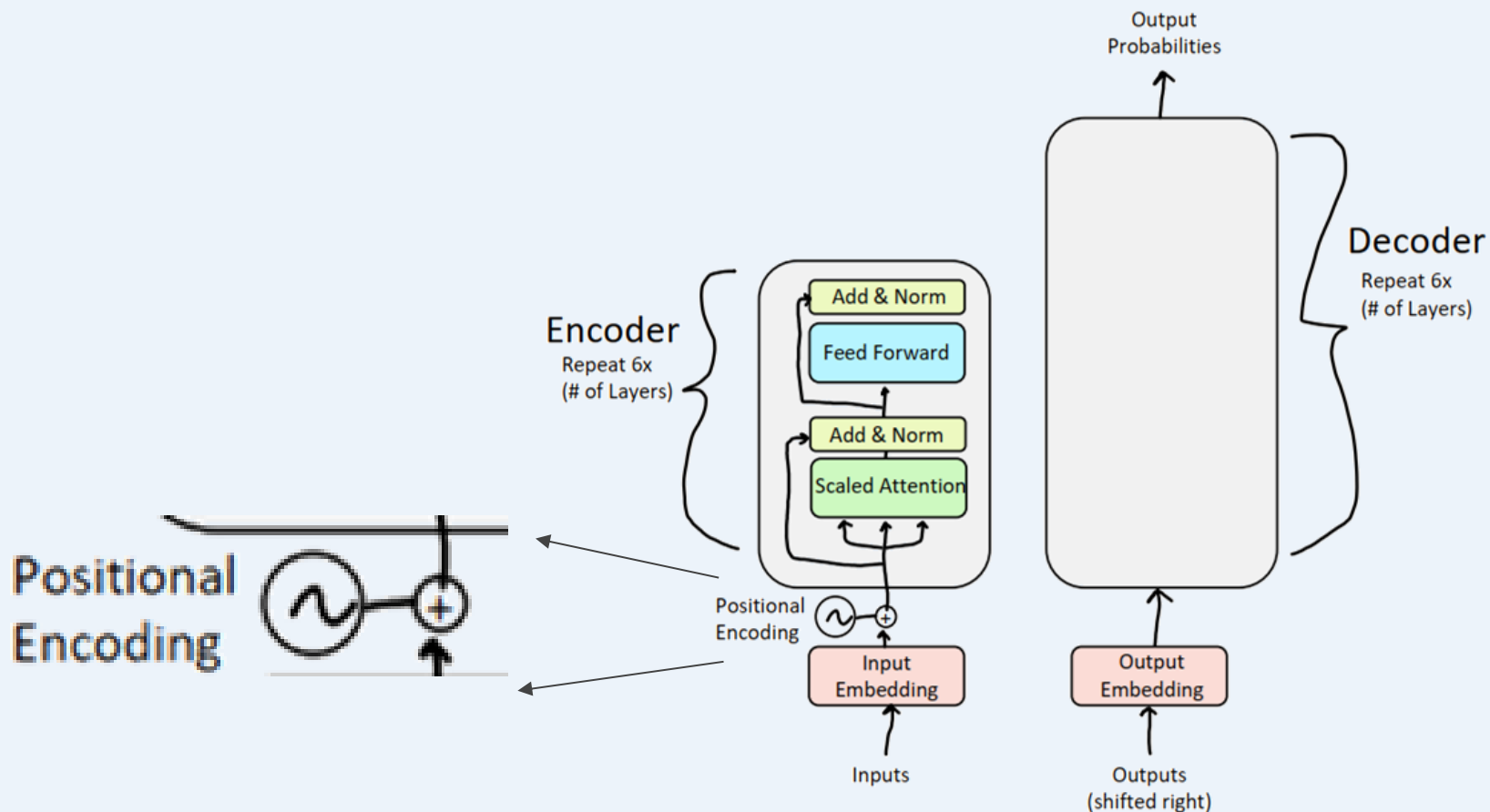
- 我们已经在编码器中应用了注意力，但**还有一个主要问题**
- 考虑下列句子：
 - “Man eats small dinosaur.”
- 词语的顺序似乎完全没有对网络产生影响
- 这是错误的，因为词序在许多语言(包括英语)中都有意义

$$Output = softmax(QK^T / \sqrt{d_k})V$$



Transformer: 主要问题

解决方法: 通过位置编码注入顺序信息



自注意力：位置表征

解决自注意力的第一个问题：序列的顺序

- 由于自注意力不会构建顺序信息，因此我们需要在 key、queries 和 values 中对句子的顺序进行编码。
- 考虑将每个**序列索引(sequence index)**表示成**向量(vector)**

$$p_i \in \mathbb{R}^d, \text{ for } i \in \{1, 2, \dots, T\} \text{ 为位置向量}$$

- 不需要关心 p_i 是由何而来
- 很容易就能将这种信息整合到自注意力块中：只需要把 p_i 加到输入中
- 令 $\tilde{v}_i$ 、 $\tilde{k}_i$ 、 $\tilde{q}_i$ 为旧的 values、keys、queries

$$v_i = \tilde{v}_i + p_i$$

$$q_i = \tilde{q}_i + p_i$$

$$k_i = \tilde{k}_i + p_i$$

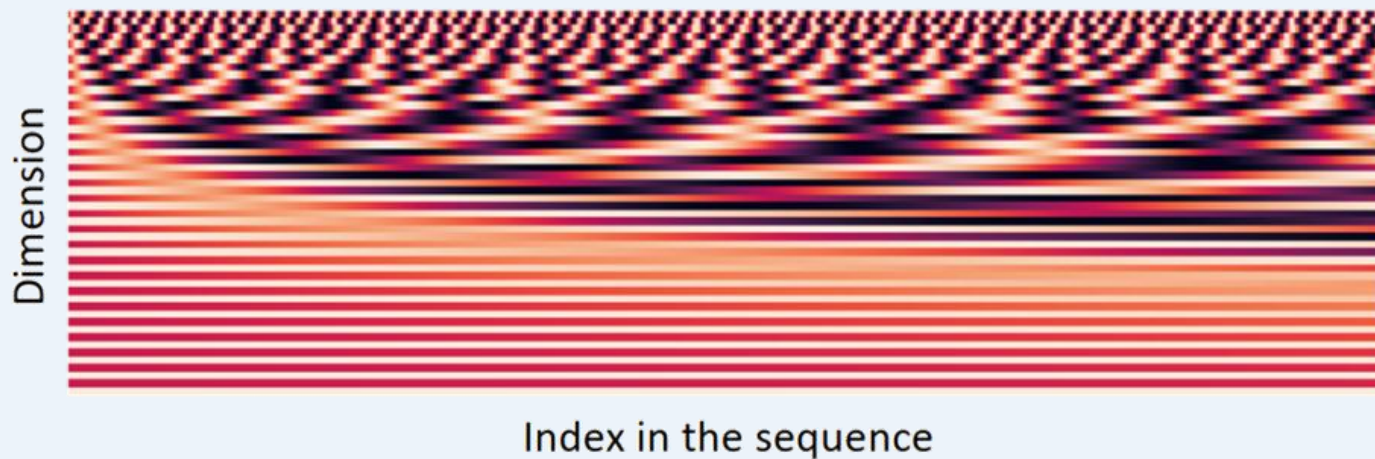
在深度自注意力网络里面，只需要在第一层这样做。此外，除了直接加之外，也可以采用拼接的方式

自注意力：位置表征

基于正弦曲线(sinusoids)的位置表征向量（原始）

- 正弦位置表征(Sinusoidal position representations): 连接不同周期的正弦函数:

$$p_i = \begin{pmatrix} \sin(i/10000^{2*1/d}) \\ \cos(i/10000^{2*1/d}) \\ \vdots \\ \sin(i/10000^{2*\frac{d}{2}/d}) \\ \cos(i/10000^{2*\frac{d}{2}/d}) \end{pmatrix}$$



- 优点:
 - 周期性表明 “绝对位置” 可能没有那么重要
 - 当周期重新开始时，或许可以外推到更长的序列
- 缺点:
 - 不可学习

扩展：相对位置编码+自注意力

关键：最突出的位置信息是单词之间的关系（例如，“cat”是“eat”之前的单词），而不是它们的绝对位置（例如，“cat”是单词 2）

Original Self-Attention Output:

$$z_i = \sum_{j=1}^n \alpha_{ij} (x_j W^V)$$

$$\text{where } \alpha_{ij} = \frac{\exp e_{ij}}{\sum_{k=1}^n \exp e_{ik}}$$

$$e_{ij} = \frac{(x_i W^Q)(x_j W^K)^T}{\sqrt{d_z}}$$

Relation-Aware Self-Attention Output:

$$z_i = \sum_{j=1}^n \alpha_{ij} (x_j W^V + a_{ij}^V)$$

$$\text{where } \alpha_{ij} = \frac{\exp e_{ij}}{\sum_{k=1}^n \exp e_{ik}}$$

$$e_{ij} = \frac{x_i W^Q (x_j W^K + a_{ij}^K)^T}{\sqrt{d_z}}$$

$$a_{ij}^K = w_{\text{clip}(j-i, k)}^K$$

$$a_{ij}^V = w_{\text{clip}(j-i, k)}^V$$

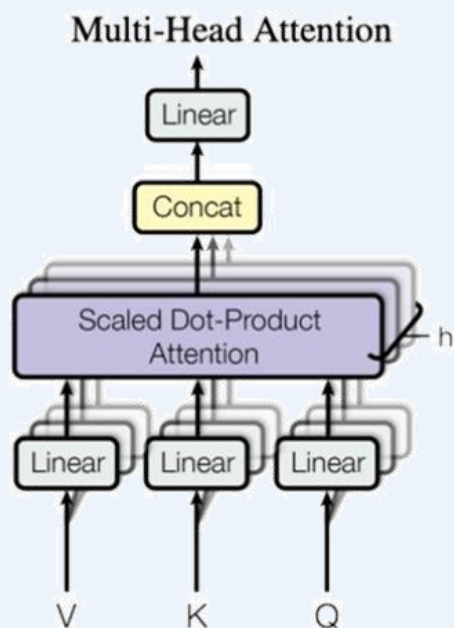
$$\text{clip}(x, k) = \max(-k, \min(k, x))$$

k	EN-DE BLEU
0	12.5
1	25.5
2	25.8
4	25.9
16	25.8
64	25.9
256	25.8

We then learn relative position representations
 $w^K = (w_{-k}^K, \dots, w_k^K)$ and $w^V = (w_{-k}^V, \dots, w_k^V)$

多头自注意力

思想：进行多次自注意力操作然后将它们的结果结合起来



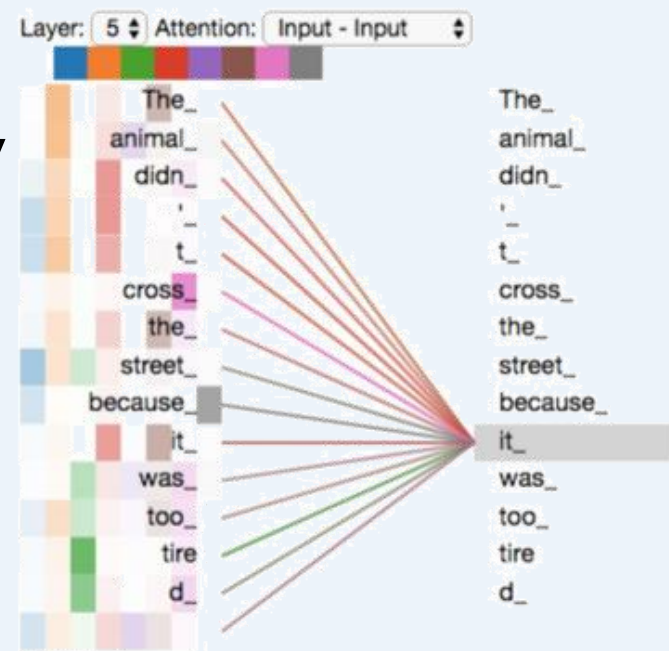
[Vaswani et al. 2017]



Wizards of the Coast, Artist: Todd Lockwood

Transformer编码器：多头自注意力

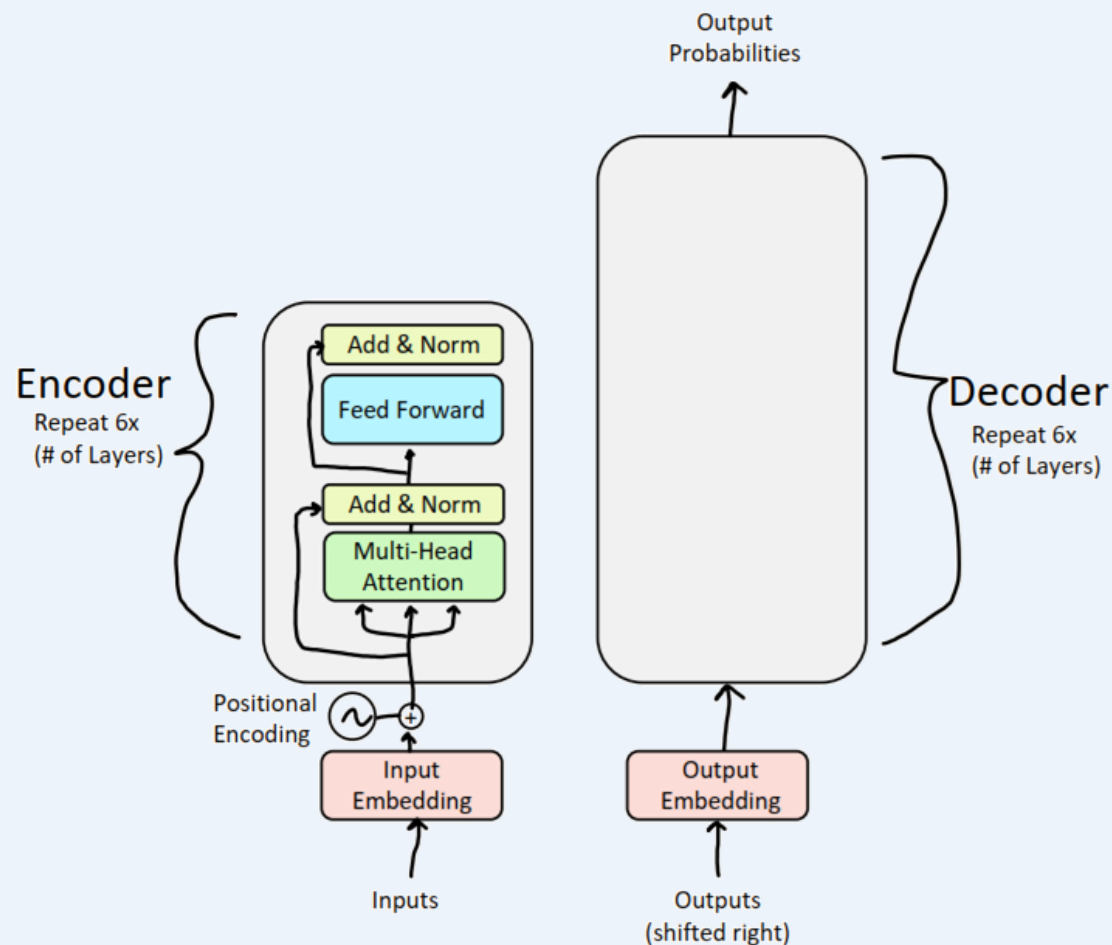
- 如果想要一次查看句子中的多个地方怎么办？
 - 对于单词 i ，自注意力“查看” $x_i^T Q^T K x_j$ 最高的地方，但如果因为不同的原因，想要查看不同的 j 呢？
- 通过构建多个 Q 、 K 、 V 矩阵，设计**多个自注意力“头”**
- 令 $Q_\ell, K_\ell, V_\ell \in \mathbb{R}^{d \times d/h}$ ，其中 h 为注意力头的个数， ℓ 的范围1到 h
- 每个注意力头独立地进行注意力操作：
 - $output_\ell = \text{softmax}(X Q_\ell K_\ell^T X^T * X V_\ell)$ ，其中 $output_\ell \in \mathbb{R}^{d/h}$
- 然后组合所有头的输出
 - $output = Y[output_1; \dots; output_h]$ ，其中 $Y \in \mathbb{R}^{d \times d}$
- 每个头“查看”不同的东西，并构造不同的value向量



Credit to <https://jalammar.github.io/illustrated-transformer/>

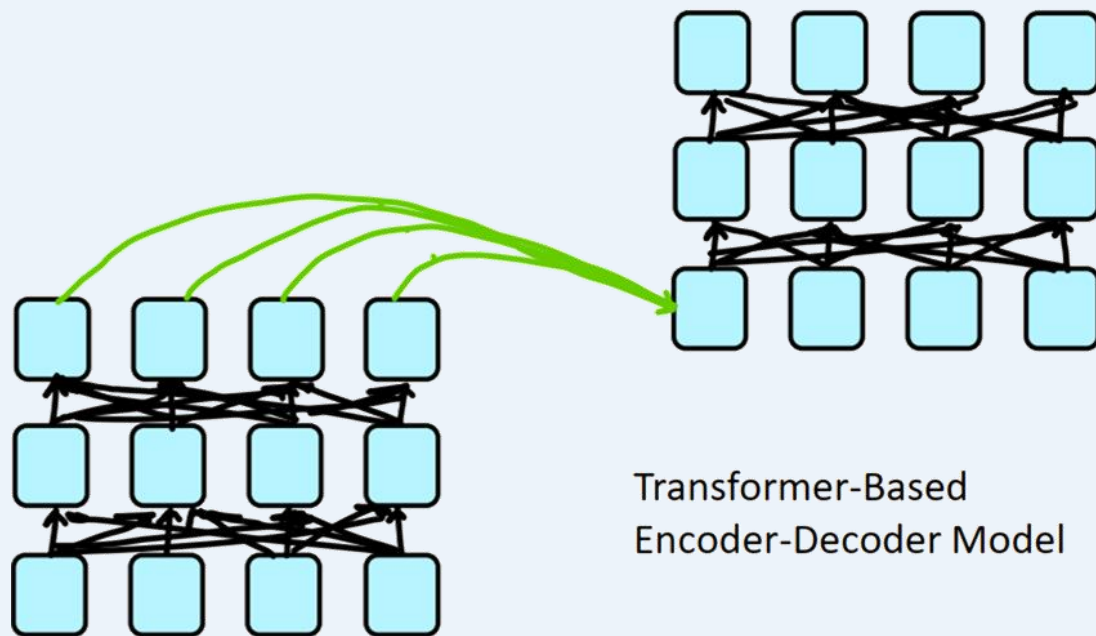
Transformer编码器：多头自注意力

至此，编码器完成，对于解码器：

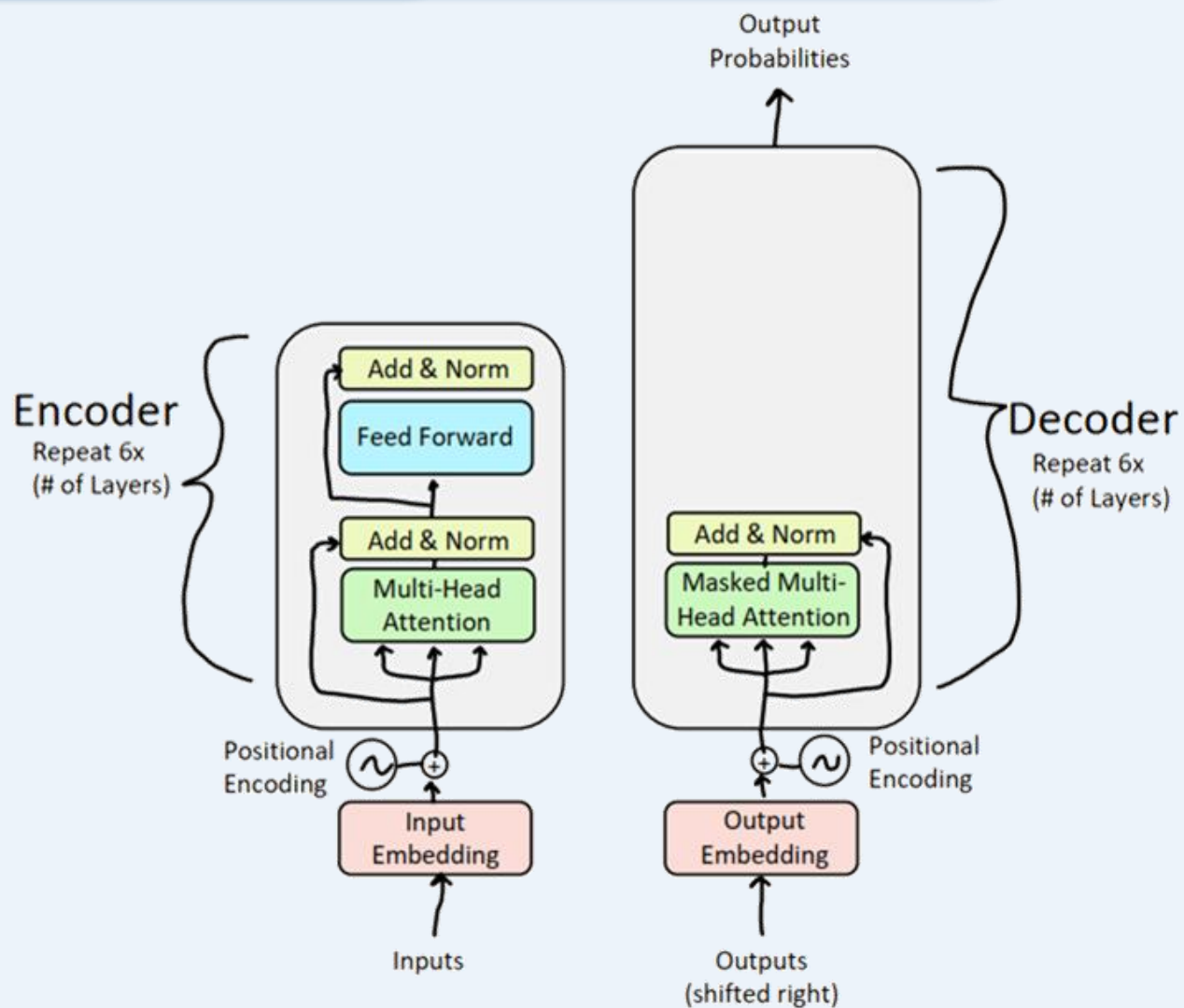


Transformer解码器：掩码多头自注意力

- **问题：**如何防止解码器训练中“作弊”？如果我们有一个语言建模目标，那么网络就不能向前看并“看到”答案吗？
- **解决：**掩码多头自注意力。概括地说，我们从模型中隐藏（屏蔽）有关未来词元(Token)的信息。



Transformer解码器：掩码多头自注意力

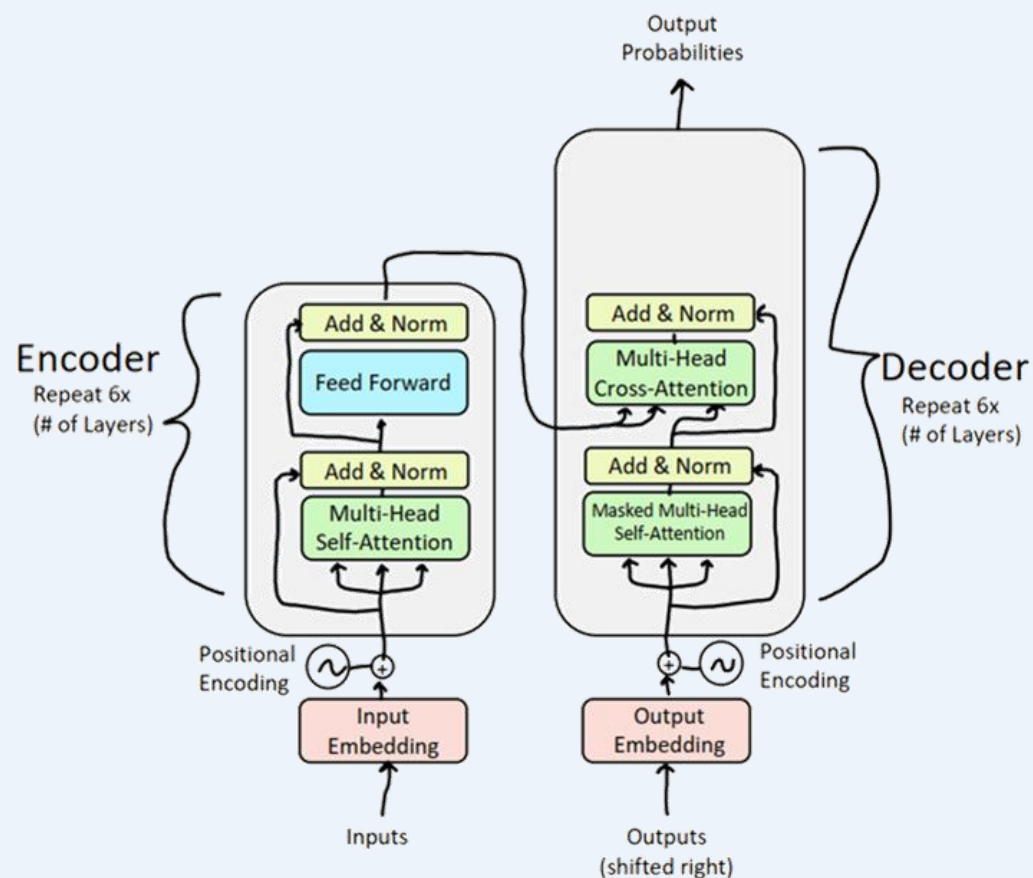


编码器-解码器注意力

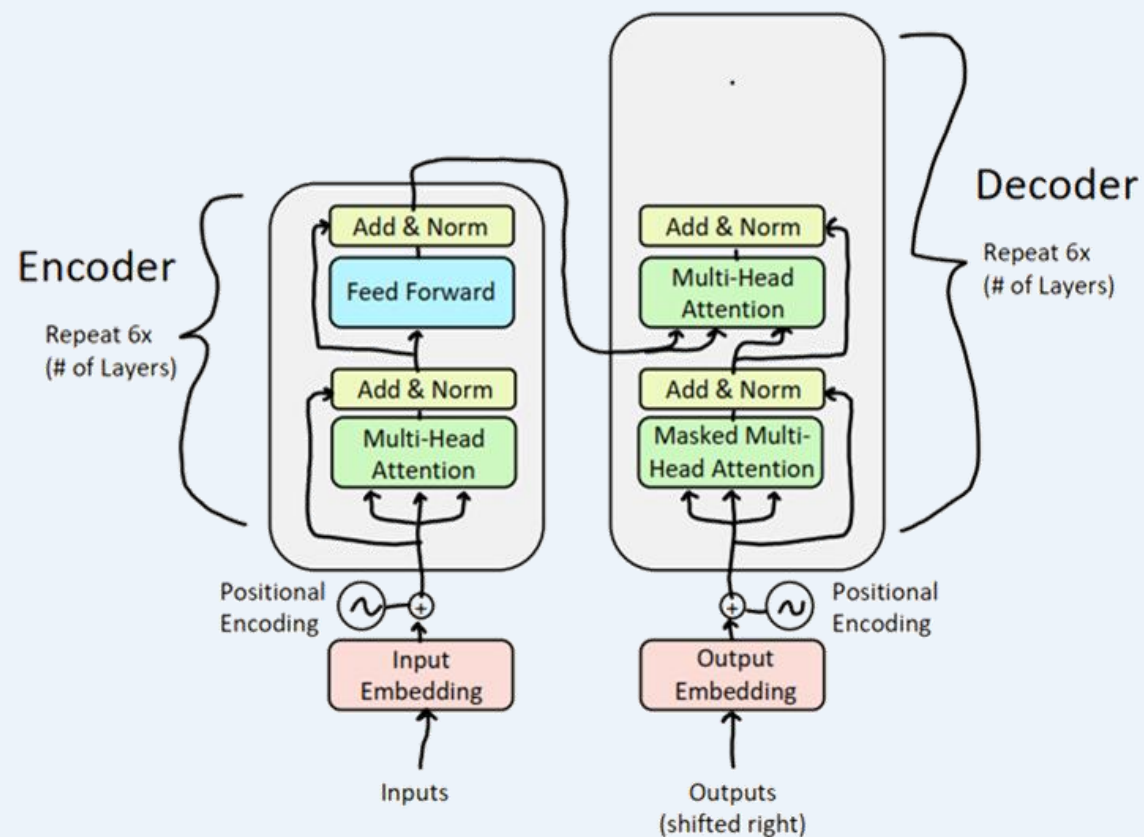
- 可以看到自注意力即key、query、value均有相同的来源
- 在解码器中，注意力看起来更像我们之前看过的
- 令 $h_1, \dots, h_T$ 为Transformer**编码器**的输出向量, $h_i \in \mathbb{R}^d$
- 令 $z_1, \dots, z_T$ 为Transformer**解码器**的输入向量, $z_i \in \mathbb{R}^d$
- 那么，key和value则由**编码器**取得（像一段记忆一样）：

$$k_i = Kh_i \quad v_i = Vh_i$$

- 并且query从**解码器**取得，有： $q_i = Qz_i$

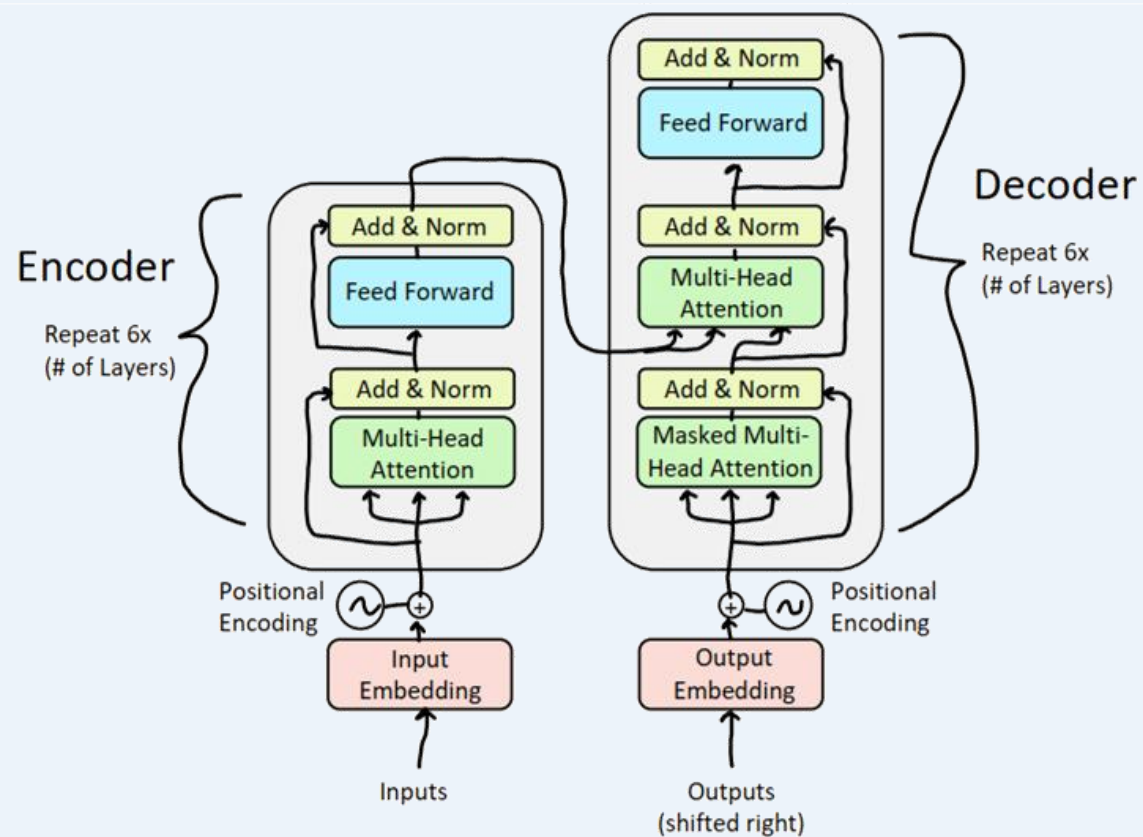


解码器：收尾



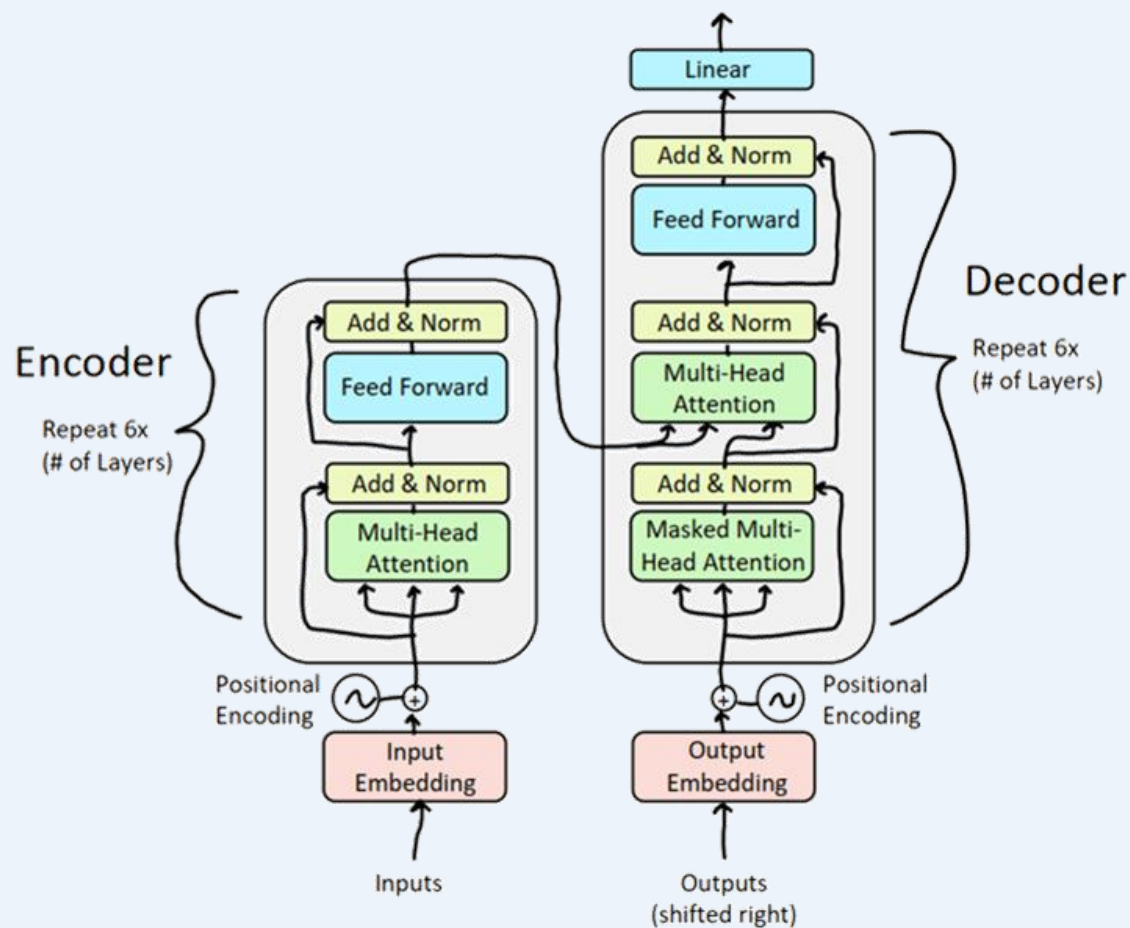
解码器：收尾

- 添加一个前馈层 (使用残差连接以及层归一化)



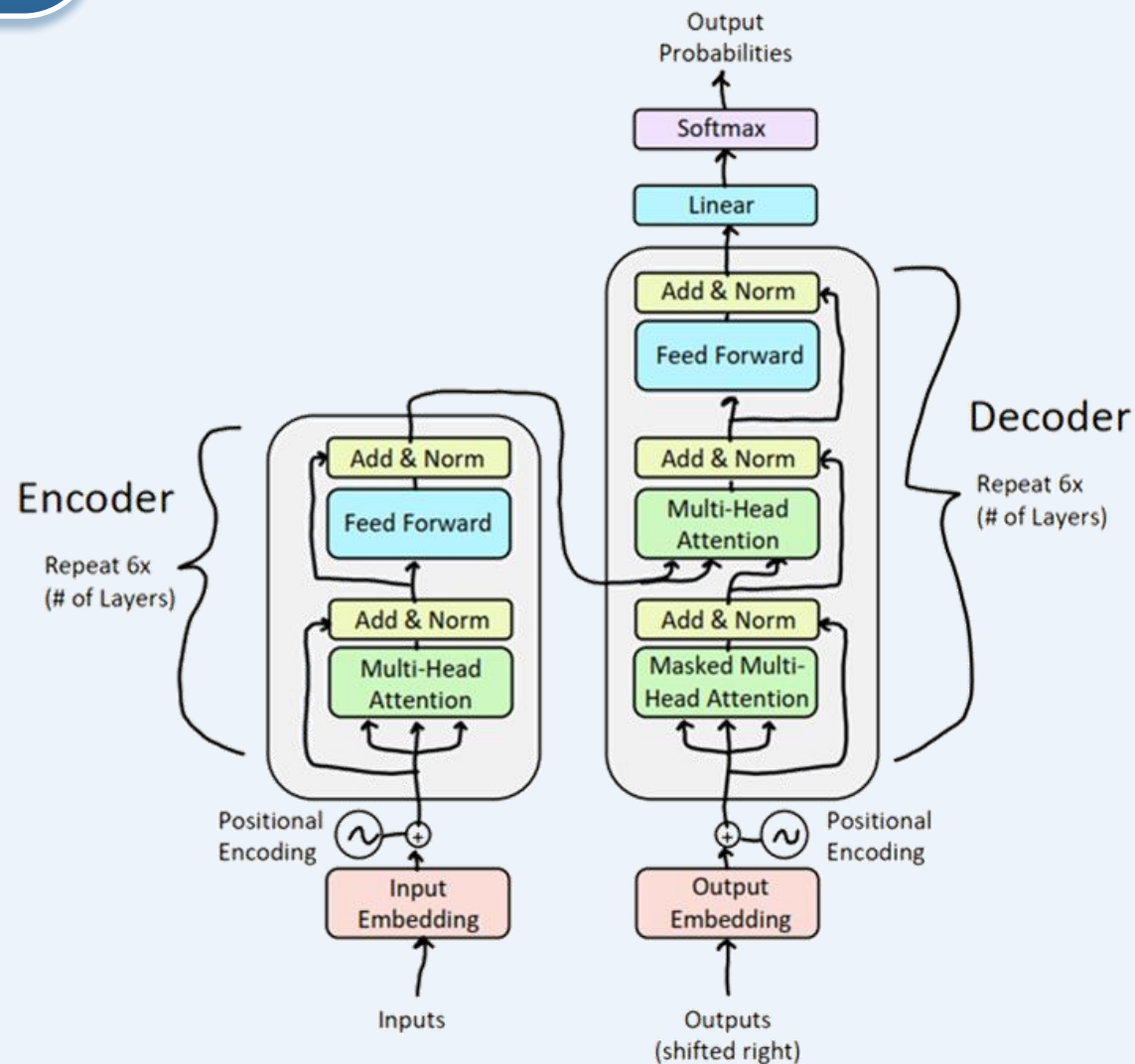
解码器：收尾

- 添加一个前馈层 (使用残差连接以及层归一化)
- 添加最终线性层, 将嵌入投影到一个比词汇量大小更长的向量中 (对数概率)

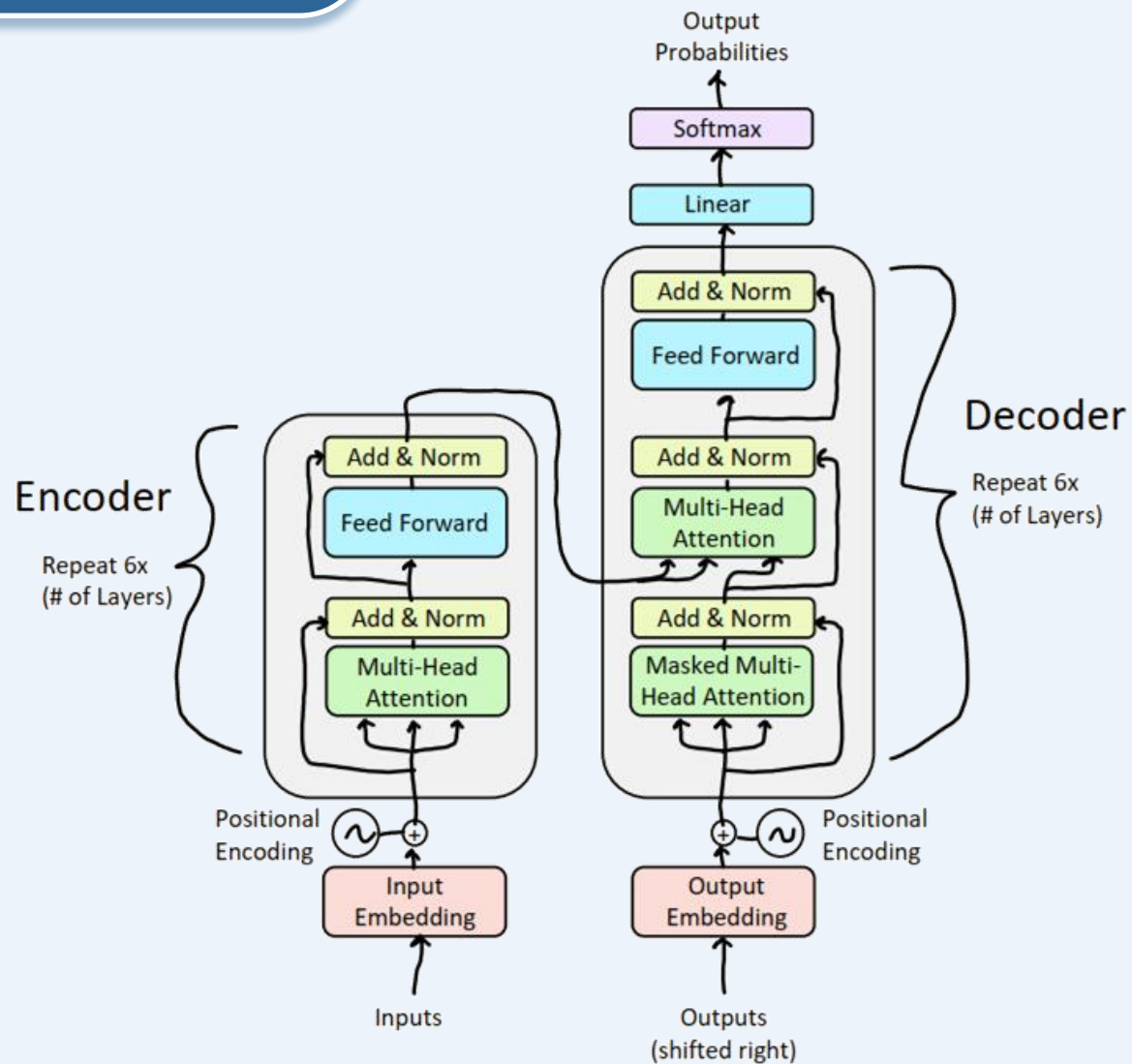


解码器：收尾

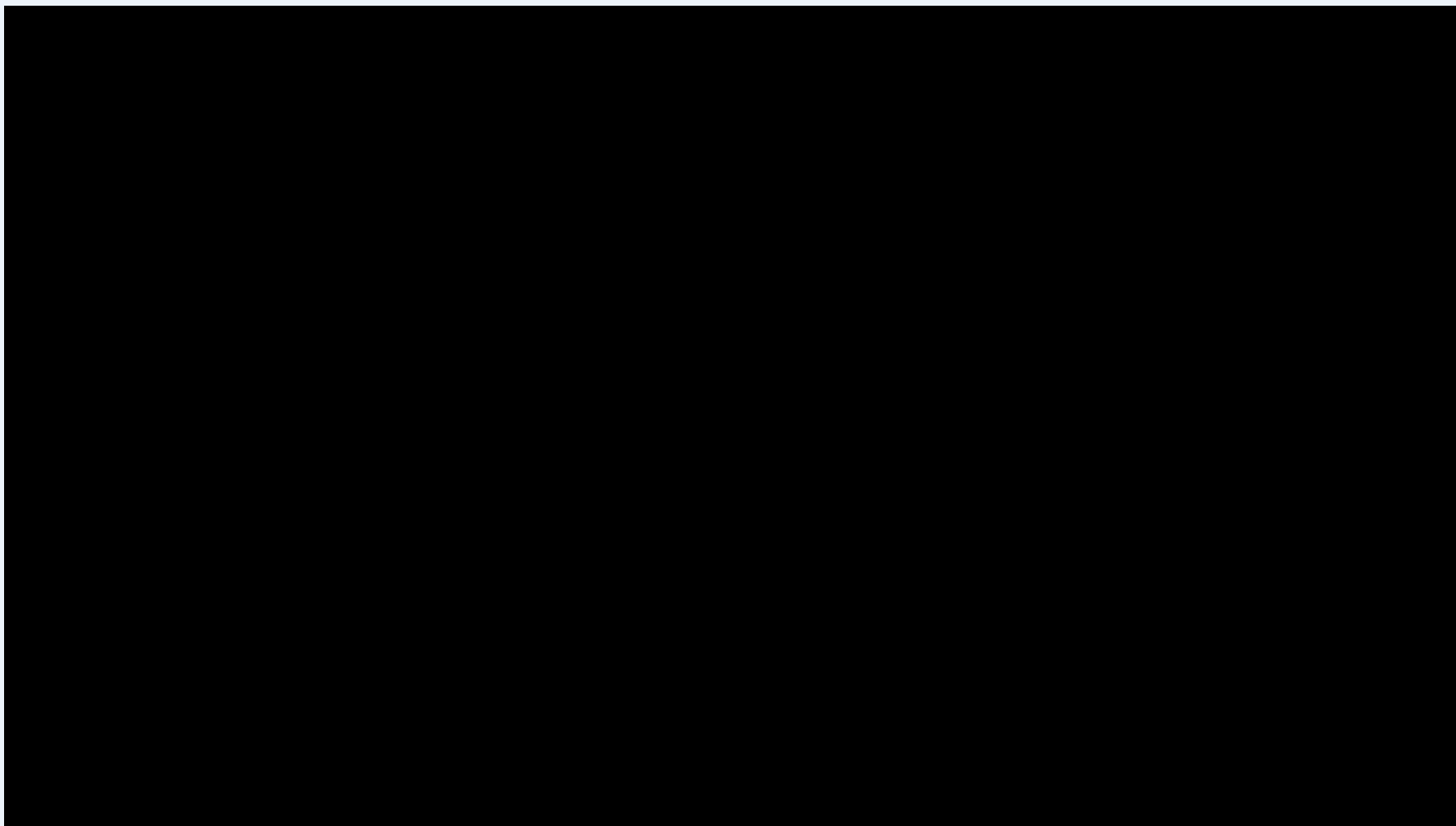
- 添加一个前馈层 (使用残差连接以及层归一化)
- 添加最终线性层, 将嵌入投影到一个比词汇量大小更长的向量中 (对数概率)
- 添加Softmax层, 生成下一个可能单词的概率分布



Transformer完整架构



Transformer in Vision



谢谢大家！

相关课程资源及参考文献请浏览

超算习堂: <https://easyhpc.net/course/143>